



UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica y Automática Industrial

TRABAJO FIN DE GRADO

Banco de ensayos para una caracola de radio variable: control del servoaccionamiento, adquisición de fuerza y validación del modelo

Javier López [**COMPLETAR:** segundo apellido]

Cotutor (si lo hay):
[**COMPLETAR:** nombre y apellidos]
Departamento:
[**COMPLETAR:** departamento]

Tutor: [**COMPLETAR:** nombre y apellidos]
Departamento:
[**COMPLETAR:** departamento]

Madrid, [**COMPLETAR:** mes], 2026





UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica y Automática Industrial

TRABAJO FIN DE GRADO

Banco de ensayos para una caracola de radio variable

Firma del autor: Javier López [**COMPLETAR:** segundo apellido]

Firma del cotutor (si lo hay) *Firma del tutor*

Madrid, [**COMPLETAR:** mes], 2026

Copyright © 2026. Javier López [**COMPLETAR:** segundo apellido]
Esta obra está licenciada bajo la licencia Creative Commons Atribución-NoComercial-SinDerivadas 3.0 Unported (CC BY-NC-ND 3.0). Para ver una copia de esta licencia, visite `http://creativecommons.org/licenses/by-nc-nd/3.0/deed.es` o envíe una carta a Creative Commons, 444 Castro Street, Suite 900, Mountain View, California, 94041, EE.UU.
[**COMPLETAR:** confirmar la licencia que se quiere aplicar; ésta es la que propone la plantilla]
Todas las opiniones aquí expresadas son del autor, y no reflejan necesariamente las opiniones de la Universidad Politécnica de Madrid.

Título: Banco de ensayos para una caracola de radio variable: control del servoaccionamiento, adquisición de fuerza y validación del modelo
Autor: Javier López **[COMPLETAR:** segundo apellido]
Tutor: **[COMPLETAR:** nombre del tutor]
Cotutor: **[COMPLETAR:** nombre del cotutor]

EL TRIBUNAL

Presidente:

Vocal:

Secretario:

Realizado el acto de defensa y lectura del Trabajo Fin de Grado el día ..... de ..... de ... en ....., en la Escuela Técnica Superior de Ingeniería y Diseño Industrial de la Universidad Politécnica de Madrid, acuerda otorgarle la CALIFICACIÓN de:

VOCAL

SECRETARIO PRESIDENTE

Agradecimientos

[**COMPLETAR:** Agradecimientos: tutoría, departamento o laboratorio que ha prestado el material, compañeros y familia.]

Resumen

Este Trabajo Fin de Grado desarrolla el banco de ensayos de un mecanismo de elevación basado en un tambor de radio variable —aquí llamado *caracola*— que, accionado por un servomotor AC de lazo cerrado, enrolla un cable y levanta una barra articulada con una masa de 2 kg. La idea que se quiere contrastar es que, modulando el radio con que trabaja el cable, el par que ve el motor deje de seguir a la tensión y se reparta de forma más uniforme a lo largo del recorrido.

Para poder contrastarla se ha construido una plataforma completa: una placa adaptadora diseñada en KiCad alrededor de una Raspberry Pi Pico 2, con bus RS-485, entradas digitales protegidas, entradas analógicas acondicionadas, salidas de potencia y dos zócalos mikroBUS; un firmware en C++ que gobierna el accionamiento por Modbus RTU, lee la célula de carga a través de un acondicionador ZSC31014, hace el *homing* con dos finales de carrera y ejecuta ciclos automáticos B→A→B; un protocolo binario propio con CRC-8; y una aplicación web (FastAPI y WebSocket) con tres páginas: monitor en tiempo real, configuración del mecanismo con simulador, y visor de ensayos que superpone la curva teórica a cada medida.

El banco muestrea a 20,0 Hz con una dispersión del periodo de 0,06 ms y graba cada ensayo en cuentas brutas. El modelo del mecanismo se ha reformulado: el cable no sale de un punto fijo sino *tangente* al tambor, y el contacto se resuelve sobre el perfil real, lo que obliga a integrar la trayectoria paso a paso. Su consistencia se comprueba contra el balance de energía, que se cumple en la cuarta cifra significativa sin ajustar ningún parámetro.

Sobre los diez ensayos de la campaña decisiva —consecutivos y repetibles al 0,6 %—, el resultado es que **el par del tambor es prácticamente plano**: varía un −5 % a lo largo del tramo con carga, con un rizado del 12,1 %, mientras que un tambor cilíndrico equivalente lo haría *crecer* un 15 %. El mecanismo, por tanto, hace lo que se diseñó que hiciera, y se mide. El perfil que explica la medida no es la espiral tal y como está descrita en la configuración —que sobrecompensa—, sino esa misma espiral precedida de su arco de radio constante: ajustando la longitud de ese arco como único parámetro se obtiene $R^2 = 0{,}84$, y el valor ajustado (61°) coincide con el que la pieza tiene medido en el modelo CAD (50°).

El trabajo arroja además un resultado de diseño que contradice la expectativa de partida. El criterio correcto es el par de pico, porque es lo que dimensiona el accionamiento; y como todos los tambores que recorren lo mismo en las mismas vueltas entregan el mismo par medio, el ahorro se reduce al cociente de sus relaciones pico/media. Medido, sale entre el −1,5 % y el 4 %: en este banco el tambor de radio variable *no ahorra motor*. La razón no es que el principio falle, sino que aquí la tensión apenas varía —un 13 % según el modelo, un 28 % según la célula— porque el brazo del peso y el del cable crecen a la vez y se cancelan: no hay nada que repartir. Donde la tensión variase el doble o el triple, el mismo tambor recortaría el motor en esa proporción. Perseguir además un perfil de par exactamente constante choca con una restricción física: el cable sólo se asienta sobre perfiles convexos y de pendiente suave.

Se entrega, por tanto, un banco de ensayos reproducible, un modelo verificado y un procedimiento de contraste, junto con un diagnóstico de lo que falta para cerrar la validación: calibrar la célula con masas patrón, medir el ángulo de reposo libre de la barra y levantar el perfil real del tambor.

Palabras clave: tambor de radio variable, caracola, compensación de par, servomotor AC, Modbus RTU, RS-485, célula de carga, ZSC31014, Raspberry Pi Pico 2, adquisición de datos.

Abstract

This Bachelor's Thesis develops the test bench for a lifting mechanism based on a variable-radius drum —here called *caracola*— driven by a closed-loop AC servo motor, which winds a cable and lifts a hinged bar carrying a 2 kg mass. The idea under test is that, by modulating the radius at which the cable works, the motor torque stops following the cable tension and is spread more evenly along the stroke.

A complete platform was built to test it: a KiCad-designed adapter board around a Raspberry Pi Pico 2, with an RS-485 bus, protected digital inputs, conditioned analogue inputs, high-side outputs and two mikroBUS sockets; C++ firmware that commands the drive over Modbus RTU, reads the load cell through a ZSC31014 conditioner, performs homing with two limit switches and runs automatic B→A→B cycles; a custom binary protocol with CRC-8; and a web application (FastAPI and WebSocket) with three pages: real-time monitor, mechanism configuration with a simulator, and a test viewer that overlays the theoretical curve on each measurement.

The bench samples at 20,0 Hz with a period spread of 0,06 ms and logs every test in raw counts. The mechanism model was reformulated: the cable does not leave a fixed point but is *tangent* to the drum, and contact is solved against the actual profile, which requires integrating the trajectory step by step. Its consistency is checked against the energy balance, satisfied to the fourth significant figure without fitting any parameter.

Across the ten tests of the decisive campaign —consecutive and repeatable to 0,6 %—, the result is that **the drum torque is essentially flat**: it varies by −5 % over the loaded stroke, with a ripple of 12,1 %, whereas an equivalent cylindrical drum would make it *rise* by 15 %. The mechanism therefore does what it was designed to do, and this is measurable. The profile that explains the measurement is not the spiral as described in the configuration —which overcompensates— but that same spiral preceded by its constant-radius arc: fitting the length of that arc as the only additional parameter yields $R^2 = 0.84$, and the fitted value (61°) agrees with the one measured on the CAD model of the part (50°).

The work also yields a design result that contradicts the initial expectation. The right figure of merit is peak torque, since that is what sizes the drive; and because every drum covering the same stroke in the same number of turns delivers the same mean torque, the saving reduces to the ratio of their peak-to-mean values. Measured, it falls between −1,5 % and 4 %: on this bench the variable-radius drum *does not save motor*. The reason is not that the principle fails, but that here the tension barely varies —13 % per the model, 28 % per the load cell— because the weight lever arm and the cable lever arm grow together and cancel: there is nothing to redistribute. Where the tension varied twice or three times as much, the same drum would cut the motor by that proportion. Pursuing an exactly constant-torque profile also conflicts with a physical constraint: a cable only seats on convex profiles of gentle slope.

The thesis therefore delivers a reproducible test bench, a verified model and a contrast procedure, together with a diagnosis of what remains in order to close the validation: calibrating the load cell with reference masses, measuring the free rest angle of the bar, and surveying the actual drum profile.

Keywords: variable radius drum, torque compensation, AC servo drive, Modbus RTU, RS-485, load cell, ZSC31014, Raspberry Pi Pico 2, data acquisition.

Índice general

Agradecimientos vii

Resumen ix

Abstract xi

Índice xvii

Acrónimos y símbolos xxiii

1. Introducción **1**
1.1. Motivación del proyecto . . . 1
1.2. Objetivos . . . 1
1.3. Materiales utilizados . . . 2
1.4. Alcance y limitaciones . . . 2
1.5. Metodología . . . 2
1.6. Estructura del documento . . . 3

2. Marco teórico **5**
2.1. Transmisión por cable y radio variable . . . 5
2.2. Accionamiento: servomotor AC de lazo cerrado . . . 5
2.3. Medida de fuerza con células de carga . . . 6
2.3.1. Galgas extensométricas y puente de Wheatstone . . . 6
2.3.2. Acondicionadores integrados . . . 6
2.4. Microcontrolador . . . 6
2.5. Buses de comunicación . . . 7
2.5.1. RS-485 y Modbus RTU . . . 7
2.5.2. I^2C y mikroBUS . . . 7
2.6. Adquisición y supervisión en el ordenador . . . 7

3. Estado del arte **9**
3.1. Antecedentes: variar el radio para repartir el par . . . 9
3.2. Compensación de gravedad con poleas no circulares . . . 9
3.3. Lo que el cable impone al perfil . . . 10
3.4. Transmisiones por cable: el *capstan drive* . . . 10
3.5. Cuerda sintética frente a cable de acero . . . 11
3.6. Bancos de ensayo y validación experimental . . . 11
3.7. Conclusiones del estado del arte . . . 11

4. Modelo del mecanismo **13**
4.1. Descripción del mecanismo . . . 13
4.2. Los dos ángulos límite del mecanismo . . . 14
4.3. Tensión del cable . . . 15
4.3.1. El cable equivalente . . . 15
4.3.2. Contacto exacto con el perfil . . . 15

4.3.3. Comprobación: balance de energía . . . 16
4.4. Parámetros del mecanismo . . . 16
4.5. Qué hace realmente este mecanismo . . . 16
4.5.1. La tensión no baja: sube . . . 16
4.5.2. Por qué el par de pico, y por qué a igual número de vueltas . . . 16
4.5.3. Comparación de perfiles a igual recorrido . . . 17
4.5.4. Por qué no se persigue una leva de par constante . . . 18
4.6. Método experimental: separación del par y brazo efectivo . . . 18
4.6.1. Por qué no basta con medir la fuerza . . . 18
4.7. Limitaciones del modelo . . . 19

5. Requisitos y arquitectura del banco **21**
5.1. Requisitos . . . 21
5.1.1. Requisitos funcionales . . . 21
5.1.2. Requisitos no funcionales . . . 21
5.2. Arquitectura general . . . 22
5.3. Flujo de datos y temporización . . . 23
5.4. Decisiones de diseño . . . 23
5.5. Seguridad . . . 24

6. Diseño hardware **27**
6.1. Placa adaptadora: objetivos . . . 27
6.2. Revisiones del diseño . . . 27
6.3. Microcontrolador . . . 28
6.4. Interfaz RS-485 . . . 28
6.5. Entradas digitales . . . 29
6.6. Entradas analógicas . . . 29
6.7. Salidas de potencia . . . 29
6.8. Célula de carga . . . 29
6.9. Alimentación . . . 30
6.10. Placa de circuito impreso . . . 30
6.11. Validación de la placa . . . 30

7. Firmware del microcontrolador **33**
7.1. Entorno y organización . . . 33
7.2. Lazo principal . . . 33
7.3. Máquina de estados del servo . . . 34
7.4. Lectura de la célula de carga . . . 34
7.4.1. Árbitro del bus I^2C . . . 35
7.5. Posición integrada . . . 35
7.6. *Homing* . . . 35
7.7. Ciclo de ensayo B→A→B . . . 36
7.8. Configuración del acondicionador de la célula . . . 37
7.8.1. Escritura verificada . . . 37
7.8.2. Mantenimiento del reinicio para medida . . . 37
7.9. Evolución del firmware . . . 37

8. Comunicaciones **39**
8.1. Protocolo binario entre el microcontrolador y el ordenador . . . 39
8.1.1. Formato de trama . . . 39
8.1.2. Recepción y resincronización . . . 39
8.1.3. Paquetes del microcontrolador al ordenador . . . 40
8.1.4. Órdenes del ordenador al microcontrolador . . . 40
8.2. Modbus RTU con el accionamiento . . . 41
8.2.1. Registros utilizados . . . 41
8.2.2. Secuencia de configuración . . . 42
8.2.3. Transacciones por periodo . . . 42

8.3. Interfaz entre el servidor y el navegador . . . 42
8.3.1. WebSocket . . . 42
8.3.2. Servicios HTTP . . . 43

9. Software de supervisión y análisis 45
9.1. Visión general . . . 45
9.2. Servidor . . . 45
9.2.1. Estructura concurrente . . . 45
9.2.2. Configuración del mecanismo . . . 46
9.2.3. Grabación de ensayos . . . 46
9.3. Modelo compartido . . . 46
9.4. Monitor en tiempo real . . . 46
9.5. Configuración del mecanismo y simulador . . . 47
9.6. Visor de ensayos . . . 47
9.7. Herramientas auxiliares . . . 48
9.8. Interfaz de usuario . . . 48
9.9. Observaciones de calidad . . . 48

10. Problemas técnicos y soluciones 49
10.1. UART sobre PIO y latencia del enlace Modbus . . . 49
10.2. Configuración del ZSC31014: la ventana del modo comando . . . 49
10.2.1. Primer obstáculo: provocar el reinicio . . . 50
10.2.2. Segundo obstáculo: alimentación parásita por el bus . . . 50
10.2.3. Tercer obstáculo: acertar la ventana . . . 50
10.2.4. Cuarto obstáculo: integridad de la EEPROM . . . 50
10.3. Lecturas de la célula siempre “antiguas” . . . 50
10.4. Depuración del ciclo de ensayo . . . 50
10.5. El modelo que no cerraba: la salida del cable desplazada . . . 51
10.6. Lo que la fuerza no puede decidir . . . 52
10.7. Tramo inicial sin carga y calibraciones incoherentes . . . 52
10.8. Segunda célula de carga en el mismo bus . . . 52
10.9. Discrepancias con la hoja de datos . . . 52
10.10. Otros problemas . . . 52
10.11. Lecciones aprendidas . . . 53

11. Resultados 55
11.1. Campañas de ensayos . . . 55
11.2. Temporización del lazo de control . . . 55
11.3. Ensayo tipo . . . 56
11.4. Repetibilidad . . . 56
11.5. Fuerza y par a lo largo del recorrido . . . 56
11.5.1. La escala del accionamiento . . . 56
11.6. Contraste con el modelo: qué perfil explica la medida . . . 56
11.6.1. La meseta, y por qué la espiral sola no bastaba . . . 59
11.6.2. Lo que descarta el cilindro . . . 59
11.6.3. Las dos métricas no dicen lo mismo . . . 59
11.7. La tensión: célula frente a modelo . . . 60
11.8. Qué aporta realmente el tambor . . . 60
11.8.1. Cuánto motor se ahorra, exactamente . . . 60
11.8.2. Y el cilindro pequeño, cuantificado . . . 61
11.9. Limitaciones de las medidas . . . 61
11.10. Trabajo pendiente para cerrar la validación . . . 62

12. Gestión del proyecto **63**
12.1. Ciclo de vida . . . 63
12.2. Planificación . . . 63
12.2.1. Planificación real . . . 63
12.3. Presupuesto . . . 64
12.3.1. Personal . . . 64
12.3.2. Material . . . 65
12.3.3. Equipos y software . . . 65
12.3.4. Resumen de costes . . . 66
12.4. Impacto . . . 66
12.4.1. Impacto técnico y económico . . . 66
12.4.2. Objetivos de Desarrollo Sostenible . . . 67
12.4.3. Aspectos éticos, legales y de seguridad . . . 67

13. Conclusiones **69**
13.1. Conclusión . . . 69
13.1.1. Sobre el banco de ensayos . . . 69
13.1.2. Sobre el modelo del mecanismo . . . 69
13.1.3. Sobre la validación experimental . . . 69
13.1.4. Grado de cumplimiento de los objetivos . . . 70
13.2. Desarrollos futuros . . . 71
13.2.1. Para cerrar la validación . . . 71
13.2.2. Sobre el mecanismo . . . 71
13.2.3. Sobre el sistema de medida . . . 71
13.2.4. Sobre la seguridad y la explotación . . . 71
13.3. Valoración personal . . . 72

A. Asignación de pines **73**
A.1. Raspberry Pi Pico 2 (revisión B de la placa) . . . 73
A.2. Bornas de campo . . . 74

B. Especificación del protocolo binario **75**
B.1. Paquete 0x02: telemetría por lotes . . . 75
B.2. Paquete 0x03: estado de *homing* (11 bytes) . . . 76
B.3. Paquete 0x04: resultado de la célula (13 bytes) . . . 76
B.4. Paquete 0x05: estado del ciclo (18 bytes) . . . 76
B.5. Órdenes . . . 77
B.6. Tablas de códigos . . . 77

C. Registros Modbus del accionamiento **79**
C.1. Secuencia de configuración . . . 79

D. Formato de los ficheros de ensayo **81**
D.1. Conversión y análisis . . . 81
D.2. Notas sobre las cabeceras . . . 82

E. Manual de uso **83**
E.1. Instalación . . . 83
E.1.1. Firmware . . . 83
E.1.2. Aplicación de supervisión . . . 83
E.2. Configuración del mecanismo . . . 83
E.3. Procedimiento de ensayo . . . 83
E.4. Programación de la célula de carga . . . 84
E.5. Diagnóstico . . . 84
E.6. Precauciones . . . 84

F. Estructura del repositorio **85**
F.1. Reproducción de los resultados 86

Bibliografia **87**

Índice de figuras

4.1. Esquema del mecanismo. El cable sale *tangente* al tambor, en el punto T, y no de un punto fijo: al cambiar el radio, la línea del cable se desplaza. 14

5.1. Arquitectura del banco de ensayos. 23

6.1. Esquema de la placa adaptadora, revisión B. 31
6.2. PCB de la revisión A: capas de cobre, máscara y serigrafía superiores. 31
6.3. Vista 3D de la PCB de la revisión A generada con KiCad. 32

7.1. Máquina de estados del enlace con el accionamiento. 34
7.2. Máquina de estados del *homing*. 36
7.3. Máquina de estados del ciclo de ensayo. 36

8.1. Formato común de trama. El CRC-8 cubre los LEN bytes de la carga útil (ID + datos). 39

9.1. Estructura del servidor de supervisión. 45
9.2. Página de configuración del mecanismo con el simulador. 47

11.1. Ensayo del 14 de octubre a las 18:58:48. De arriba abajo: avance del motor con el ángulo de la barra que predice el modelo, velocidad frente a consigna, célula de carga frente a la tensión del modelo, y par del accionamiento. La banda gris es el tramo en que el cable todavía no tira. 57
11.2. Promedio de los diez ensayos. Arriba: célula en subida y bajada frente a la tensión del modelo. Centro: par en los dos sentidos, con el par gravitatorio τ_g y el de pérdidas τ_f. Abajo: brazo efectivo deducido de la medida frente a las hipótesis de perfil. . . 58
11.3. (a) Par gravitatorio medido frente a las cuatro hipótesis de tambor. (b) Tensión del cable: célula de carga, escalada, frente al modelo. 60

12.1. *Commits* por semana en el repositorio del proyecto. 63
12.2. Diagrama de Gantt reconstruido (semanas desde el 2 de marzo de 2026). 64

Índice de tablas

2.1. Características del microcontrolador RP2350 en la Raspberry Pi Pico 2 [42]. 7

4.1. Parámetros del mecanismo [**VERIFICAR:** cotas tomadas del CAD y del banco; conviene contrastarlas con una medición directa sobre el montaje]. 16
4.2. Perfiles de tambor a igual recorrido (barra hasta $-2{,}6°$ en 1,78 rev de motor). El par de pico es lo que dimensiona el motor; el rizado, lo que mide la compensación. 17

5.3. Decisiones de diseño y alternativas consideradas. 23

6.1. Problemas detectados en la revisión A y corrección en la revisión B. 28
6.2. Pruebas de la placa adaptadora. 30

7.1. Versiones del firmware conservadas en el repositorio. 38

8.1. Enlaces de comunicación del sistema. 39
8.2. Paquetes de telemetría y estado. 40
8.3. Órdenes aceptadas por el firmware. 41
8.4. Parámetros configurables con SET_PARAM. 41
8.5. Servicios HTTP del servidor. 43

11.1. Ensayos del 14 de octubre de 2026. Los diez son consecutivos, tomados en menos de tres minutos y con la misma configuración. 55
11.2. Contraste de las hipótesis sobre el tramo con carga. R^2 ajusta una sola escala positiva (ecuación (4.15)); ρ^2 es el cuadrado de la correlación, ciego al signo. 59
11.3. Relación pico/media sobre el tramo con carga. Como el par medio es el mismo para todos los tambores que recorren lo mismo en las mismas vueltas, el cociente de estas relaciones es directamente lo que se ahorra en par de pico. 61

12.1. Dedicación estimada y coste de personal. 65
12.2. Material del sistema. 65
12.3. Amortización de equipos. 66
12.4. Presupuesto total. 66
12.5. Relación del trabajo con los Objetivos de Desarrollo Sostenible [36]. 67

A.1. Asignación de GPIO según el esquema de la revisión B y el firmware. 73
A.2. Bornas de la placa (revisión B). 74

B.1. Cabecera del lote (14 bytes). 75
B.2. Muestra (14 bytes), repetida n veces. 75
B.3. Estados de la máquina. 77
B.4. Códigos de error. 77
B.5. Resultados de ModbusMaster. 77
B.6. Resultados de las órdenes de la célula. 78

C.1. Registros utilizados por el firmware. 79

D.1. Columnas del fichero de ensayo. 81

Acrónimos y símbolos

Acrónimos

Sigla	Significado
A/D, ADC	Convertidor analógico-digital (*Analog-to-Digital Converter*)
API	Interfaz de programación de aplicaciones
CDC	Clase de dispositivo USB de comunicaciones (puerto serie virtual)
CRC	Código de redundancia cíclica (*Cyclic Redundancy Check*)
CSV	Valores separados por comas (*Comma-Separated Values*)
DE / RE	Habilitación del emisor / receptor de un transceptor RS-485
DI / DO	Entrada digital / salida digital
EEPROM	Memoria no volátil borrable eléctricamente
ESD	Descarga electrostática (*Electrostatic Discharge*)
FC	Final de carrera
GPIO	Entrada/salida de propósito general
GUI	Interfaz gráfica de usuario
HTTP	Protocolo de transferencia de hipertexto
I^2C	Bus serie síncrono de dos hilos (*Inter-Integrated Circuit*)
LC	Célula de carga (*Load Cell*); LC1 y LC2 en el firmware
MISR	Registro de firma de la EEPROM del ZSC31014 (*Multiple-Input Signature Register*)
NC	Contacto normalmente cerrado
ODS	Objetivos de Desarrollo Sostenible
PCB	Placa de circuito impreso (*Printed Circuit Board*)
PIO	E/S programable del RP2350 (*Programmable I/O*)
POR	Reinicio por encendido (*Power-On Reset*)
RPM, rpm	Revoluciones por minuto
RS-485	Estándar TIA/EIA-485 de transmisión diferencial multipunto
RTU	Modo binario de Modbus sobre línea serie (*Remote Terminal Unit*)
SMD	Componente de montaje superficial
STO	Desconexión segura del par (*Safe Torque Off*)
TFG	Trabajo Fin de Grado
UART	Transmisor-receptor asíncrono universal
USB	Bus serie universal
VRD	Tambor de radio variable (*Variable Radius Drum*)
WS	WebSocket

Símbolos principales

Símbolo	Descripción	Unidad
O	Articulación de la barra	–

Símbolo	**Descripción**	**Unidad**
C	Eje de giro del tambor, en $(-x_t,\, L_2)$	–
T	Punto de tangencia: por donde el cable abandona el tambor	–
A′	Punto de la barra donde tira el cable, e_1 sobre su eje	–
P′	Punto del que cuelga la masa, e_2 bajo el eje de la barra	–
L_2	Altura del eje del tambor sobre la articulación O	mm
x_t	Eje del tambor a la izquierda de la vertical de O	mm
$\lvert OC\rvert$	Distancia de O a C, $\sqrt{L_2^2 + x_t^2}$	mm
L_3	De O al punto de tiro, medido sobre el eje de la barra	mm
e_1	El cable tira e_1 por encima del eje de la barra	mm
L_4	Del punto de tiro al de suspensión de la masa	mm
e_2	La masa cuelga e_2 por debajo del eje de la barra	mm
m	Masa suspendida	kg
g	Aceleración de la gravedad	$\mathrm{m/s^2}$
θ	Ángulo de la barra respecto a la horizontal	°
θ_0	Valor de θ en B, donde arranca el ensayo	°
θ_{muerto}	Punto muerto: la línea del cable pasa por O	°
θ_{libre}	Reposo de la barra sin cable	°
α	Ángulo en O entre la línea O→C y la barra	°
c	Longitud libre de cable, de T a A′	mm
g_{eq}	Cable equivalente, $c - \text{lado} \cdot r\, a_T$	mm
T	Tensión del cable	N
φ	Giro del tambor contado desde B	rad
ψ	Parámetro que recorre el perfil del tambor	rad
$r(\psi)$	Radio del perfil del tambor	mm
p	Brazo del cable en el eje del tambor, $p \le r$	mm
b_c, b_p	Brazos del cable y del peso respecto de O	mm
r_0, r_1	Radios de la caracola al principio y al final del barrido	mm
τ	Par en el eje del tambor	$\mathrm{N \cdot m}$
N	Reducción motor:tambor (vueltas de motor por vuelta de tambor)	–
p	Avance del eje del motor contado desde B	rev
τ_g, τ_f	Componentes gravitatoria y de pérdidas del par medido	u. a.
r_{ef}	Radio efectivo estimado, $\propto \tau_g/T$	u. a./N
u. a.	Unidades internas del accionamiento (registro 16387)	–

Capítulo 1

Introducción

1.1. Motivación del proyecto

Muchos mecanismos de elevación no presentan una carga constante al actuador. En una barra articulada que se levanta tirando de un cable, la tensión necesaria para sostener la carga depende del ángulo que forman el cable y la barra; al girar la barra, esa geometría cambia y con ella la tensión. Si el cable se enrolla en un tambor cilíndrico, el par que el motor debe aportar, producto de la tensión por el radio, sigue fielmente esas variaciones, y el actuador hay que dimensionarlo para el peor punto del recorrido aunque la mayor parte del tiempo trabaje muy por debajo.

Una solución clásica es cambiar la relación de transmisión a lo largo del recorrido haciendo que el radio del tambor no sea constante. El antecedente más conocido es el *fusee* de la relojería del siglo XV, un cono con ranura helicoidal que compensaba la pérdida de par del muelle real a medida que se desenrollaba [22]; conviene advertir desde ahora que el mecanismo de este trabajo no es un fusee ni lo imita —es una leva plana, y la fuerza que debe repartir no decae sino que crece—, y que la sección 3.3 delimita hasta dónde llega el parecido. La idea general de modular el brazo reaparece hoy en los tambores de radio variable de la robótica de cables [44, 3, 1] y en los sistemas pasivos de compensación de gravedad [45, 6]. En este trabajo, al tambor en espiral se le llama *caracola* por su forma.

El mecanismo objeto del proyecto es precisamente una caracola que, accionada por un servomotor AC de lazo cerrado, enrolla un cable y levanta una barra articulada con una masa de 2 kg. La hipótesis de partida es que el perfil de radio variable aplana el par que ve el motor frente a la variación de la tensión del cable. Comprobarlo exige dos cosas: un modelo que diga qué cabe esperar y un sistema de medida capaz de registrar a la vez la posición del eje, la tensión del cable y el par del motor en ensayos repetibles. Construir ese sistema, y usarlo para contrastar el modelo, es el contenido de este Trabajo Fin de Grado.

1.2. Objetivos

El objetivo general es diseñar, construir y validar un banco de ensayos para el mecanismo de caracola: la electrónica de accionamiento, control y adquisición, el software de supervisión y el modelo con el que se interpretan las medidas.

De él se derivan los siguientes objetivos específicos:

O1. Desarrollar un modelo estático del mecanismo que relacione la geometría, la tensión del cable y el par en el tambor, y deducir de él el perfil de radio que hace constante el par.

O2. Diseñar una placa de circuito impreso que adapte el microcontrolador al entorno de la máquina: bus RS-485, entradas digitales protegidas, entradas analógicas acondicionadas, salidas de potencia y conexión de células de carga.

O3. Desarrollar el firmware del microcontrolador, capaz de gobernar el servoaccionamiento por Modbus RTU, leer la célula de carga, ejecutar el *homing* y los ciclos de ensayo, y vigilar los límites de seguridad con una temporización determinista.

O4. Definir un protocolo de comunicación robusto entre el microcontrolador y el ordenador, que transporte la telemetría por lotes y las órdenes con detección de errores.

O5. Desarrollar una aplicación de supervisión web para operar la máquina, visualizar los datos en tiempo real, grabar cada ensayo en un fichero reproducible, configurar la geometría del mecanismo y comparar cada ensayo con el modelo.

O6. Caracterizar el banco (temporización, repetibilidad y comportamiento de la medida) y analizar los ensayos para contrastar el modelo y cuantificar el efecto de la caracola.

O7. Documentar los problemas técnicos encontrados y las soluciones adoptadas, así como la planificación, el presupuesto y el impacto del trabajo.

1.3. Materiales utilizados

- **Mecanismo**: caracola de radio variable, cable, barra articulada con una masa de 2 kg y dos finales de carrera **[COMPLETAR:** indicar si la caracola y la estructura se han diseñado y fabricado en este TFG o proceden de un trabajo previo].
- **Accionamiento**: servomotor AC de lazo cerrado con accionamiento Modbus RTU sobre RS-485 **[COMPLETAR:** marca y modelo].
- **Electrónica**: placa adaptadora propia diseñada en KiCad [21] alrededor de un módulo Raspberry Pi Pico 2 [42], con transceptor THVD1400 [47], interruptor de potencia TPS4H000-Q1 [48] y amplificadores MCP6294 [30].
- **Medida de fuerza**: célula de carga con acondicionador ZSC31014 [43] montado en una placa *Load Cell 2 Click* [31].
- **Software**: entorno Arduino con el núcleo Arduino-Pico [16] y la biblioteca ModbusMaster [50] para el firmware; Python con FastAPI [41], pySerial [24] y Chart.js [4] para la supervisión; matplotlib [13] para el análisis, y git para el control de versiones.

1.4. Alcance y limitaciones

Entra en el alcance todo el sistema electrónico y de software (placa, firmware, protocolo, supervisión y análisis), el modelo del mecanismo y la campaña de ensayos. El diseño mecánico se toma como dado.

Las limitaciones conocidas, que se discuten en los capítulos correspondientes, son:

- La posición del eje no se lee del codificador del accionamiento, sino que se obtiene integrando la velocidad que devuelve el propio servo (sección 7.5).
- La célula de carga no está calibrada con masas patrón; los ensayos se guardan en cuentas brutas y la conversión a newtons es provisional (sección 11.9).
- El par se obtiene en las unidades internas del accionamiento, cuya escala física no se ha confirmado, de modo que los resultados se expresan en variaciones relativas.
- Las cotas del mecanismo no se han medido: los parámetros geométricos proceden de un ajuste sobre los propios ensayos (capítulo 11).
- Los finales de carrera y el límite de fuerza se implementan por software; no hay una cadena de seguridad cableada (sección 5.5).

1.5. Metodología

El desarrollo ha sido incremental y guiado por pruebas. Cada subsistema (bus RS-485, célula de carga, finales de carrera, entradas y salidas de la placa) se validó primero con un programa de prueba aislado, que se conserva en el repositorio (anexo F), antes de integrarlo en el firmware principal.

El código se ha versionado con git desde el primer día, y ese historial ha servido además para reconstruir la planificación real (capítulo 12). Los ensayos se guardan en crudo, con las constantes de conversión en la cabecera de cada fichero, de modo que cualquier resultado puede recalcularse sin repetir el ensayo; las figuras de resultados de esta memoria se generan con programas reproducibles a partir de esos ficheros.

1.6. Estructura del documento

- El capítulo 2 resume los conceptos necesarios: accionamientos, medida de fuerza con galgas, buses de campo y microcontroladores.
- El capítulo 3 sitúa el trabajo frente a los tambores de radio variable y los mecanismos de compensación publicados.
- El capítulo 4 desarrolla el modelo del mecanismo y el método experimental con el que se contrasta.
- El capítulo 5 recoge los requisitos y la arquitectura del banco.
- Los capítulos 6 a 9 describen el hardware, el firmware, las comunicaciones y el software de supervisión.
- El capítulo 10 reúne los problemas técnicos y sus soluciones.
- El capítulo 11 presenta la caracterización del banco y los resultados de las dos campañas de ensayos.
- El capítulo 12 trata el ciclo de vida, la planificación y el presupuesto, y el capítulo 13 las conclusiones y el trabajo futuro.

Los anexos contienen la asignación de pines, el protocolo, los registros Modbus, el formato de los ficheros de ensayo, el manual de uso y la estructura del repositorio.

Capítulo 2

Marco teórico

Este capítulo resume los conceptos necesarios para seguir el resto de la memoria: cómo se acciona el eje, cómo se mide la fuerza, con qué microcontrolador y por qué buses viaja la información.

2.1. Transmisión por cable y radio variable

En una transmisión por cable, el par que hay que aplicar al tambor es el producto de la tensión del cable por el radio con el que ese cable sale del tambor:

$$\tau = T\, r. \tag{2.1}$$

Si el radio es constante, el par reproduce exactamente la forma de la tensión. Si el radio cambia con el giro, $r(\varphi)$, se dispone de un grado de libertad para moldear el par: basta con que el radio sea grande donde la tensión es pequeña, y pequeño donde es grande, para que el producto se mantenga aproximadamente constante. Ésa es la idea de los tambores de radio variable, y el capítulo 4 la desarrolla para la geometría concreta del prototipo.

Hay que añadir enseguida una advertencia que el capítulo del modelo convierte en resultado: la ecuación (2.1) usa el radio como si el cable saliera del tambor por un punto fijo y tangente a una circunferencia. Ninguna de las dos cosas es exacta en un tambor de radio variable. El brazo que multiplica a la tensión es la distancia del eje a la recta del cable, que donde el perfil tiene pendiente es *menor* que el radio, y el punto por el que el cable abandona el tambor se desplaza al girar. Ambos efectos son del orden del 10 % en este prototipo, es decir, del mismo orden que lo que se quiere medir.

Conviene separar dos cosas que se confunden con facilidad. Un mecanismo de compensación *pasiva* (un muelle con una polea no circular) reduce la energía que el actuador tiene que aportar. Una caracola accionada por un motor no reduce esa energía: el trabajo de subir la masa es el mismo. Lo que hace es repartirlo de manera uniforme a lo largo del giro, lo que reduce el par de pico y, con él, el tamaño del motor necesario.

2.2. Accionamiento: servomotor AC de lazo cerrado

Para un banco cuya magnitud de medida es el par del propio accionamiento, la elección del motor no es indiferente. Un motor paso a paso convencional funciona en lazo abierto: el controlador impone la secuencia de fases y se da por hecho que el rotor la sigue; es sencillo y tiene buen par a baja velocidad, pero si la carga supera el par disponible se pierden pasos sin que el sistema lo detecte [20], y la corriente de fase no guarda una relación limpia con el par entregado.

El accionamiento de este trabajo es un servomotor síncrono de imanes permanentes de corriente alterna con codificador absoluto. El accionamiento cierra el lazo de corriente sobre la posición del rotor mediante control vectorial, de modo que la corriente en cuadratura es proporcional al par y puede publicarse como magnitud de supervisión: eso es lo que hace utilizable la lectura de par del capítulo 11. Ofrece modos de control en posición, velocidad y par, y entrega la velocidad y el par reales por el bus.

Sus características son 400 W de potencia, 1,27 N m de par nominal, 3000 rpm de velocidad nominal, codificador absoluto de 17 bits —esto es, 131.072 cuentas por vuelta— y protección IP67. Incorpora además la función STO (*Safe Torque Off*), una entrada de seguridad cableada que corta la excitación de las etapas de potencia sin pasar por el firmware, de acuerdo con [15]. **[VERIFICAR:** confirmar el modelo exacto en la etiqueta del accionamiento: la serie se comercializa en variante EtherCAT y en variante RS-485, y el firmware de este trabajo usa la segunda]

Dos detalles importan para el resto de la memoria. El primero es que un servo AC frena de forma regenerativa: al bajar la carga, el motor devuelve energía al bus de continua, y por eso el accionamiento necesita una resistencia de frenado, cuyos parámetros el firmware programa explícitamente (anexo C). El segundo es que la interfaz elegida es RS-485 con protocolo Modbus RTU, por la que se configura, se habilita, se envía la consigna de velocidad y se leen la velocidad y el par. Tenerlo todo en un bus de dos hilos simplifica el cableado frente a la alternativa de señales de paso y dirección más entradas analógicas, a costa de una latencia que el capítulo 7 analiza en detalle.

2.3. Medida de fuerza con células de carga

2.3.1. Galgas extensométricas y puente de Wheatstone

Una célula de carga convierte la fuerza en una deformación elástica, que se mide con galgas extensométricas dispuestas en un puente de Wheatstone [12]. La salida del puente es una tensión diferencial de unos pocos milivoltios por voltio de excitación a plena escala, que hay que amplificar con alta ganancia y bajo ruido antes de digitalizarla; además hay que compensar el desequilibrio de cero del puente y su deriva con la temperatura [40]. De ahí que la cadena de medida necesite, ante todo, una calibración con masas patrón: sin ella se conoce la forma de la señal, pero no su escala ni su cero.

2.3.2. Acondicionadores integrados

Existen circuitos integrados que agrupan excitación, amplificación, conversión A/D y corrección digital. En este trabajo se han evaluado dos:

- **ZSC31014** de Renesas [43], con salida digital I^2C, preamplificador de ganancia programable, conversor de 14 bits y corrección almacenada en EEPROM. Cada lectura incluye dos bits de estado que indican si el dato es nuevo. Su configuración sólo puede modificarse en un *modo comando* al que se accede durante una ventana corta tras el encendido, lo que ha condicionado el diseño (sección 10.2). Se usa en una placa *Load Cell 2 Click* [31].
- **NAU7802** de Nuvoton [38], conversor sigma-delta de 24 bits con amplificador programable y regulador interno. Se probó como segunda célula y se retiró del firmware final.

2.4. Microcontrolador

El proyecto empezó con una placa basada en ESP32-S3 (Seeed XIAO) y se migró a una Raspberry Pi Pico 2. La tabla 2.1 resume las características del RP2350 [42]. El motivo del cambio y sus consecuencias se explican en la sección 10.1: los pines que impone el trazado de la placa no admiten una UART hardware, pero el RP2350 dispone de periféricos PIO capaces de implementar una UART en cualquier pin.

Tabla 2.1: Características del microcontrolador RP2350 en la Raspberry Pi Pico 2 [42].

Característica	Raspberry Pi Pico 2 (RP2350)
Núcleos	2 × Arm Cortex-M33 o 2 × RISC-V Hazard3, a 150 MHz
Memoria	520 kB SRAM, 4 MB flash QSPI
Periféricos serie	2 × UART, 2 × SPI, 2 × I^2C
E/S programable	12 máquinas de estado PIO
Entradas analógicas	3 canales ADC
Entorno usado	Arduino-Pico [16]

2.5. Buses de comunicación

2.5.1. RS-485 y Modbus RTU

RS-485 [46] es un estándar de capa física diferencial y multipunto, extendido en la industria por su inmunidad al ruido, su alcance y la posibilidad de conectar varios equipos al mismo par trenzado. En modo semidúplex cada nodo activa su emisor sólo mientras transmite, y ese control de dirección (líneas DE y $\overline{\text{RE}}$ del transceptor) corresponde al maestro.

Modbus [34] es un protocolo maestro-esclavo de petición y respuesta que modela los datos del esclavo como registros de 16 bits. En su variante RTU sobre línea serie [33], cada trama lleva la dirección del esclavo, el código de función, los datos y un CRC-16, y las tramas se delimitan por silencios de al menos 3,5 caracteres. Aquí se usan la función 0x03 (lectura de registros) y la 0x06 (escritura de un registro), a través de la biblioteca ModbusMaster [50].

2.5.2. I^2C y mikroBUS

I^2C [39] es un bus síncrono de dos hilos con resistencias de *pull-up*, pensado para comunicar circuitos dentro de una misma placa; es el que usan los acondicionadores de célula de carga. Las placas Click siguen el estándar mikroBUS [32], que define un zócalo de 16 pines con I^2C, SPI, UART, alimentación y señales auxiliares.

2.6. Adquisición y supervisión en el ordenador

La supervisión de máquinas se ha hecho tradicionalmente con paneles HMI o paquetes SCADA comerciales. Para un banco de laboratorio, una aplicación web local es una alternativa ligera: no requiere licencias, funciona en cualquier navegador y admite varios clientes a la vez. En este trabajo se emplea FastAPI [41] con pySerial [24] para el puerto USB, WebSocket [7] para el envío continuo de datos al navegador y Chart.js [4] para las gráficas.

Dos conceptos de adquisición son importantes para interpretar los resultados. El primero es la *base de tiempos*: si las muestras se marcan con el reloj del ordenador, la latencia del enlace se confunde con el fenómeno medido; por eso aquí cada muestra lleva la marca de tiempo del microcontrolador. El segundo es la distinción entre *dato bruto* y *dato calibrado*: guardar las cuentas del conversor, y no los newtons, permite recalibrar un ensayo sin repetirlo, que es justo lo que ha hecho falta en este trabajo.

Capítulo 3

Estado del arte

Este capítulo sitúa el trabajo frente a lo publicado sobre tambores de radio variable y mecanismos de compensación, y frente a las plataformas con las que se ensayan.

3.1. Antecedentes: variar el radio para repartir el par

La idea de hacer variable el radio con que trabaja un cable, para que el par no siga ciegamente a la fuerza, tiene un antecedente conocido en relojería. En los relojes de muelle el par disminuye a medida que el muelle se desenrolla, y el *fusee* lo corrige con un cono de perfil curvo y una ranura helicoidal por la que trepa una cadena: con el muelle tenso la cadena tira del radio pequeño y, al debilitarse, pasa al radio grande [22]. Lloyd [26] sitúa su origen en el siglo XV, y Weaver [51] fue quien, casi cinco siglos después, planteó por primera vez la ecuación diferencial de su perfil.

Conviene ser preciso con el alcance de ese parecido, porque es más superficial de lo que sugiere la analogía. El mecanismo de este trabajo **no es un fusee** ni lo imita. Un fusee es un cuerpo de revolución cónico en el que el radio cambia *axialmente*, a medida que la cadena avanza por la hélice; aquí el tambor es una leva plana y el cable trabaja siempre en el mismo plano, con el radio cambiando con el ángulo girado. Y, sobre todo, el problema físico es distinto: el fusee compensa una fuerza que *decae* de forma monótona, mientras que en este banco la tensión del cable no decae —el capítulo 4 muestra que crece—, de modo que ni el sentido de la corrección ni la ley del perfil se trasladan. Lo único que comparten es el principio general de modular el brazo.

La familia a la que este mecanismo sí pertenece es la de los tambores de radio variable (*Variable Radius Drums*, VRD) y las poleas no circulares, formalizada en robótica.

Seriani y Gallina [44] plantean la síntesis del perfil de un tambor para que la carga siga una ley prescrita y obtienen soluciones analíticas cerradas para varios casos. En los robots paralelos accionados por cables, Bieber *et al.* [3] diseñan el perfil a partir del perfil de carga máxima del espacio de trabajo, de modo que el par del motor no supere un valor dado para ninguna longitud de cable. Arsenault [1] aborda el problema desde la fabricación y la cinemática del enrollado, y establece las condiciones de *convexidad* que debe cumplir el perfil para que el cable se apoye sobre él sin despegarse —una restricción que, como se verá, es la que acota la excentricidad admisible en el tambor de este trabajo—. Ludovico *et al.* [27] formulan la síntesis del perfil como un problema de optimización convexa, con lo que la obtención del perfil deja de depender de un buen punto de partida. La fabricación aditiva ha abaratado mucho estas piezas, lo que explica el interés reciente.

3.2. Compensación de gravedad con poleas no circulares

Un segundo campo de aplicación es la compensación estática de peso. Herder [11] estudia de forma sistemática los mecanismos estáticamente equilibrados, en los que la energía potencial total se mantiene constante y el actuador sólo vence las pérdidas. Endo *et al.* [6] proponen una polea no circular combinada con un muelle para compensar el par gravitatorio de un brazo, y deducen numéricamente la forma de la polea. Shen y Yano [45] refinan ese cálculo teniendo en cuenta la variación de la tensión del cable a lo largo del recorrido, y mejoran así la calidad de la compensación.

El mecanismo de este trabajo comparte el principio, con una diferencia importante: aquí no hay muelle que almacene energía, sino un motor que aporta todo el trabajo de elevación. El tambor no ahorra energía, sino que reparte el par.

3.3. Lo que el cable impone al perfil

La literatura anterior sintetiza perfiles *libres*: dada una ley de par, se obtiene la curva $r(\varphi)$ que la produce, y se fabrica. El planteamiento es general, pero arrastra dos restricciones que no siempre se hacen explícitas y que en este trabajo resultan decisivas.

La primera es de tolerancias. Johnston *et al.* [18] analizan la sensibilidad de los mecanismos de leva enrollada a la incertidumbre en los parámetros y muestran que la ventaja teórica de un perfil optimizado se degrada con los errores de fabricación y montaje. Gao *et al.* [9] resuelven el mismo problema inverso —del par deseado al perfil— en mecanismos leva-muelle, y Litvin *et al.* [25] tratan la generación de funciones de transmisión variables en engranajes no circulares.

La segunda es más fundamental, y es la que marca el límite de lo que se puede pedir a un tambor accionado por cable: **un cable tenso sólo se asienta sobre un perfil convexo y de pendiente suave**. En un tramo cóncavo, o en uno cuyo radio varíe demasiado rápido, el cable puentea el perfil como una cuerda y el brazo que trabaja deja de ser el del perfil. Arsenault [1] establece precisamente las condiciones de convexidad que debe cumplir un tambor de radio variable para que el cable se apoye sobre él sin despegarse; en la práctica se traduce en mantener el ángulo de presión $\arctan(|r'|/r)$ por debajo de unos $25°$.

Esta condición es la que, en el capítulo 4, acaba descartando el perfil de par exactamente constante para este banco: los perfiles que resuelven $r = \tau_0/T$ con un radio de arranque alto exigen al cable pendientes de $60°$ a $80°$, que no puede seguir. La aportación de este trabajo en ese punto no es un perfil nuevo, sino una cuantificación: sobre un mecanismo real se muestra que, una vez impuesta la restricción del cable y comparando a igual recorrido, la diferencia de par de pico entre todos los perfiles admisibles cae por debajo del rozamiento del propio banco.

3.4. Transmisiones por cable: el *capstan drive*

La reducción entre el motor y el tambor se resuelve en este trabajo con una transmisión por cable. Su fundamento es la ecuación del cabrestante, que relaciona exponencialmente las tensiones a ambos lados de un arrollamiento con el ángulo abrazado; Levin [23] la contrasta experimentalmente, y Jung *et al.* [19] la extienden para incluir la rigidez a flexión de la cuerda y un rozamiento no lineal, efectos que la formulación clásica ignora y que importan cuando el radio del cabrestante es pequeño.

Sobre ese principio, Werkmeister y Slocum [52] caracterizan teórica y experimentalmente la rigidez de un *capstan drive*, y Baser y Konukseven [2] hacen lo propio con su error de deslizamiento, las dos magnitudes que deciden si la transmisión sirve para posicionar. En robótica, Mazumdar *et al.* [28] demuestran que una transmisión de este tipo con fibra sintética alcanza rendimientos en torno al 95 % y un ancho de banda en par muy superior al de un reductor de engranajes, lo que la hace idónea para control de par —que es justamente el uso que aquí se le da a la lectura del accionamiento—. Hwangbo *et al.* [14] la llevan a robots de alta dinámica.

La configuración concreta que sigue este trabajo está tomada de un proyecto abierto de Musa [35], que documenta una reducción de 8,55:1 con cuerda de Dyneema y recorrido limitado a $120°$. Conviene señalar que se trata de literatura gris, sin revisión por pares; se cita por la utilidad de sus detalles constructivos, apoyándose para las afirmaciones de fondo en las referencias académicas anteriores.

Las ventajas que importan aquí son la holgura nula y la transparencia de par: cualquier holgura o rozamiento del reductor contaminaría precisamente la señal que el banco quiere medir. Las limitaciones —recorrido acotado y necesidad de retensado— son asumibles en un mecanismo que trabaja entre dos finales de carrera.

3.5. Cuerda sintética frente a cable de acero

La elección del material de la cuerda no es accesoria. El criterio determinante en una transmisión de radio pequeño es la relación entre el diámetro del tambor y el del cable, D/d: por debajo de cierto valor, el cable de acero falla por fatiga de flexión en muy pocos ciclos, y Feyrer [8] recoge el tratamiento canónico de esa vida a fatiga. Las fibras de polietileno de ultra alto peso molecular (UHMWPE, comercialmente Dyneema) toleran radios mucho menores, lo que permite cabrestantes pequeños y, con ellos, reducciones altas en poco espacio. McKenna *et al.* [29] es la referencia general de mecánica de cuerda de fibra, y Davies *et al.* [5] proponen un modelo empírico de vida bajo flexión cíclica sobre polea que permite dimensionar D/d con criterio.

El punto débil de estas fibras es la fluencia: un alargamiento permanente bajo carga mantenida que se traduce en pérdida de tensión y, por tanto, en error de posicionamiento. Vlasblom y Bosman [49] modelan la vida por fluencia y muestran su fuerte dependencia del grado de fibra y de la temperatura. De ahí que una transmisión de este tipo necesite un tensor y un retensado periódico, y que el modelo del capítulo 4 advierta de que la hipótesis de cable inextensible sólo vale entre retensados.

3.6. Bancos de ensayo y validación experimental

En los trabajos citados, la validación experimental suele consistir en medir el par o la corriente del actuador a lo largo del recorrido y compararlos con el perfil previsto. Esa comparación exige tres medidas sincronizadas: posición del eje, fuerza en el cable y par del actuador. Los equipos comerciales de adquisición que resuelven esto tienen un coste muy superior al del propio prototipo, y los accionamientos industriales modernos ya publican velocidad y par por bus de campo, de modo que buena parte de la instrumentación puede sustituirse por software.

Ésa es la oportunidad que aprovecha este TFG: construir, con componentes comerciales de bajo coste y herramientas libres, un banco que registre las tres magnitudes con la misma base de tiempos, guarde los datos en bruto y permita contrastar el modelo del mecanismo con la medida. Un resultado colateral del trabajo, que se detalla en el capítulo 11, es que esa instrumentación completa resulta *necesaria*: la medida de fuerza por sí sola no basta para determinar el perfil del tambor, porque la calibración desconocida de la célula admite soluciones con el signo invertido; es el par del accionamiento el que deshace la ambigüedad.

3.7. Conclusiones del estado del arte

Los tambores de radio variable están bien estudiados como principio y existen métodos de síntesis de su perfil, igual que están bien caracterizadas las transmisiones por cable y las fibras sintéticas que las hacen viables. De ese panorama se desprenden las dos líneas que sigue este trabajo.

La primera es de diseño: casi toda la literatura da por supuesto que afinar el perfil merece la pena, y lo demuestra sobre el perfil de par. Aquí se compara a igual recorrido y midiendo lo que de verdad dimensiona el accionamiento, que es el par de pico; el capítulo 4 muestra que en este mecanismo la diferencia entre todos los perfiles admisibles queda por debajo del rozamiento del banco, de modo que la elección del perfil importa mucho menos de lo esperado.

La segunda es experimental: falta una plataforma asequible que permita medir en un prototipo real cómo se reparte el par y contrastarlo con el modelo. Para ello hay que integrar un accionamiento industrial por Modbus, un acondicionador de célula de carga con reglas de configuración estrictas y una cadena de adquisición con marcas de tiempo fiables, que es lo que desarrollan los capítulos siguientes. El capítulo 11 muestra además por qué esa instrumentación tiene que ser completa: la fuerza sola no determina la geometría.

Capítulo 4

Modelo del mecanismo

Este capítulo desarrolla el modelo estático del mecanismo: relaciona el giro del tambor con el ángulo de la barra, la longitud del cable y su tensión, y de ahí obtiene el par. Las mismas ecuaciones están implementadas en `static/mecanismo.js`, de modo que el simulador, el visor de ensayos y las figuras de esta memoria calculan exactamente lo mismo; el anexo F indica cómo regenerar las cifras.

Conviene avisar de entrada de que este modelo sustituye a uno anterior, y que la diferencia no es de refinamiento sino de física. En la primera versión el cable salía de un punto fijo D, desplazado una distancia dx de la vertical de la articulación. Ese modelo no podía representar la barra colgando por debajo de cierto ángulo, y el ajuste lo compensaba invirtiendo el sentido de la espiral, de donde salían unas conclusiones que la sección 4.5 explica y descarta. El cable no sale de un punto: sale *tangente* al tambor, y el punto de salida se mueve cuando el radio cambia.

4.1. Descripción del mecanismo

La figura 4.1 muestra el esquema. Una barra rígida articulada en O gira en un plano vertical. El tambor va montado sobre un carril a la izquierda de O, con su eje de giro en

$$C = (-x_t,\ L_2), \tag{4.1}$$

y el cable baja desde el tambor hasta la barra. Se usan dos convenios que conviene fijar porque atraviesan todo el capítulo:

- El ángulo de la barra, θ, se mide *desde la horizontal*. Es lo que se configura y lo que se puede comprobar con un inclinómetro apoyado en la barra, sin modelo de por medio. Internamente se usa también $\alpha = \mathrm{inc}_C - \theta$, el ángulo en O entre la línea O→C y la barra, que es el que entra en la ley del coseno.
- La barra lleva dos desfases respecto de su propio eje: el cable no tira del eje sino de un punto A′ situado e_1 por encima, y la pesa no cuelga del eje sino de P′, e_2 por debajo. Con $\hat{u} = (\cos\theta, \operatorname{sen}\theta)$ la dirección de la barra y $\hat{n}$ su perpendicular,

$$A' = L_3\,\hat{u} + e_1\,\hat{n}, \qquad P' = (L_3 + L_4)\,\hat{u} - e_2\,\hat{n}. \tag{4.2}$$

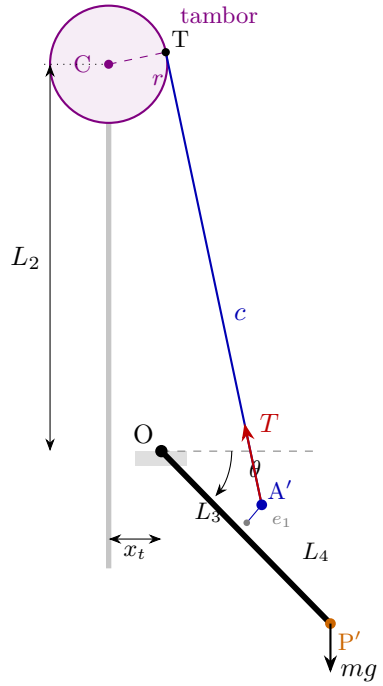


Figura 4.1: Esquema del mecanismo. El cable sale *tangente* al tambor, en el punto T, y no de un punto fijo: al cambiar el radio, la línea del cable se desplaza.

El modelo parte de las siguientes hipótesis:

H1. Régimen cuasiestático: a velocidad constante se desprecian las fuerzas de inercia.

H2. Cable inextensible y sin masa, que se mantiene tenso y se asienta sobre el perfil.

H3. Se desprecia la masa de la barra frente a la masa suspendida.

H4. Las pérdidas por rozamiento no entran en el modelo estático: se tratan aparte en la sección 4.6.

4.2. Los dos ángulos límite del mecanismo

Antes de las ecuaciones conviene fijar dos ángulos que no son parámetros de diseño sino propiedades de la geometría, y que acotan el recorrido útil.

El **punto muerto** es el ángulo al que la línea del cable pasa por O. Ahí el brazo del cable se anula mientras la pesa sigue haciendo momento, de modo que la tensión se iría a infinito y el mecanismo no puede sostener la carga. Ocurre cuando A′ cae sobre la tangente trazada desde O al tambor, que forma un ángulo $\gamma = \arcsin(r/|OC|)$ con la línea O→C:

$$\theta_{\text{muerto}} = \text{inc}_C - \text{lado}\cdot\gamma - \delta - 180^\circ, \qquad \delta = \arctan\frac{e_1}{L_3}, \tag{4.3}$$

donde lado $= +1$ si el cable baja por la derecha del tambor. Con la geometría del banco cae en $-106{,}4^\circ$.

El **reposo libre** es el ángulo al que queda la barra sin cable: la pesa se sitúa justo bajo el pivote y su brazo se anula. Como el cable sólo puede tirar, nunca empujar, es el θ mínimo al que la barra puede quedarse:

$$\theta_{\text{libre}} = -\arctan\frac{L_3 + L_4}{e_2}, \tag{4.4}$$

que con $L_3 + L_4 = 205\,\text{mm}$ y $e_2 = 20\,\text{mm}$ da $-84{,}4^\circ$. Es una cota que se mide sin modelo —inclinómetro sobre la barra con el cable flojo— y de ella sale e_2, que resulta ser el parámetro más sensible del banco. Explica además las lecturas minúsculas y erráticas de la célula al principio del ensayo: ahí el mecanismo no necesita fuerza.

4.3. Tensión del cable

Con $|OC| = \sqrt{L_2^2 + x_t^2}$ y $L_3' = \sqrt{L_3^2 + e_1^2}$, la ley del coseno en el triángulo O–C–A′ da la distancia del eje del tambor al punto de tiro:

$$d^2 = |OC|^2 + L_3'^2 - 2\,|OC|\,L_3'\cos\beta, \qquad \beta = \alpha - \delta. \tag{4.5}$$

Como el radio es perpendicular a la tangente, el cable libre es el cateto del triángulo rectángulo C–T–A′:

$$c = \sqrt{d^2 - r^2}. \tag{4.6}$$

El equilibrio de momentos en O enfrenta el brazo del cable, b_c —la distancia de O a la recta T–A′—, con el brazo del peso, que por colgar la pesa por debajo del eje tiene dos términos:

$$\boxed{T = \frac{m\,g\,b_p}{b_c}} \qquad b_p = (L_3 + L_4)\cos\theta + e_2\,\mathrm{sen}\,\theta. \tag{4.7}$$

El segundo término de b_p es el que hace que el brazo del peso se anule en θ_{libre} en lugar de en $-90°$.

4.3.1. El cable equivalente

Aquí está el punto más delicado del modelo. Al moverse la barra, el punto de tangencia T *migra* por la superficie del tambor, y esa migración enrolla o desenrolla cable sin que el tambor haya girado. Dicho de otro modo: el cable que el tambor paga al girar no es la variación del cable libre c. Lo que se conserva es el *cable equivalente*

$$g = c - \text{lado}\cdot r\,a_T, \tag{4.8}$$

siendo a_T el ángulo del punto de tangencia sobre el tambor. La comprobación es por trabajos virtuales: $-\mathrm{d}g/\mathrm{d}\theta$ es exactamente el brazo del cable en O, mientras que $-\mathrm{d}c/\mathrm{d}\theta$ se queda entre un 10 % y un 30 % corto. Sin este término el modelo recogía unos 10 mm de más en cada recorrido, un error del mismo orden que lo que se quería medir.

4.3.2. Contacto exacto con el perfil

Las ecuaciones anteriores suponen que el cable sale tangente a una *circunferencia* de radio r. En un tambor de radio variable eso es una aproximación: el perfil tiene pendiente, y la recta tangente a la circunferencia del radio local cortaría el lóbulo que viene detrás. El cable real toca el perfil donde es tangente *al perfil*, en un punto ψ^* que se obtiene de

$$\big(P(\psi) - A'\big) \times \frac{\mathrm{d}P}{\mathrm{d}\psi} = 0, \tag{4.9}$$

y de ahí salen las dos magnitudes que importan: el brazo del par en el tambor es la distancia del eje a esa recta, que es *menor* que r donde el perfil tiene pendiente, y el cable enrollado es la longitud de *arco* del perfil, no r por el ángulo. El par en el tambor y en el motor son

$$\tau = T\,p(\psi^*), \qquad \tau_m = \frac{\tau}{N\,\eta}, \tag{4.10}$$

con p ese brazo, N la reducción y η el rendimiento de la transmisión.

Con el radio variando no queda relación cerrada entre el giro del tambor y el ángulo de la barra, de modo que la trayectoria $\theta(\varphi)$ se integra paso a paso imponiendo que el cable total —libre más arco enrollado— se conserve. Es lo que hace `Mec.trayectoria()`, y lo que consultan después todas las demás funciones.

4.3.3. Comprobación: balance de energía

El modelo no se ajusta a nada; conviene por tanto comprobarlo contra una identidad física. El trabajo que entrega el tambor a lo largo del recorrido tiene que ser la energía potencial que gana la masa:

$$\int_0^{\varphi_{\text{tot}}} \tau \, \mathrm{d}\varphi \;=\; m\, g\, \Delta h_{P'}. \tag{4.11}$$

Integrando numéricamente con la geometría del banco se obtiene 3,5685 J por el primer miembro y 3,5686 J por el segundo: coinciden en la cuarta cifra. Es una comprobación exigente, porque recorre toda la cadena —perfil, contacto, cable equivalente y trayectoria— y cualquier inconsistencia en el brazo o en el cable pagado rompería la igualdad.

4.4. Parámetros del mecanismo

Tabla 4.1: Parámetros del mecanismo [**VERIFICAR:** cotas tomadas del CAD y del banco; conviene contrastarlas con una medición directa sobre el montaje].

Parámetro	Valor	Comentario
L_2	330 mm	altura del eje del tambor sobre O
x_t	30 mm	eje del tambor a la izquierda de O (es el carril)
$\lvert OC\rvert$	331,4 mm	línea O→C, a 95,2°
L_3	95 mm	del pivote al punto de tiro, sobre el eje de la barra
e_1	20 mm	el cable tira por encima del eje
L_4	110 mm	resto de barra hasta la pesa
e_2	20 mm	la pesa cuelga por debajo del eje
m	2 kg	masa suspendida
θ_0	−87°	barra en B, medida con inclinómetro
$r_0 \to r_1$	56 mm a 28,18 mm	radios de la espiral, medidos sobre el CAD
Barrido	0,367 rev de tambor	= 1,835 rev de motor medidas
N	5	reducción motor:tambor
θ_{muerto}	−106,4°	de (4.3), no es un parámetro
θ_{libre}	−84,4°	de (4.4), no es un parámetro

4.5. Qué hace realmente este mecanismo

Con el modelo cerrado se puede responder a la pregunta de diseño, y la respuesta obliga a corregir lo que se creía al empezar.

4.5.1. La tensión no baja: sube

La intuición de partida era la del compensador clásico: al subir la barra la tensión del cable disminuye, y el tambor tiene que ir dando radio para mantener el par. En esta geometría *no es así*. Sobre el tramo en que el cable trabaja con carga, la tensión del modelo *crece* de 36,7 a 41,5 N, un 13 %, y pasada la horizontal sigue creciendo hasta dispararse cerca de 70°, donde el cable se alinea con la barra. La razón es que el brazo del peso y el brazo del cable crecen casi a la vez y se cancelan en buena medida: la barra, por sí sola, ya está casi compensada.

De ahí se sigue lo importante: *aquí hay poco que compensar*. El tambor no está corrigiendo una tensión que se desploma, sino modulando una tensión que varía poco.

4.5.2. Por qué el par de pico, y por qué a igual número de vueltas

Antes de comparar perfiles hay que fijar con qué criterio, porque de eso depende todo lo demás.

La energía no sirve como criterio. El trabajo de subir la masa no depende del perfil: es $m\,g\,\Delta h$ y punto. Un tambor no ahorra energía, sólo la reparte a lo largo del giro.

El par de pico sí. Un motor y su accionamiento se eligen por el par máximo que tienen que dar, aunque sea un instante. Si el par sube un 30 % a lo largo del recorrido, hace falta un equipo un 30 % mayor que el resto del tiempo va sobrado. Aplanar la curva baja el pico hacia la media, y la relación pico/media es, por tanto, la medida directa del beneficio. Hay además dos razones de segundo orden que apuntan en el mismo sentido: el par es corriente y las pérdidas en el cobre van con su cuadrado, de modo que para el mismo trabajo la corriente eficaz —y el calentamiento— es mínima cuando la corriente es constante; y el lazo de velocidad ve una perturbación constante en lugar de una rampa, lo que mejora el seguimiento sin tocar ganancias.

Y la comparación tiene que ser a igual número de vueltas. Ésta es la condición que hace no trivial todo el problema, y conviene plantearla como la objeción natural: *si lo que se quiere es menos par, ¿no basta con poner un cilindro más pequeño?* Un cilindro más pequeño baja el par, sí, pero no gratis: el tambor sólo tiene dos cosas que ofrecer, par y cable por vuelta, y las dos son el mismo radio,

$$\tau = T\,r, \qquad \frac{\mathrm{d}s}{\mathrm{d}\varphi} = r. \tag{4.12}$$

Bajar el radio baja las dos a la vez: menos par, pero también menos recorrido por vuelta, es decir más vueltas y más tiempo de ciclo. Lo que un tambor de radio variable ofrece es precisamente desacoplarlas: usar radio grande mientras la tensión es baja —avanzando deprisa sin pasarse de par— y pasar a radio pequeño justo cuando la tensión sube. En el mejor de los casos consigue *el pico del cilindro pequeño con las vueltas del grande*. Dicho al revés: para un motor dado y un tiempo de ciclo dado, es lo que permite levantar más carga.

Por eso la tabla siguiente escala cada perfil para que la barra llegue al mismo ángulo final en las mismas vueltas de motor. Sin esa normalización se estarían comparando ciclos de duración distinta, y cualquier tambor pequeño parecería mejor.

4.5.3. Comparación de perfiles a igual recorrido

La comparación tiene que hacerse a igual recorrido, o el tambor más grande parece peor sólo porque mueve más carga por vuelta. La tabla 4.2 escala cada perfil para que la barra llegue al mismo ángulo final en las mismas vueltas de motor.

Tabla 4.2: Perfiles de tambor a igual recorrido (barra hasta $-2{,}6°$ en 1,78 rev de motor). El par de pico es lo que dimensiona el motor; el rizado, lo que mide la compensación.

Perfil	r inicial → final (mm)	τ de pico (N · m)	Rizado
Cilindro	44,6 → 44,6	1,84	13,4 %
Espiral de la pieza, 56 → 28 mm	48,8 → 33,3	1,80	25,2 %
Espiral de la pieza con su meseta	46,2 → 38,3	1,82	13,6 %
Espiral suave, 52 → 44 mm	**45,7 → 41,4**	**1,75**	5,5 %
Espiral invertida	39,8 → 58,0	2,39	52,5 %

Dos lecturas, y las dos van contra lo que se esperaba:

- **El par de pico apenas depende del perfil.** Entre todas las opciones razonables va de 1,75 a 1,84 N m: un 5 % de diferencia. Como es el par de pico lo que dimensiona el motor, *optimizar el perfil no ahorra motor*. Sólo la espiral invertida se sale, y lo hace para mal.
- **La pieza fabricada no es la mejor elección.** Con 56 → 28 mm compensa de más y deja un rizado del 25 %, casi el doble que un simple cilindro (13,4 %). Una espiral mucho más suave, de 52 a 44 mm, baja el rizado al 5,5 % *y* da el par de pico más bajo de la tabla.

4.5.4. Por qué no se persigue una leva de par constante

Es natural preguntarse por el perfil que haría el par exactamente constante, $r = \tau_0/T$. Se ha implementado y se descarta, por tres razones que conviene dejar escritas:

1. El beneficio es marginal. Visto lo anterior, el par de pico que se ganaría frente a una espiral suave está dentro del 5 %, mientras que el rozamiento seco medido es del 18 % del par (sección 11.5). Se optimizaría muy por debajo del ruido del sistema.
2. No hay solución en el arranque. Con el cable flojo la tensión tiende a cero y el radio que pide el par constante tiende a infinito; hay que acotarlo a un radio máximo, y ese tope acaba fijando el perfil tanto como el criterio de par constante.
3. **El cable no sigue cualquier perfil.** Un cable tenso sólo se asienta sobre un perfil *convexo* y de pendiente suave; en un tramo cóncavo lo puentea como una cuerda, y entonces ni el radio ni el par son los que dice el modelo. El criterio práctico es mantener el ángulo de presión $\arctan(|r'|/r)$ por debajo de unos 25°. Los perfiles de par constante con radio de arranque alto lo violan de largo, con ángulos de 60° a 80°.

Esta última condición es la restricción física de fondo, y vale para cualquier tambor de radio variable accionado por cable: no se puede pedir al perfil una variación de radio más rápida de lo que el cable puede seguir.

4.6. Método experimental: separación del par y brazo efectivo

El accionamiento entrega el par en unidades internas de escala desconocida, y el par medido incluye las pérdidas por rozamiento. Para comparar con el modelo sin depender de esa escala se recorre el mismo punto en los dos sentidos, a la misma velocidad. Las pérdidas se oponen siempre al movimiento, mientras que el par gravitatorio conserva el signo:

$$\tau_g = \frac{|\tau_\uparrow| + |\tau_\downarrow|}{2}, \qquad \tau_f = \frac{|\tau_\uparrow| - |\tau_\downarrow|}{2}. \tag{4.13}$$

El mismo promedio aplicado a la señal de la célula elimina en primera aproximación el retardo de su filtro, que desplaza en sentidos opuestos las curvas de ida y de vuelta.

Con τ_g limpio de pérdidas, (4.10) permite estimar el *brazo efectivo*

$$p_{\mathrm{ef}}(\varphi) \;\propto\; \frac{\tau_g(\varphi)}{T(\varphi)}, \tag{4.14}$$

donde T se toma del modelo, que no necesita la célula de carga: sólo la geometría y la masa. En un tambor cilíndrico p_{ef} sería constante; su forma es, por tanto, una medida directa del perfil que el cable está recorriendo.

4.6.1. Por qué no basta con medir la fuerza

La célula no está calibrada con masas patrón, de modo que la comparación con el modelo se hace ajustando una escala. El coeficiente de determinación de ese ajuste es el cuadrado del coeficiente de correlación, $R^2 = \rho^2$, y *no cambia si se invierte el signo de la escala*: una geometría que prediga una tensión creciente y otra que la prediga decreciente pueden puntuar igual. La medida de fuerza, por sí sola, no puede decidir en qué sentido trabaja el mecanismo.

El par del accionamiento sí puede, porque su signo y su escala son propios del equipo y no dependen de ninguna calibración pendiente. Por eso el criterio del capítulo 11 es comparar τ_g con el par del modelo ajustando *una sola* constante positiva k —el paso de unidades internas a newton-metro— y medir

$$R^2 = 1 - \frac{\sum_i \left(\tau_{g,i} - k\,\tau_i^{\mathrm{mod}}\right)^2}{\sum_i \left(\tau_{g,i} - \bar{\tau}_g\right)^2}, \qquad k > 0. \tag{4.15}$$

Usar en su lugar ρ^2 premia a un modelo que prediga la tendencia contraria a la medida, y la sección 11.6 muestra que la diferencia no es académica.

4.7. Limitaciones del modelo

- Las cotas de la tabla 4.1 proceden del CAD y de medidas sobre el banco, no de una metrología cuidada. e_2 es el parámetro más sensible y el que conviene medir mejor, a través de θ_{libre}.
- Se supone que el cable trabaja en una sola capa y que se asienta sobre el perfil. Con varias capas el brazo crece con el espesor acumulado, y en tramos de pendiente fuerte el cable puentea el perfil.
- Se desprecian la elasticidad del cable y la masa de la barra. Con cuerda sintética la hipótesis de cable inextensible sólo vale a corto plazo, entre retensados.
- El rendimiento y las pérdidas se suponen iguales en los dos sentidos, en valor absoluto, para poder aplicar (4.13).
- El modelo es estático: no describe el tramo inicial con el cable destensado ni los transitorios de arranque e inversión.

Capítulo 5

Requisitos y arquitectura del banco

5.1. Requisitos

Los requisitos salen de lo que exige el ensayo de la caracola (capítulo 4). El eje debe moverse a velocidad constante y conocida en los dos sentidos, y hay que registrar simultáneamente, con la misma base de tiempos, la posición, la tensión del cable y el par del motor. Todo ello de forma segura y repetible.

5.1.1. Requisitos funcionales

Id.	Requisito
RF1	Configurar y habilitar el accionamiento y gobernar su velocidad por Modbus RTU sobre RS-485.
RF2	Leer la velocidad y el par del motor en cada periodo de control.
RF3	Leer la tensión del cable con una célula de carga y un acondicionador digital.
RF4	Detectar los finales de carrera A (arriba) y B (abajo) y detener el movimiento hacia un final activado.
RF5	Ejecutar una rutina de *homing* que localice A, mida el recorrido hasta B y fije el origen en un desplazamiento configurable.
RF6	Ejecutar ciclos de ensayo B→A→B: colocarse en el extremo inferior, subir hasta A y volver a bajar, con velocidad y número de repeticiones configurables.
RF7	Enviar al ordenador la telemetría de cada periodo con su marca de tiempo del dispositivo.
RF8	Recibir órdenes y parámetros desde el ordenador con detección de errores.
RF9	Consultar y programar la ganancia y el cero del acondicionador de la célula, sólo con el eje parado.
RF10	Mostrar el estado de la máquina y los datos en tiempo real en un navegador.
RF11	Grabar automáticamente cada ensayo en un fichero con datos brutos y constantes de conversión.
RF12	Visualizar los ensayos grabados y compararlos con el modelo teórico.
RF13	Mantener una única configuración de la geometría del mecanismo, editable desde la interfaz y compartida por el simulador y el visor de ensayos.

5.1.2. Requisitos no funcionales

Id.	Requisito
RNF1	**Determinismo**: periodo de control y muestreo fijo de 50 ms, con dispersión inferior a 1 ms.
RNF2	**Latencia de parada**: una orden de parada, un final de carrera o el límite de fuerza deben anular la consigna en el mismo periodo en que se detectan.
RNF3	**Robustez de la comunicación**: una trama corrupta no debe ejecutar ninguna orden, y el receptor debe resincronizarse solo.
RNF4	**Recuperación**: la aplicación debe reconectarse sola si se desconecta o reinicia el microcontrolador.
RNF5	**Reproducibilidad**: cualquier resultado debe poder recalcularse a partir de los ficheros grabados.
RNF6	**Coste y disponibilidad**: componentes comerciales y herramientas libres.
RNF7	**Mantenibilidad**: validación de cada subsistema con un programa de prueba propio y código documentado.
RNF8	**Coherencia del modelo**: una sola implementación de la geometría para todas las páginas, para que el simulador y el análisis no puedan discrepar.

5.2. Arquitectura general

La figura 5.1 muestra la arquitectura, organizada en tres niveles:

1. **Nivel de campo**: el servomotor AC con su accionamiento, la transmisión, la caracola, el cable y la barra con la masa; la célula de carga intercalada en el cable y los dos finales de carrera.
2. **Nivel de control**: la placa adaptadora con la Raspberry Pi Pico 2. Ejecuta el lazo de 50 ms y es el único elemento que actúa sobre la máquina.
3. **Nivel de supervisión**: un ordenador con el servidor `web_monitor.py`, que traduce entre el protocolo binario del puerto USB y mensajes JSON por WebSocket, graba los ensayos, guarda la configuración del mecanismo y sirve las tres páginas web.

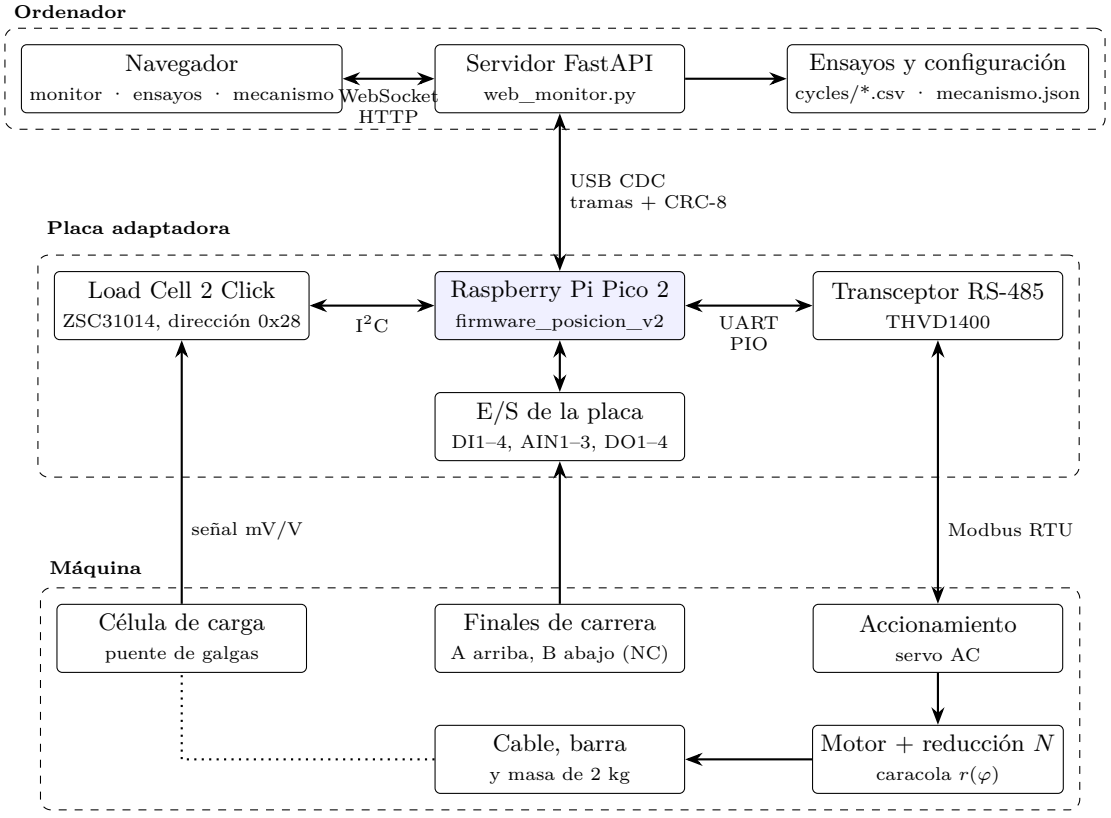


Figura 5.1: Arquitectura del banco de ensayos.

5.3. Flujo de datos y temporización

El microcontrolador ejecuta un lazo de periodo fijo (sección 7.2). En cada vuelta lee el estado del servo y de la célula, integra la posición, procesa las órdenes recibidas, actualiza las máquinas de estado de *homing* y de ciclo, aplica las protecciones y escribe la nueva consigna; además guarda una muestra de la vuelta en un búfer. Cada 200 ms (o antes, si el búfer se llena) el búfer se envía en una sola trama, con una marca de tiempo por muestra. El servidor desempaqueta las muestras, las reenvía al navegador y, si hay un ensayo en marcha, las escribe en su fichero. Toda la cadena usa el reloj del microcontrolador, de modo que las gráficas y los ficheros reflejan los instantes reales de medida y no los de llegada al ordenador.

La geometría del mecanismo sigue el camino contrario: se edita en la página de configuración, se valida y se guarda en el servidor (`mecanismo.json`), y desde ahí la leen tanto el simulador como el visor de ensayos, de modo que el modelo con el que se compara una medida es siempre el mismo.

5.4. Decisiones de diseño

Tabla 5.3: Decisiones de diseño y alternativas consideradas.

Aspecto	Elección	Justificación / alternativa descartada
Microcontrolador	Raspberry Pi Pico 2	Sustituye al ESP32-S3 inicial. La placa se diseñó para el módulo Pico, y su PIO permite una UART en los pines que imponía el trazado.

Aspecto	Elección	Justificación / alternativa descartada
Modo del servo	Velocidad	Los primeros firmwares usaban el modo par con un regulador sobre la célula. El ensayo cuasiestático y el método de separación del par (sección 4.6) exigen velocidad constante e igual en ambos sentidos.
Organización del firmware	Un único dueño de cada bus en el núcleo 0	La versión de doble núcleo dedicaba el núcleo 1 a Modbus con variables compartidas. El lazo único de periodo fijo elimina las condiciones de carrera y hace predecible la temporización.
Origen del ciclo	Arranca en B, el extremo inferior	El ensayo tiene sentido físico desde la extensión completa del mecanismo. Como el *homing* deja el eje en A, el ciclo se coloca primero en B, y ese tramo ni cuenta como repetición ni se graba.
Extremos del ciclo	Finales de carrera	Reconocer A y B por los finales, y no por la posición integrada, hace que el ciclo funcione esté donde esté el cero y sin depender de la escala de velocidad del accionamiento.
Protocolo con el PC	Binario con CRC-8 y telemetría por lotes	Un protocolo de texto ocupa más y es ambiguo de delimitar. Las tramas binarias con cabecera, longitud y CRC se resincronizan solas.
Posición	Integración de la velocidad del servo	No requiere cablear el codificador. Su deriva se acota con el *homing* y los finales de carrera (sección 7.5).
Células de carga	Una sola (ZSC31014)	El NAU7802 se evaluó como segunda célula y se retiró: compartían bus y el ZSC31014 no tolera transacciones ajenas entre la petición y la recogida del dato.
Supervisión	Aplicación web local	No requiere licencias, admite varios clientes y separa la operación en vivo del análisis.
Modelo del mecanismo	Una implementación única en el servidor	Cada página tenía su copia de las fórmulas y llegaron a discrepar: el monitor seguía con un cilindro mientras el visor usaba la caracola. Ahora la geometría vive en `mecanismo.json` y las fórmulas en `mecanismo.js`.
Datos grabados	Cuentas brutas + constantes en la cabecera	Permite recalibrar un ensayo sin repetirlo, que es justo lo que ha hecho falta.

5.5. Seguridad

El prototipo mueve una masa pequeña a baja velocidad, pero el sistema incluye varias medidas de seguridad, todas implementadas en el firmware:

- **Finales de carrera normalmente cerrados** conectados entre la entrada y masa, con *pull-up* interna. En reposo la entrada lee nivel bajo, y un final pulsado o un cable roto dan nivel alto: la rotura de cable se interpreta como final activado (comportamiento a prueba de fallos).
- **Parada hacia el final activado**: en cada periodo se anula cualquier consigna que empuje hacia un final pulsado, sea cual sea el modo (manual, *homing* o ciclo).
- **Límite de fuerza** configurable: si el valor absoluto de la célula tarada supera el umbral, la consigna se anula. Sólo actúa con una lectura válida reciente (menos de 150 ms).
- **Temporizadores de tramo** de 30 s en el *homing* y en los ciclos.

- **Bloqueo durante la programación de la célula**, que la deja sin medida durante cientos de milisegundos: se rechaza si el eje no está parado y, mientras dura, se fuerza la consigna a cero en cada vuelta.
- **Orden de parada** que corta también el *homing* y los ciclos, y orden de apagado que deshabilita el accionamiento.

Estas medidas dependen del software y del enlace Modbus. En una máquina industrial, las normas de seguridad funcional exigirían una cadena de parada independiente del control [17], por ejemplo cableando los finales de carrera a la entrada de desconexión segura del par (STO) del accionamiento [15]. Se recoge como trabajo futuro en el capítulo 13.

Capítulo 6

Diseño hardware

6.1. Placa adaptadora: objetivos

El módulo Raspberry Pi Pico 2 trabaja a 3,3 V y no puede conectarse directamente al entorno de la máquina. La placa adaptadora (proyecto KiCad `tfg_adaptador`) resuelve esa adaptación y reúne en un solo circuito:

- un transceptor RS-485 semidúplex para el bus Modbus del accionamiento;
- cuatro entradas digitales protegidas para finales de carrera y otros contactos;
- entradas analógicas de 0 V a 5 V acondicionadas para el ADC del microcontrolador;
- cuatro salidas de potencia de lado alto con diagnóstico;
- dos zócalos mikroBUS para placas Click de célula de carga;
- bornas de tornillo para todo el cableado de campo y taladros de fijación M3.

6.2. Revisiones del diseño

El repositorio contiene dos estados del diseño, que conviene distinguir:

- **Revisión A**: los ficheros `tfg_adaptador.kicad_sch` y `.kicad_pcb`, creados entre el 12 y el 17 de marzo de 2026. Tienen el esquema, la colocación de los componentes y los planos de cobre (figuras 6.2 y 6.3).
- **Revisión B**: el esquema de la figura 6.1, incorporado al repositorio como imagen. Es la asignación de pines que usan todos los programas de prueba de la placa y el firmware final [**VERIFICAR:** añadir al repositorio los ficheros KiCad de la revisión B, que no están versionados, y regenerar las figuras de la PCB].

El análisis de la lista de conexiones de la revisión A, extraída con `kicad-cli`, muestra una serie de problemas que la revisión B corrige (tabla 6.1). Documentarlos es útil, porque son errores típicos de un primer diseño.

Tabla 6.1: Problemas detectados en la revisión A y corrección en la revisión B.

Bloque	**Revisión A (lista de conexiones)**	**Revisión B**
RS-485	La salida RO del transceptor está en la misma red que 3,3 V y GPIO13. DI, DE y $\overline{\text{RE}}$ están unidas entre sí y a GPIO14 y GPIO15. El transceptor no puede funcionar en semidúplex.	DE y $\overline{\text{RE}}$ a GPIO12 con *pull-down* de 10 kΩ; RO a GPIO13 con *pull-up* de 10 kΩ; DI a GPIO14.
Entradas analógicas	Las salidas de los seguidores llegan a GPIO16–19, que no tienen ADC.	Salidas a GPIO26–28 (ADC0–2).
Amplificadores	Cuatro MCP6001 sin alimentación: pines V+ y V− sin conectar.	Un MCP6294 cuádruple alimentado a 5 V con desacoplo de 100 nF.
Salidas de potencia	El TPS4H000 se alimenta de VBUS (5 V del USB); diagnóstico ($\overline{\text{ST}}$), DIAG_EN, CL y THER sin conectar; entradas directas.	Alimentación desde la borna VS_IN; $\overline{\text{ST1}}$–$\overline{\text{ST4}}$ unidas a DST con *pull-up*; DIAG_EN controlado; resistencias de 10 kΩ en serie con las entradas.
Células de carga	Cada zócalo en un bus I^2C distinto (GPIO0/1 y GPIO2/3), sin control de EN ni lectura de INT.	Ambos zócalos en I^2C1 (GPIO2/3), habilitación EN1 (GPIO1) e interrupciones INT1 (GPIO0) e INT2 (GPIO4).
Entradas digitales	Llegan a GPIO4–7.	Llegan a GPIO16–19.
Rutado	El fichero de PCB no contiene pistas (0 segmentos): sólo colocación, 23 vías y planos.	[**COMPLETAR:** indicar cómo se fabricó la placa definitiva: PCB de revisión B, placa perforada o cableado manual]

6.3. Microcontrolador

La placa aloja el módulo Raspberry Pi Pico 2 en sus dos filas de 20 pines, de modo que puede sustituirse sin desoldar. El módulo se alimenta y comunica por USB. De él se toman 5 V (VBUS) para los amplificadores y 3,3 V (salida 3V3 del regulador del módulo) para el transceptor y las placas Click. La asignación completa de pines está en el anexo A.

6.4. Interfaz RS-485

Se usa el transceptor THVD1400 de Texas Instruments [47]: semidúplex, alimentación de 3 V a 5,5 V, hasta 500 kbit/s, 1/8 de carga unitaria (hasta 256 nodos), protección ESD de ±16 kV HBM y ±12 kV IEC 61000-4-2 por contacto, y protección ante fallos de bus de ±16 V. Esta robustez es adecuada para un cable que convive con los de potencia del motor.

Las señales DE y $\overline{\text{RE}}$ están unidas y se controlan desde GPIO12. Una resistencia de 10 kΩ a masa deja el transceptor en recepción durante el arranque, cuando el pin aún no está configurado, para que no ocupe el bus. La salida RO lleva una *pull-up* de 10 kΩ para que la línea de recepción no quede flotando. El bus sale por la borna de tres polos J5 (A, B y masa). El firmware activa el emisor antes de cada trama y lo desactiva 200 µs después del último byte (sección 10.1). No hay terminación en la placa: el enlace es punto a punto y corto [**VERIFICAR:** longitud del cable y si el accionamiento incorpora terminación de 120 Ω].

6.5. Entradas digitales

Cada entrada digital (DI1–DI4) pasa por una resistencia serie de 1 kΩ y un diodo zéner BZX384 a masa [37] antes de llegar al GPIO (figura 6.1). La resistencia limita la corriente y el zéner recorta las sobretensiones y las descargas que entran por el cable. Los finales de carrera, de contacto normalmente cerrado, se conectan entre la borna y masa, y el firmware activa la *pull-up* interna del GPIO. En el banco, DI1 es el final A (arriba, donde queda el *homing*) y DI2 el final B (abajo, donde arranca el ensayo).

La tensión que admite la entrada de forma continua está limitada por la potencia de la resistencia. Con un zéner de tensión V_Z y una resistencia de 0,125 W (encapsulado 0805), la tensión de entrada máxima es

$$V_{\text{in,max}} = V_Z + \sqrt{P_R\, R} \approx V_Z + \sqrt{0{,}125 \cdot 1000} \approx V_Z + 11{,}2\,\text{V}. \tag{6.1}$$

Para contactos secos es más que suficiente. Si se quisieran conectar sensores de 24 V, habría que subir la resistencia a unos 10 kΩ **[COMPLETAR:** tensión nominal del zéner BZX384 montado].

6.6. Entradas analógicas

Las entradas AIN1–AIN3 admiten 0 V a 5 V. Cada canal tiene un divisor resistivo de 17 kΩ y 33 kΩ, con relación $33/50 = 0{,}66$, que lleva 5 V a 3,3 V, seguido de un amplificador operacional en seguidor de tensión. El seguidor presenta al ADC una impedancia de fuente baja, necesaria para cargar el condensador de muestreo en cada conversión sin error, y aísla el divisor del ADC. Se usa el MCP6294 [30], cuádruple, *rail-to-rail* en entrada y salida y alimentado a 5 V.

Con el ADC de 12 bits del RP2350 y referencia de 3,3 V, la resolución referida a la entrada es

$$\Delta V_{\text{in}} = \frac{3{,}3\,\text{V}}{4095 \cdot 0{,}66} \approx 1{,}22\,\text{mV}. \tag{6.2}$$

El cuarto amplificador se usa como comparador con un umbral de 2,5 V, obtenido de 5 V con un divisor de dos resistencias de 10 kΩ. Su salida se adapta a 3,3 V con otro divisor 17/33 y llega a una entrada digital (COMP).

El valor de 17 kΩ no pertenece a la serie E24. Para la fabricación conviene usar 17,4 kΩ (E96), con relación 0,655, o recalibrar el factor en el firmware.

6.7. Salidas de potencia

Las cuatro salidas usan el interruptor inteligente de lado alto TPS4H000-Q1 [48]: cuatro canales, alimentación de 3,4 V a 40 V, 1000 mΩ de resistencia en conducción, límite de corriente ajustable entre 0,1 y 1,3 A, detección de sobrecorriente, cortocircuito y carga abierta, protección térmica y capacidad para cargas inductivas. Las entradas IN1–IN4 se gobiernan desde GPIO7–10 a través de resistencias de 10 kΩ. Las salidas de estado, en drenador abierto, se unen en una línea común de fallo (DST) con *pull-up*, y la habilitación del diagnóstico se controla con DIAG_EN. El firmware final no usa todavía estas salidas; su funcionamiento se verificó con el programa `pcbtest/test_digital_outputs` **[COMPLETAR:** uso previsto de las salidas y resistencia de límite de corriente elegida].

6.8. Célula de carga

La célula de carga se conecta a una placa *Load Cell 2 Click* de MikroElektronika [31], basada en el acondicionador ZSC31014 [43], que se inserta en el zócalo mikroBUS MD1. El chip responde en la dirección I^2C `0x28`, admite alimentación de 2,7 V a 5,5 V y ofrece ocho ganancias de preamplificador (×1,5 a ×192). Su señal EN, controlada desde GPIO1 con la corriente de salida del pin configurada a 12 mA, permite apagarlo y encenderlo para forzar un reinicio y abrir la ventana del modo comando. Esta posibilidad resultó imprescindible, y también problemática (sección 10.2).

El segundo zócalo (MD2) comparte el bus y se usó para evaluar una *Load Cell 4 Click* con NAU7802 [38] [**COMPLETAR:** modelo, capacidad nominal y sensibilidad (mV/V) de la célula de carga, y forma de montaje en el cable].

6.9. Alimentación

La lógica se alimenta del USB del ordenador a través del módulo Pico. Las salidas de potencia toman su tensión de la borna VS_IN y el accionamiento tiene su propia fuente [**COMPLETAR:** tensión y potencia de la fuente del accionamiento]. Las masas se unen en la placa, y la masa del bus RS-485 se lleva también a la borna J5 para dar al transceptor una referencia común con el accionamiento.

6.10. Placa de circuito impreso

La PCB de la revisión A es de dos capas, mide 123 mm × 75 mm y tiene cuatro taladros M3 en las esquinas. Los componentes pasivos son SMD 0805, los diodos SOD-323, los amplificadores SOT-23-5, el transceptor SOIC-8 y el interruptor de potencia HTSSOP-20 con almohadilla térmica. Las bornas de campo están agrupadas en el lado izquierdo, el módulo Pico en el lado derecho con el conector USB hacia el borde, y los dos zócalos mikroBUS arriba a la derecha, de modo que las placas Click quedan accesibles.

6.11. Validación de la placa

Cada bloque se validó con un programa de prueba independiente antes de la integración (tabla 6.2).

Tabla 6.2: Pruebas de la placa adaptadora.

Programa	Procedimiento y criterio	Resultado
`pcbtest/test_analog`	Llevar AIN a GND, 3,3 V y 5 V; la tensión reconstruida debe ser 0, 3,3 y 5,0 V.	[**COMPLETAR:** lecturas]
`pcbtest/test_din`	Cambiar el estado de DIN1–4 y comprobar que se detecta cada flanco.	[**COMPLETAR:**]
`pcbtest/test_digital_outputs`	Conmutar SIN1 cada segundo y medir la salida.	[**COMPLETAR:**]
`test_limit_switches`	Pulsar cada final de carrera: nivel bajo en reposo y alto activado.	Correcto (usado en todos los ensayos)
`test_rs485_loopback`	Puente GP14–GP13 sin transceptor: debe recibirse la trama enviada a 9600 y 115200 baudios.	Correcto
`test_rs485_tx/rx`	Transmisión y recepción a través del transceptor.	Correcto
`test_modbus_scan`	Lectura periódica del registro 0 del esclavo 1 a 115200 baudios.	Correcto (código 0x00)
`test_loadcell`	Detección del ZSC31014 en 0x28 y lectura del puente con estado.	Correcto

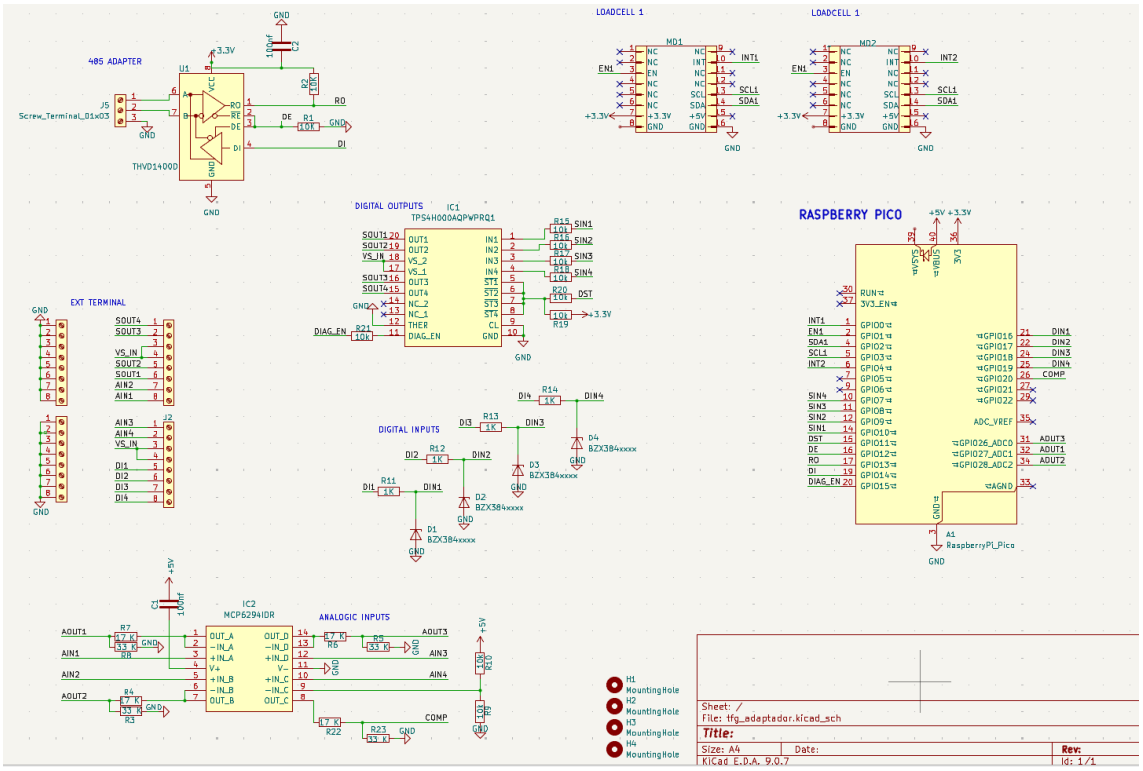


Figura 6.1: Esquema de la placa adaptadora, revisión B.

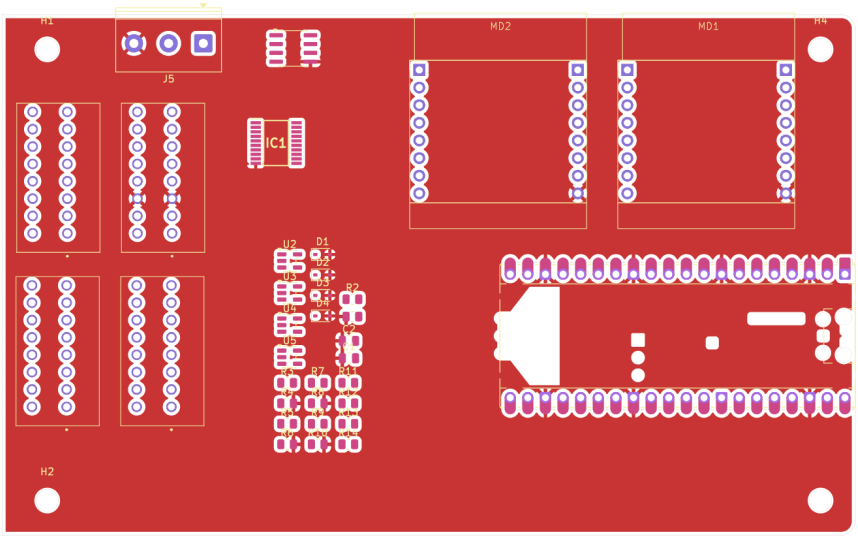


Figura 6.2: PCB de la revisión A: capas de cobre, máscara y serigrafía superiores.

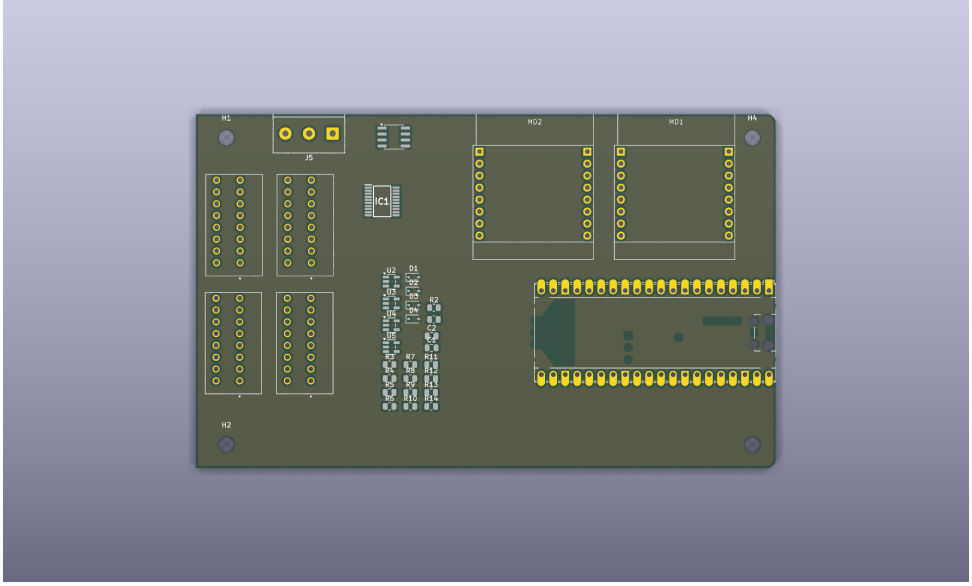


Figura 6.3: Vista 3D de la PCB de la revisión A generada con KiCad.

Capítulo 7

Firmware del microcontrolador

7.1. Entorno y organización

El firmware está escrito en C++ con el entorno Arduino y el núcleo Arduino-Pico [16], que da acceso a los periféricos del RP2350 y a su SDK. Usa las bibliotecas `Wire` (I^2C), `SerialPIO` (UART implementada con PIO) y ModbusMaster [50]. El código final está en `firmware_posicion_v2/`: un fichero principal de unas 1500 líneas y la cabecera `i2c_bus.h` con el árbitro del bus I^2C.

El diseño sigue un principio que se repite en todo el código: *cada bus tiene un único dueño y un único punto del lazo donde se usa*. Sólo dos funciones tocan el bus RS-485 (`servoReadState` y `servoAction`) y sólo dos tocan el I^2C durante el funcionamiento (`i2cUpdate` y `lcRequestService`). El intérprete de órdenes (`processCommand`) nunca accede a un bus: se limita a modificar variables de estado, que el lazo lee en el punto que corresponde. Así, la llegada de una orden no puede interrumpir una transacción a medias.

El estado vive en dos estructuras: `ControlState ctrl` (modo, errores, parámetros, célula, posición, *homing* y ciclo) y `Servo sv` (fase del enlace, dirección, conexión, contadores de fallos, consigna pendiente y últimas lecturas).

7.2. Lazo principal

El núcleo 0 ejecuta un lazo con periodo mínimo de 50 ms (`LOOP_MS`). Cada vuelta tiene cinco fases (código 7.1).

```
void loop(){
  unsigned long now = millis();  uint32_t t0us = micros();

  // == LECTURA ==
  servoReadState();      // RS-485: velocidad y par (2 transacciones)
  lcRequestService();    // consulta/programación de la célula si se ha pedido
  i2cUpdate();           // I2C: una lectura de la célula
  positionUpdate();      // integra rpm -> posición (sin buses)

  // == LÓGICA == (sin acceso a buses)
  checkCommands();       // órdenes USB -> variables de estado
  if (ctrl.lcProgBusy)      { sv.speedCmd = 0; ctrl.cycPhase = CYC_IDLE; }
  else if (homing en curso)   homingUpdate();
  else                      { homingUpdate(); cycleUpdate(); }
  // protecciones: límite de fuerza y finales de carrera anulan la consigna
  // derivación del estado de la aplicación para la telemetría

  // == ACTUACIÓN ==
  servoAction();         // RS-485: máquina de estados del servo y escritura de consigna

  // == REGISTRO ==
  recordSample(now, micros() - t0us);
  if (sampleCount >= MAX_SAMPLES || now - ctrl.lastTelemetryTime >= 200)
    telemetryFlush();    // + tramas de homing y de ciclo si están activos

```

```
26    // == RITMO ==
27    unsigned long dt = millis() - now;
28    if (dt < LOOP_MS) delay(LOOP_MS - dt);
29  }
```

Código 7.1: Estructura del lazo principal (simplificada).

Este orden tiene consecuencias deliberadas:

- Las protecciones se evalúan *después* de la lógica y *antes* de la escritura: ninguna consigna generada por el *homing*, el ciclo o una orden manual llega al accionamiento sin pasar por ellas.
- Una orden recibida se aplica en la misma vuelta, porque la escritura de la consigna es posterior a la lectura de órdenes.
- El tiempo de trabajo (`work_us`) se mide antes del retardo de ritmo y viaja con cada muestra. Es la base de la caracterización temporal de la sección 11.2.
- El retardo final hace que el periodo no dependa de la duración del trabajo mientras ésta no supere 50 ms.

El núcleo 1 no se usa (`loop1()` sólo espera). La evolución desde la versión de doble núcleo se explica en la sección 10.1.

7.3. Máquina de estados del servo

La figura 7.1 muestra la máquina de estados del enlace con el accionamiento, que ejecuta `servoAction`.

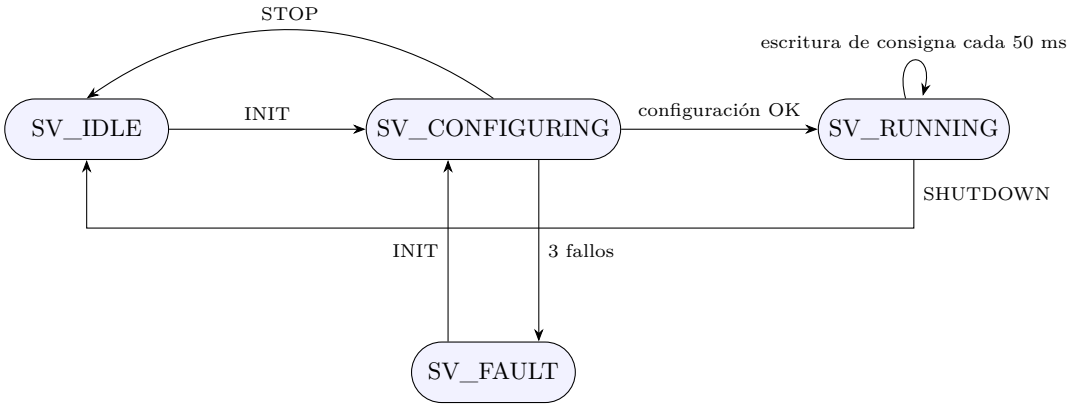


Figura 7.1: Máquina de estados del enlace con el accionamiento.

En `SV_CONFIGURING` se ejecuta la secuencia de configuración del capítulo 8. En `SV_RUNNING` cada vuelta escribe la consigna: un fallo marca el servo como desconectado y pone a cero las lecturas, pero no provoca un nuevo escaneo del bus, que es bloqueante y lento (hasta un segundo por dirección vacía). El escaneo sólo se hace bajo demanda con la orden SCAN.

La orden INIT, además de reiniciar la posición, invalida el recorrido A→B medido: poner el cero donde esté el eje deja sin sentido la referencia anterior, y sin esa invalidación el ciclo podría creer que A es el punto actual.

7.4. Lectura de la célula de carga

El ZSC31014 trabaja en modo de conversión continua: convierte a su propio ritmo, fijado por la palabra ZMDI_Config1, de 0,5 a 125 ms según el reloj y la tasa elegidos [43], y guarda el último resultado. Una lectura no dispara una conversión: devuelve el último dato con dos bits de estado que indican si es nuevo o si ya se había leído.

Por ello, `loadCellUpdate` hace *una sola lectura de dos bytes por vuelta*:

1. Pide el bus al árbitro. Si está ocupado, pierde el turno y lo intenta en la vuelta siguiente.
2. Lee 14 bits de dato y 2 de estado. Si el estado es "dato válido", lo introduce en una media móvil de 8 posiciones y calcula la lectura tarada `baseRead = media − zeroOffset`.
3. Si pasan más de 150 ms sin un dato válido, marca la célula como no válida y el límite de fuerza deja de actuar, en lugar de hacerlo con un dato viejo.

7.4.1. Árbitro del bus I²C

La cabecera `i2c_bus.h` implementa un árbitro mínimo: una variable con el dueño actual (`I2C_FREE`, `I2C_LC1`, `I2C_LC2` o `I2C_CFG`) y dos funciones, `i2cAcquire` e `i2cRelease`. La reserva se deniega si el bus tiene otro dueño o si se pide desde el núcleo 1, y cada denegación se cuenta para el diagnóstico. El árbitro nació al compartir el bus con el NAU7802 (sección 10.8) y se conserva para reservar el bus en exclusiva durante la programación de la EEPROM. Nadie espera con el bus ocupado, así que el árbitro no puede bloquear el lazo.

7.5. Posición integrada

La posición del eje del motor se obtiene integrando la velocidad que devuelve el accionamiento:

$$p_k = p_{k-1} + \frac{n_k\,\Delta t_k}{60000}, \qquad [p] = \text{rev},\ [n] = \text{rpm},\ [\Delta t] = \text{ms}. \tag{7.1}$$

Es una integración rectangular con la última velocidad leída y el intervalo real medido con `millis()`. Hay tres aspectos que tener en cuenta:

- **Rebase tras un hueco.** Si el lazo se detiene, por ejemplo durante la programación de la célula, la siguiente vuelta vería un Δt de casi un segundo y lo multiplicaría por la última velocidad, inventando un desplazamiento que no ha ocurrido. Por eso se anula la base de tiempo (`lastPosTime = 0`) tras INIT, al iniciar el *homing* y tras cualquier operación bloqueante.
- **Resolución.** El registro de velocidad tiene resolución de 1 rpm: si el accionamiento redondea, el error es de media nula; si trunca, a 30 rpm introduciría un sesgo de hasta el 3 %.
- **Acotación.** La deriva no se acumula entre ensayos, porque el ciclo reconoce sus extremos por los finales de carrera y no por la posición integrada.

Esta última decisión es la que hace que el banco siga siendo utilizable pese a la incertidumbre de la posición: el recorrido del ensayo queda definido por dos contactos físicos, y la posición integrada sólo sirve como eje de referencia dentro del ensayo.

7.6. *Homing*

La rutina de *homing* (figura 7.2) establece un origen repetible y mide el recorrido disponible:

1. **Buscar A**: avanza hacia A (arriba) a la velocidad de *homing* hasta que se activa el final A, y fija ahí $p = 0$.
2. **Buscar B**: avanza hacia B (abajo) hasta el final B y guarda el recorrido p_B.
3. **Ir al origen**: se mueve hasta el desplazamiento configurado, acotado a $[0, p_B]$, con tolerancia de ±0,02 rev.
4. Fija ese punto como nuevo origen ($p = 0$) y sigue informando del resultado durante 1,5 s para que la interfaz lo muestre.

Cualquier tramo que dure más de 30 s, o la pérdida del servo, aborta con fallo. Durante el *homing* se ignoran las órdenes de movimiento manual.

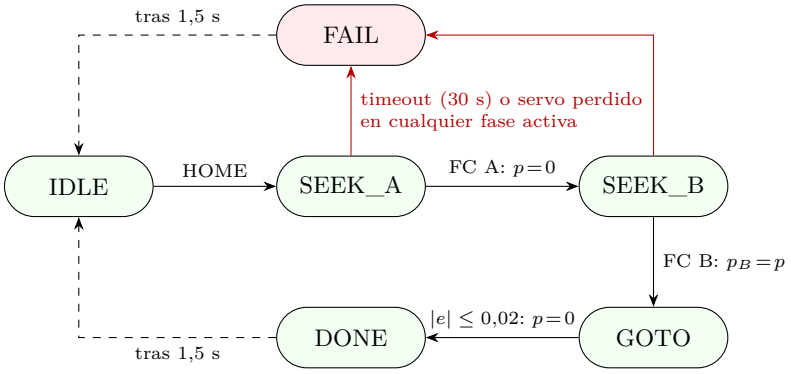


Figura 7.2: Máquina de estados del *homing*.

7.7. Ciclo de ensayo B→A→B

El ensayo tiene sentido físico empezando por el extremo inferior, con el mecanismo en extensión completa y la masa en su punto más bajo. Como el *homing* deja el eje en A, el ciclo se organiza en tres fases (figura 7.3):

`CYC_POS_B` — colocación
Baja hasta B. Este tramo no cuenta como repetición y el servidor no lo graba: el ensayo empieza cuando el eje ya está en B. Al llegar se guarda `cycPosB`, la posición integrada en ese punto, que será el origen de medida del ensayo.

`CYC_TO_A` — ida
Sube hasta A. Es el tramo cargado del ensayo.

`CYC_TO_B` — vuelta
Baja de nuevo hasta B y cierra la repetición; si quedan repeticiones, se vuelve a `CYC_TO_A`.

Por convenio, la consigna negativa lleva el eje hacia A (arriba) y la positiva hacia B (abajo).

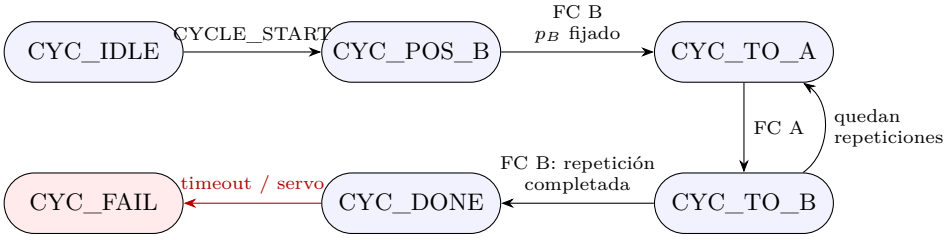


Figura 7.3: Máquina de estados del ciclo de ensayo.

Los extremos se reconocen por los *finales de carrera*, no por la posición integrada. Así el ciclo funciona esté el cero donde esté —el *homing* puede haberlo dejado en A, en B o en medio— y sin depender de que la escala de velocidad del accionamiento sea exacta, que es de donde sale la posición. Si se pide un recorrido parcial, la distancia se mide *desde B*:

```
static bool cycleAtB(bool limB){            // B: el final de abajo
  if(ctrl.cycToLimitB) return limB;
  return limB || (ctrl.posRev >= ctrl.cycPosB - HOMING_TOL_REV);
}
static bool cycleAtA(bool limA){            // A: el final de arriba, o la distancia pedida
  if(ctrl.cycToLimitB) return limA;
  return limA || ((ctrl.cycPosB - ctrl.posRev) >= (ctrl.cycTargetRev - HOMING_TOL_REV));
}
```

Código 7.2: Reconocimiento de los extremos del ciclo.

El ciclo sólo se acepta con el servo habilitado, sin *homing* en curso y sin programación de la célula. Los temporizadores y la pérdida del servo lo abortan, y los finales de carrera y el límite de fuerza siguen actuando en el lazo como red de seguridad. La prioridad es: programación de la célula > *homing* > ciclo > movimiento manual.

7.8. Configuración del acondicionador de la célula

La consulta y la programación de la EEPROM del ZSC31014 son las operaciones más delicadas del firmware. Por las restricciones que se analizan en la sección 10.2, obligan a reiniciar el chip y dejan la célula sin medida durante cientos de milisegundos. Se ejecutan así:

1. `processCommand` sólo *anota* la petición y un plazo de 1 s.
2. En su punto fijo del lazo, `lcRequestService` comprueba que el eje está en reposo: estado IDLE o STOPPED, consigna nula y sin *homing*. Si no lo está, responde "rechazado".
3. Reserva el bus en exclusiva (`I2C_CFG`). Si no lo consigue antes del plazo, responde "bus ocupado".
4. Pone la consigna a cero, pasa al estado PROGRAMMING y envía la telemetría pendiente para que la interfaz vea el silencio que viene.
5. Ejecuta la consulta o la programación, devuelve el bus a 400 kHz, libera el árbitro, reinicializa la célula, anula la base de la integración de posición y responde con el paquete `0x04`.

7.8.1. Escritura verificada

`loadCellProgram` aplica dos reglas de la hoja de datos [43]: la palabra ZMDI_Config1 sólo se carga tras un ciclo de alimentación, y la firma MISR de la EEPROM (palabra `0x12`) se regenera al salir del modo comando con Start_NOM si hubo escrituras. La función hace dos pasadas:

1. **Primera pasada**: entra en modo comando, lee B_Config (`0x0F`) y ZMDI_Config1 (`0x01`) y calcula las palabras deseadas. Si coinciden con las actuales, termina sin escribir, porque la EEPROM admite 100.000 ciclos de escritura. Si no, escribe las que cambian, espera 20 ms por escritura (típico 15 ms, peor caso 17,3 ms), envía Start_NOM y espera 150 ms antes de cortar la alimentación.
2. **Segunda pasada**: desde un reinicio limpio, vuelve a leer ambas palabras. Si coinciden con lo pedido, el resultado es "grabado y verificado"; si no, "fallo".

Las palabras deseadas se construyen a partir de las actuales, protegiendo los bits que la interfaz no debe tocar: en B_Config se conservan los reservados [15:13]; en ZMDI_Config1 se conservan [15:9] (signos de las correcciones de segundo orden y de temperatura, calibrados en fábrica) y se fuerzan el interfaz I^2C (bit 4 = 0) y el campo reservado [2:0] = 001.

7.8.2. Mantenimiento del reinicio para medida

La orden LC_HOLD deja la célula sin alimentación durante 8 s, con las líneas del bus en el estado elegido, para medir con un polímetro la tensión de la placa Click. A diferencia de la programación, esta espera *no* bloquea: el lazo sigue girando, leyendo el servo, vigilando los finales de carrera y enviando telemetría, y en cada vuelta sólo comprueba si ha vencido el plazo. Mientras dura se fuerza la consigna a cero en cada vuelta: alguien tiene las manos dentro de la máquina, y una orden de movimiento que llegue por USB no debe poder arrancar el eje.

7.9. Evolución del firmware

Tabla 7.1: Versiones del firmware conservadas en el repositorio.

Versión	Placa	Características
`old/speedtorque_mode`	XIAO ESP32-S3	Escaneo Modbus y selección de modo par o velocidad.
`firmware`	XIAO ESP32-S3	Protocolo binario v2, parámetros del accionamiento y regulador de fuerza en modo par con zona muerta, umbral y ganancia sobre la célula filtrada.
`firmwarerasp`	Pico 2	Migración con SerialPIO. Doble núcleo: el núcleo 1 escribe la consigna y lee la supervisión Modbus, comunicado con el núcleo 0 por variables `volatile`.
`firmware_posicion`	Pico 2	Modo velocidad sin lazo de fuerza, integración de posición, telemetría por lotes y diseño de dueño único en el núcleo 0.
`firmware_posicion_v2`	Pico 2	Versión final: *homing*, ciclo B→A→B con extremos por finales de carrera, programación verificada de la EEPROM, árbitro I^2C y diagnóstico en la telemetría.

Capítulo 8

Comunicaciones

El sistema usa cuatro enlaces (tabla 8.1). Este capítulo describe el protocolo binario propio entre el microcontrolador y el ordenador, el uso de Modbus RTU con el accionamiento y la interfaz entre el servidor y el navegador. La especificación byte a byte está en el anexo B.

Tabla 8.1: Enlaces de comunicación del sistema.

Enlace	Extremos	Capa física	Protocolo
USB	PC ↔ Pico	USB CDC (puerto serie virtual)	Binario propio + CRC-8
RS-485	Pico ↔ accionamiento	UART PIO, 115200 8N1	Modbus RTU
I^2C1	Pico ↔ ZSC31014	400 kHz (100 kHz en modo comando)	Propio del chip
Red local	Navegador ↔ servidor	TCP, puerto 8080	HTTP + WebSocket (JSON)

8.1. Protocolo binario entre el microcontrolador y el ordenador

8.1.1. Formato de trama

Todas las tramas, en ambos sentidos, tienen el mismo formato (figura 8.1): dos bytes de sincronía `0xAA 0x55`, un byte de longitud, la carga útil y un CRC-8 calculado sobre la carga útil. El primer byte de la carga útil identifica el tipo de paquete o la orden. Los enteros multibyte van en orden *little-endian*.

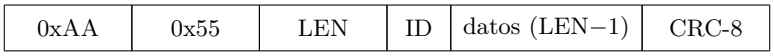


Figura 8.1: Formato común de trama. El CRC-8 cubre los LEN bytes de la carga útil (ID + datos).

El CRC usa el polinomio $x^8 + x^2 + x + 1$ (`0x07`), valor inicial `0x00`, sin reflexión ni XOR final (la variante conocida como CRC-8/SMBUS). Se implementa bit a bit en el firmware y en Python con el mismo algoritmo. Para tramas cortas como éstas detecta todos los errores de uno y dos bits y todas las ráfagas de hasta 8 bits.

8.1.2. Recepción y resincronización

El firmware decodifica las órdenes con una máquina de cuatro estados (`RX_SYNC0`, `RX_SYNC1`, `RX_LEN`, `RX_PAYLOAD`). Una longitud nula o mayor de 60 devuelve el decodificador a la búsqueda de sincronía, y una trama con CRC incorrecto se descarta sin ejecutarse. Si se pierden bytes, el receptor vuelve a encontrar el patrón `AA 55` en la siguiente trama. El servidor hace lo mismo con la telemetría, con un tiempo de espera de 100 ms por lectura.

Para depurar desde un terminal, el firmware acepta también órdenes de una letra (`i`, `a`, `b`, `x`, `h`, `q`), pero *sólo* cuando el decodificador está esperando el primer byte de sincronía. Antes se comprobaban en cualquier estado, y eso provocaba un error difícil de ver: la orden de programar la célula con ganancia 7 y cero 8 genera la trama `AA 55 03 09 07 08 69`, cuyo CRC `0x69` coincide con el carácter `i`. El atajo se comía el último byte, la orden se perdía y además se reinicializaba el servo. Con ganancia 6 el CRC vale `0x7C` y la misma orden funcionaba. Ambos valores se han comprobado con la implementación del CRC.

8.1.3. Paquetes del microcontrolador al ordenador

Tabla 8.2: Paquetes de telemetría y estado.

ID	Nombre	Tamaño de carga útil	Cuándo se envía
`0x02`	Telemetría por lotes	$14 + 14n$ B, $n \leq 8$	cada 200 ms o con 8 muestras
`0x03`	Estado de *homing*	11 B	junto al lote, mientras hay *homing*
`0x04`	Resultado de la célula	13 B	una vez por consulta o programación
`0x05`	Estado del ciclo	18 B	junto al lote, mientras hay ciclo

Telemetría por lotes. Cada vuelta del lazo añade al búfer una muestra de 14 bytes: desfase en milisegundos respecto al instante base del lote, velocidad, par, consigna, fuerza tarada, estado de los finales de carrera, estado de la máquina y tiempo de trabajo de la vuelta. Los datos comunes (instante base de 32 bits, lectura bruta de la célula y su estado, indicadores, código de error, dirección del servo y códigos de resultado de las dos lecturas Modbus) viajan una sola vez en una cabecera de 14 bytes.

Agrupar las muestras reduce las escrituras por USB a cinco por segundo sin perder resolución temporal, porque cada muestra conserva su instante exacto. Con el lazo de 50 ms cada lote lleva típicamente cuatro muestras: 70 bytes de carga útil, 74 de trama y unos 370 B/s, muy por debajo de la capacidad del enlace. El precio es un retardo de visualización de hasta 200 ms y que la lectura bruta de la célula sólo se refresque una vez por lote (sección 11.9).

Estado del ciclo. El paquete `0x05` creció de 14 a 18 bytes al reorganizar el ciclo: además de la fase, las repeticiones y la posición, transporta `cycPosB`, la posición en la que ha quedado fijado el punto B. Sin ese dato no se puede saber desde fuera contra qué está midiendo el ciclo un recorrido parcial. Las fases son ahora seis: reposo, colocándose en B, hacia A, volviendo a B, completado y abortado.

Para que un firmware y un servidor desparejados no se queden mudos, el servidor acepta también la trama de 14 bytes de la versión anterior y asume `cycPosB = 0`. Esa compatibilidad se añadió precisamente porque un desajuste de versiones despistó durante la depuración del ciclo: el firmware enviaba una trama que el servidor rechazaba entera, y la interfaz no mostraba ninguna fase.

Modelo pregunta-respuesta de la célula. Cada orden de consulta, programación o mantenimiento de la célula produce exactamente un paquete `0x04`, también cuando se rechaza. El paquete incluye el código de la orden a la que responde, para que la interfaz nunca se quede esperando, y lleva información de diagnóstico: el primer byte que devolvió el chip, si el pin de reinicio corta de verdad la alimentación y en qué intento del barrido de retardos se consiguió entrar en modo comando (sección 10.2).

8.1.4. Órdenes del ordenador al microcontrolador

Tabla 8.3: Órdenes aceptadas por el firmware.

Código	Nombre	Datos	Efecto
`0x01`	INIT	[dirección Modbus] opcional	Reinicia la posición, invalida el recorrido medido y configura y habilita el servo.
`0x02`	STOP	–	Consigna a cero; aborta *homing* y ciclo.
`0x03`	SET_PARAM	id (1 B), valor int16	Modifica un parámetro (tabla 8.4).
`0x04`	MOVE_A	–	Movimiento manual hacia A (arriba).
`0x05`	MOVE_B	–	Movimiento manual hacia B (abajo).
`0x06`	SHUTDOWN	–	Deshabilita el accionamiento.
`0x07`	SCAN	–	Búsqueda bloqueante del esclavo (1–247).
`0x08`	HOME	–	Inicia el *homing*.
`0x09`	LC_PROGRAM	B_Config, ZMDI_Config1 (uint16)	Graba la configuración de la célula (sólo parado).
`0x0A`	LC_QUERY	–	Lee la configuración de la célula (sólo parado).
`0x0B`	LC_HOLD	modo de aparcado (1 B)	Mantiene la célula sin tensión 8 s para medirla.
`0x0C`	CYCLE_START	modo, recorrido (centi-rev), velocidad, repeticiones	Inicia el ciclo B→A→B, empezando por la colocación en B.
`0x0D`	CYCLE_STOP	–	Detiene el ciclo donde esté.

Tabla 8.4: Parámetros configurables con SET_PARAM.

Id	Parámetro	Unidad	Defecto
1	Velocidad de movimiento manual	rpm	10
2	Límite de fuerza (0 = desactivado)	cuentas	0
3	Tara de la célula	cuentas	8210
4	Aceleración (C03.22)	u. del accionamiento	5
5	Deceleración (C03.24)	u. del accionamiento	5
6	Rigidez (C00.05)	nivel	10
7	Calibración de fuerza (sólo informativa)	cuentas/N	0
8	Desplazamiento del origen tras el *homing*	centi-rev	0
9	Velocidad de *homing*	rpm	30

8.2. Modbus RTU con el accionamiento

8.2.1. Registros utilizados

El firmware sólo usa las funciones 0x03 y 0x06 sobre un conjunto reducido de registros (anexo C). Los parámetros del accionamiento se nombran `Cgg.ii`, y todas las direcciones cumplen la regla

$$\text{dirección} = 256 \cdot gg + 0\text{x}ii, \tag{8.1}$$

es decir, el índice se interpreta en hexadecimal. Por ejemplo, C04.11 (habilitación) es $1024 + 17 = 1041$ y C03.22 (aceleración) es $768 + 34 = 802$. Las magnitudes de supervisión están en el grupo `0x40`: velocidad en 16385 (`0x4001`) y par en 16387 (`0x4003`). Tener esta regla evita errores al añadir registros a partir del manual.

8.2.2. Secuencia de configuración

Al recibir INIT, el firmware configura el accionamiento así: deshabilitar (C04.11 = 0), seleccionar el modo velocidad (C00.00 = 1), poner la consigna a cero (C03.21 = 0), escribir la rigidez y las rampas, configurar la resistencia de frenado externa (C00.10 a C00.13) y habilitar (C04.11 = 1). Si una escritura esencial falla se reintenta la secuencia completa, y tras tres fallos el sistema pasa a estado de error. Entre escrituras se deja un silencio de 700 µs.

8.2.3. Transacciones por periodo

En régimen normal, cada vuelta del lazo realiza tres transacciones: lectura de la velocidad, lectura del par y escritura de la consigna. Escribir la consigna en cada vuelta, aunque no cambie, sirve además de supervisión del enlace: si la escritura falla, se marca el servo como desconectado.

A 115200 baudios con formato 8N1, cada byte ocupa 86,8 µs. Las tres transacciones suman 46 bytes, unos 4,0 ms de línea. A eso se añaden 0,6 ms de retardos de conmutación del transceptor y 0,7 ms de silencio entre las dos lecturas. El tiempo de trabajo medido del lazo es 10,97 ms (sección 11.2); la diferencia, unos 5,7 ms, corresponde principalmente al tiempo de respuesta del accionamiento, aproximadamente 1,9 ms por transacción.

La guía de implementación de Modbus serie recomienda, por encima de 19200 baudios, silencios fijos de 1,75 ms entre tramas [33]. El firmware usa 700 µs y el accionamiento lo acepta sin errores, pero con otros esclavos convendría respetar el valor recomendado.

8.3. Interfaz entre el servidor y el navegador

8.3.1. WebSocket

El servidor mantiene una conexión WebSocket en `/ws` con cada navegador. En sentido servidor → navegador envía objetos JSON de cinco tipos:

- **muestra de telemetría** (sin campo `type`): una por muestra del lote, con los valores brutos, los nombres de estado y error ya traducidos, la posición integrada, los finales de carrera, la frecuencia de muestreo medida y el estado del *homing*;
- `home`: fase del *homing*, posición y recorrido medido;
- `cycle`: fase, repeticiones hechas y pedidas, posición, recorrido pedido, posición de B y fichero en grabación;
- `lc_config`: resultado de la consulta o programación de la célula;
- `cmd_ack`: confirmación de *toda* orden recibida, con indicación de éxito y un texto explicativo. Sin ella, una orden que no llegaba al puerto era indistinguible de una que sí se había ejecutado.

En sentido navegador → servidor, las órdenes son objetos `{"cmd": nombre, ...parámetros}`. El servidor las valida, limita los parámetros a rangos seguros (por ejemplo, velocidad del ciclo entre 0 y 300 rpm), construye la trama binaria y la escribe en el puerto.

8.3.2. Servicios HTTP

Tabla 8.5: Servicios HTTP del servidor.

Método	Ruta	Función
GET	`/`	Monitor en tiempo real
GET	`/ensayos`	Visor de ensayos grabados
GET	`/sim`	Configuración del mecanismo y simulador
GET	`/api/mecanismo`	Geometría actual (JSON)
PUT	`/api/mecanismo`	Guarda la geometría, validada contra rangos
GET	`/cycles`	Lista de los 50 ensayos más recientes (JSON)
GET	`/cycles/{nombre}`	Descarga de un ensayo (CSV)
DELETE	`/cycles/{nombre}`	Borrado de un ensayo (rechazado si se está grabando)
GET	`/mecanismo.js`, `/tema.css`, `/tema.js`, `/nav.js`	Recursos comunes a las tres páginas
`/fui` (montaje estático)		Kit de interfaz (hoja de estilo y tipografías)

Las rutas con nombre de fichero reducen el parámetro a su último componente antes de usarlo, de modo que una petición con `../` no puede leer ni borrar ficheros fuera de la carpeta de ensayos. La escritura de la geometría se valida en el servidor contra los mismos rangos que comprueba el navegador, de forma que una petición construida a mano tampoco puede dejar una configuración imposible.

Capítulo 9

Software de supervisión y análisis

9.1. Visión general

El software del ordenador consta de un servidor en Python (`web_monitor.py`), tres páginas web y varias herramientas auxiliares. El servidor se ejecuta con

```
python web_monitor.py [puerto_serie]      # http://localhost:8080
```

y detecta solo el puerto del microcontrolador si no se indica. Las tres páginas son el *monitor* en tiempo real (`/`), el *visor de ensayos* (`/ensayos`) y la *configuración del mecanismo* con simulador (`/sim`), enlazadas entre sí por una barra de teclas programables común.

9.2. Servidor

9.2.1. Estructura concurrente

El servidor combina un hilo del sistema operativo para el puerto serie con el bucle asíncrono de FastAPI (figura 9.1).

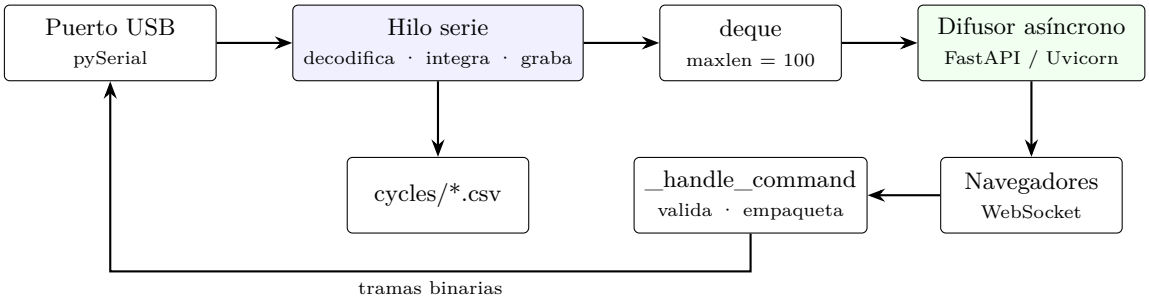


Figura 9.1: Estructura del servidor de supervisión.

Hilo serie. Abre el puerto (probando `/dev/ttyACM*`, `/dev/ttyUSB*`, `/dev/cu.usbmodem*` y `COM*`), vacía el búfer y entra en un bucle que lee tramas, comprueba el CRC y las decodifica con `struct`. Cualquier excepción cierra el puerto y provoca un reintento al cabo de un segundo, lo que cubre la desconexión física y el reinicio del microcontrolador al reprogramarlo (RNF4). Por cada muestra de telemetría:

- mide la frecuencia de muestreo contando las muestras del último segundo *con el reloj del dispositivo*, no con el del ordenador;
- detecta que el reloj del microcontrolador ha vuelto a cero (reinicio) y reinicia entonces la ventana de medida y la integración;
- integra la posición a partir de la velocidad, descartando saltos de tiempo anómalos;

- si hay un ensayo en grabación, escribe la fila correspondiente;
- deposita un mensaje JSON en la cola.

Cola y difusor. La comunicación entre el hilo y el bucle asíncrono usa una `deque` de la biblioteca estándar con un único productor y un único consumidor, segura sin cerrojos en CPython para `append` y `popleft`. La longitud máxima de 100 mensajes acota la memoria si no hay ningún navegador conectado. El difusor envía cada mensaje a todos los clientes, elimina los que han cerrado la conexión y, cuando la cola está vacía, duerme 20 ms.

Registro. Todas las tramas, órdenes y confirmaciones se registran con marca de tiempo en consola y en `web_monitor.log`. Al depurar el ciclo se reforzó ese registro, porque era insuficiente: ahora cada línea de telemetría incluye la posición, los dos finales de carrera y la fase del ciclo, y cada cambio de fase escribe una línea propia. Sin eso, un ciclo que arrancaba y terminaba en 200 ms no dejaba rastro de por dónde había pasado.

9.2.2. Configuración del mecanismo

La geometría vive en `mecanismo.json`, con dos servicios: `GET /api/mecanismo` devuelve la configuración y `PUT /api/mecanismo` la guarda. El servidor valida cada clave contra un rango (por ejemplo, α_0 entre 0° y 180°, o radios estrictamente positivos) y rechaza lo que no encaje, de modo que ni una petición construida a mano puede dejar una geometría imposible. Los ficheros guardados con la convención anterior, que medían el ángulo desde la horizontal, se convierten al leerlos mediante $\alpha = 90° - \theta_0$.

9.2.3. Grabación de ensayos

La grabación es automática y la dirigen los paquetes de ciclo: cuando la fase pasa a "hacia A" o "volviendo a B" y no hay fichero abierto, se crea `cycles/AAAAMMDD-HHMMSS_ciclo.csv`; cuando la fase pasa a completado, abortado o reposo, se cierra. La fase de colocación en B no se graba, porque el ensayo empieza cuando el eje ya está en B. Cada fichero lleva una cabecera con las constantes del ensayo y una fila por muestra (formato en el anexo D).

Se graban deliberadamente las *cuentas brutas* de la célula, no newtons: la calibración puede cambiar después del ensayo —de hecho ha cambiado varias veces— y con los datos brutos basta con aplicar la nueva constante (RNF5).

9.3. Modelo compartido

Las fórmulas del capítulo 4 están implementadas una sola vez, en `static/mecanismo.js`, y las usan por igual el simulador y el visor de ensayos. El módulo ofrece la geometría del tambor (`tambor`), la longitud de cable (`cable`), el ángulo (`barraAng`), el estado completo del mecanismo para un avance dado (`estado`) y la inversa, el avance necesario para llevar la barra a un ángulo (`revParaAngulo`), resuelta por bisección.

Esta unificación corrigió un problema real: cada página llevaba su copia de las fórmulas y habían divergido, hasta el punto de que el monitor seguía calculando con un cilindro y la barra horizontal mientras el visor ya usaba la caracola. Los cambios se propagan al instante entre pestañas del mismo navegador mediante `BroadcastChannel`, y al volver a una pestaña se relee la configuración por si se cambió desde otro equipo.

9.4. Monitor en tiempo real

La página principal (`index.html`) se organiza en tres zonas:

1. **Estado**: indicador de conexión, estado de la máquina y código de error, modo, estado del servo y de la célula, tiempo del dispositivo y frecuencia de muestreo, finales de carrera (A

arriba, B abajo) con la fase del *homing* y el recorrido medido, y una tarjeta destacada con la fuerza.

2. **Controles en pestañas**: *Servo* (dirección Modbus, inicialización, movimiento manual, *homing*, parada, puesta a cero y apagado; velocidades y desplazamiento del origen); *Célula 1* (calibración, tara y, en configuración avanzada, las palabras B_Config y ZMDI_Config1 con el botón de programar); *Ciclo B→A→B* (hasta dónde sube, velocidad y repeticiones); y *Ensayos grabados*.
3. **Gráficas**: velocidad de consigna y real con los finales de carrera y marcas verticales en los instantes en que se enviaron órdenes, y fuerza frente al tiempo.

9.5. Configuración del mecanismo y simulador

La página `/sim` (figura 9.2) reúne la edición de la geometría y un simulador:

- **Formulario** con el tipo de tambor (caracola, cilindro o leva), radios y barrido, reducción, sentido, cotas L_2, x_t, L_3, e_1, L_4, e_2, masa y θ_0. Los cambios no se aplican hasta pulsar *Guardar y propagar*, y se pueden descartar.
- **Comprobaciones** automáticas: distancia e inclinación de la línea O→C, ángulo y cable en B, ángulo en el extremo del recorrido, recorrido angular total y avisos cuando θ_0 queda fuera del margen entre el punto muerto y la barra vertical, que es donde la tensión se hace singular. También calcula la inversa: cuántas revoluciones de motor hacen falta para llevar la barra a un ángulo dado, que es el recorrido B→A que tendría que hacer el ciclo.
- **Dibujo a escala** del mecanismo en un `canvas`, con un cursor que recorre el ciclo y un botón de reproducción, y la posibilidad de cargar un ensayo grabado para que el cursor siga sus datos.
- **Gráficas** de tensión, par y ángulo frente al avance del motor.

[**COMPLETAR:** captura de pantalla de la página /sim con el mecanismo dibujado, para incluirla como figura 9.2]

[**COMPLETAR:** captura de /sim]

Figura 9.2: Página de configuración del mecanismo con el simulador.

9.6. Visor de ensayos

El visor (`ensayos.html`) lista los ensayos grabados y, al abrir uno, muestra un resumen (muestras, duración, cadencia, repeticiones y rangos), la fuerza frente a la posición con una traza por repetición, la fuerza y la posición frente al tiempo, el par frente a la posición y al tiempo, y la velocidad con los finales de carrera.

Sobre la gráfica de fuerza puede superponerse la curva teórica, calculada con la geometría configurada. El avance del ciclo se toma del propio ensayo: se resta la posición de la primera

muestra —que es B, donde arranca— y el sentido se deduce del signo de la muestra más alejada, de modo que los ensayos antiguos, que iban de A hacia B, siguen siendo interpretables.

El visor comprueba además dos consistencias que han resultado muy útiles:

1. *Escala*: compara el cable que paga el tambor durante el ensayo con el que el mecanismo puede absorber en ese sentido; si se pide más del que cabe, avisa y sugiere la reducción o los radios que lo harían cuadrar.
2. *Ángulo*: calcula el ángulo de la barra en B, en el extremo y al final del ensayo, y avisa si el modelo exigiría pasar de los topes o si el cierre del ciclo no cuadra.

Por último, ofrece un ajuste por mínimos cuadrados de la recta que convierte cuentas en newtons, $T \approx a\,(\text{cuentas}) + b$, con su R^2 y su cobertura. Conviene leerlo junto con la sección 4.6.1: un R^2 alto no valida la geometría, porque el mismo valor se obtiene con la sensibilidad de signo contrario.

9.7. Herramientas auxiliares

- `leer_ciclo.py`: resumen de un ensayo en consola, sólo con la biblioteca estándar.
- `monitor.py` y `monitor_posicion.py`: monitores de terminal de versiones anteriores del protocolo.
- `test_servo_modbus.py`: barrido de velocidades y direcciones Modbus con un adaptador USB-RS485, útil para identificar la configuración del accionamiento sin el microcontrolador.
- `memoria/figuras/generar_figuras.py` y `generar_figuras21.py`: generan las figuras y las cifras de esta memoria a partir de los ficheros de ensayo, junto con `modelo_mecanismo.py`, que es el modelo del capítulo 4 portado a Python.

9.8. Interfaz de usuario

Las tres páginas comparten el aspecto de un panel de control industrial, con un kit de interfaz externo (hoja de estilo y tipografías) publicado en `/fui` y copiado sin modificar, y los ajustes propios en `static/tema.css`. Los colores de las gráficas se leen de las variables de esa hoja de estilo (`tema.js`), de modo que hay un único sitio donde cambiarlos, y la navegación entre páginas se genera desde `nav.js`.

9.9. Observaciones de calidad

Durante la redacción de esta memoria se revisó el software y se identificaron estos puntos de mejora, que no afectan a los resultados:

- `requirements.txt` sólo declara `pyserial`; faltan `fastapi` y `uvicorn[standard]`.
- La cabecera del CSV decide si el punto de parada es “posición” o “final de carrera” según el recorrido pedido sea mayor que cero, no según el modo realmente usado. Como la interfaz envía siempre un recorrido, todos los ensayos aparecen como “posición” aunque se hicieron contra los finales de carrera. La corrección es incluir el modo en el paquete de ciclo.
- El servidor escucha en todas las interfaces (`0.0.0.0`) sin autenticación, así que cualquier equipo de la red puede mover la máquina.
- La posición se integra dos veces, en el firmware y en el servidor, con el mismo método; el CSV guarda la del servidor. Sería más limpio transmitir la del firmware en cada muestra.
- La cabecera del firmware todavía describe la versión de doble núcleo.

Capítulo 10

Problemas técnicos y soluciones

Este capítulo reúne los problemas más costosos del desarrollo. Para cada uno se describen el síntoma, la causa y la solución, tal como quedaron documentados en el código y en el repositorio. Muchos no se deben a un error de programación, sino a detalles de los componentes o del montaje que no son evidentes en una primera lectura.

10.1. UART sobre PIO y latencia del enlace Modbus

Síntoma. Tras migrar el firmware del ESP32-S3 a la Pico 2, las órdenes tardaban entre 3 y 4 s en tener efecto.

Causa. En el ESP32-S3 la UART es un periférico hardware: los bytes de la respuesta llegan a una FIFO por interrupción y, cuando ModbusMaster los pide, ya están disponibles, así que cada transacción dura 1 a 3 ms. En la Pico, el trazado lleva la transmisión a GP14 y la recepción a GP13, y se usó `SerialPIO`, una UART implementada con una máquina de estado PIO que funciona en cualquier pin [**VERIFICAR:** confirmar si el RP2350 permite la función UART en GP14 mediante su función alternativa]. Con `SerialPIO`, el bucle de espera de ModbusMaster agotaba su tiempo límite, de unos 20 ms, en muchas transacciones: diez escrituras consecutivas bloqueaban el núcleo 200 ms, y con las lecturas de supervisión se llegaba a varios segundos.

Un segundo detalle afectaba al cambio de sentido del transceptor: llamar a `flush()` antes de desactivar el emisor, como se haría con una UART hardware, rompía la recepción inmediata de la respuesta. La solución fue esperar un tiempo fijo de 200 µs tras el último byte.

Primera solución: doble núcleo. La versión `firmwarerasp` dedicó el núcleo 1 al bus Modbus, con la consigna publicada en variables `volatile` y un protocolo de relevo para que el núcleo 0 pudiera tomar el bus. Funcionaba, pero con variables compartidas sin atomicidad, esperas de relevo y lecturas cuyo instante no estaba ligado al de la muestra de telemetría.

Solución final: dueño único con lazo de periodo fijo. Ajustados los tiempos de espera y el cambio de sentido, cada transacción volvió a durar pocos milisegundos y se pudo prescindir del segundo núcleo: un único lazo de 50 ms con exactamente tres transacciones por vuelta. El resultado es un tiempo de trabajo de 11,0 ms con 0,2 ms de dispersión (sección 11.2), sin condiciones de carrera y con lecturas, lógica y escritura en la misma vuelta.

10.2. Configuración del ZSC31014: la ventana del modo comando

Contexto. Para cambiar la ganancia o el cero del acondicionador hay que escribir en su EEPROM, y para ello entrar en modo comando. Según la hoja de datos [43], tras un reinicio por

encendido (POR) el chip espera la orden Start_CM durante 6 ms si la EEPROM no está bloqueada, o 1,5 ms si lo está. Pasada la ventana, ignora Start_CM sin dar ninguna señal de error.

10.2.1. Primer obstáculo: provocar el reinicio

Tras reprogramar la Pico, la configuración nunca se aplicaba, porque la placa Click seguía alimentada y sin POR no se abre la ventana. La solución fue controlar su alimentación con el pin EN (GPIO1) y hacer un ciclo completo de apagado y encendido antes de cada intento.

10.2.2. Segundo obstáculo: alimentación parásita por el bus

Síntoma. Con EN a nivel bajo, el chip seguía respondiendo por I^2C, y la tensión de la Click se quedaba en 2,5 V a 2,7 V en lugar de caer a cero.

Causa. Las resistencias de *pull-up* de SDA, SCL e INT cuelgan del 3,3 V permanente del zócalo mikroBUS, no del carril gobernado por EN. Con el chip sin alimentar, la corriente entraba por esos pines, atravesaba los diodos de protección internos y alimentaba el carril. La hoja de datos sitúa el umbral de POR entre 1,8 y 2,5 V: con 2,5 V a 2,7 V parásitos, el chip nunca veía un reinicio.

Diagnóstico y solución. Se añadió la orden LC_HOLD, que mantiene la célula sin alimentación 8 s sin bloquear el lazo, con dos modos para las líneas del bus. Las medidas con polímetro mostraron que sólo forzándolas a nivel bajo la tensión cae a cero, y así se dejó por defecto. Hubo además un error en el propio diagnóstico: la primera versión de la prueba dejaba las líneas en alto mientras medía, de modo que siempre "detectaba" realimentación y acusaba al pin EN sin motivo.

10.2.3. Tercer obstáculo: acertar la ventana

La orden tiene que llegar después de que el chip arranque y antes de que la ventana se cierre, y ese instante depende de los condensadores de la placa Click, así que los retardos copiados de otros controladores no servían. En lugar de adivinarlo, el firmware lo mide: sondea la dirección hasta que el chip responde, y a partir de ahí prueba retardos de 0 a 1050 µs en pasos de 150 µs a lo largo de hasta ocho ciclos de alimentación, confirmando la entrada con la lectura de B_Config. El intento que funciona se devuelve a la interfaz para poder fijarlo más adelante.

10.2.4. Cuarto obstáculo: integridad de la EEPROM

La firma MISR que protege la EEPROM se recalcula al salir del modo comando con Start_NOM y se comprueba en el siguiente POR: cortar la alimentación antes deja la firma a medias y el chip arranca desde entonces en estado de diagnóstico. Por eso, tras cualquier escritura se envía Start_NOM y se espera diez veces el tiempo típico antes de cortar. Como ZMDI_Config1 sólo se carga tras un POR, la verificación se hace en una segunda pasada desde un reinicio limpio.

10.3. Lecturas de la célula siempre "antiguas"

En una versión del firmware no se registraba ni una sola muestra válida. La lectura se hacía en dos fases separadas 2 ms, quedándose con la segunda; como en modo de conversión continua leer no dispara una conversión y el chip no había producido un dato nuevo en ese tiempo, la segunda lectura venía siempre marcada como repetida. Una versión anterior funcionaba por casualidad, porque sus dos lecturas caían en vueltas distintas del lazo. La solución fue una única lectura por vuelta, aceptada sólo si el dato es nuevo.

10.4. Depuración del ciclo de ensayo

Al reorganizar el ciclo para que empezara en el extremo inferior (sección 7.7) aparecieron tres problemas encadenados:

El ciclo terminaba sin hacer nada
Con el eje ya en el final de carrera, la condición de llegada se cumplía de inmediato y el ciclo recorría todas sus fases en un par de periodos. No era un fallo de lógica, sino de arranque: faltaba la fase de colocación en B.

No quedaba rastro en el registro
Un ciclo que empieza y acaba en 200 ms no aparecía en el registro, porque sólo se escribía una línea por lote de telemetría y ninguna al cambiar de fase. Ahora cada cambio de fase escribe su propia línea, y la línea de telemetría incluye posición, finales de carrera y fase.

La interfaz no mostraba la fase
El firmware había pasado a enviar una trama de ciclo de 18 bytes y el servidor, que esperaba 14, la descartaba entera. Se añadió la compatibilidad con la trama antigua para que un desajuste de versiones degrade la información en lugar de hacerla desaparecer.

Además, el ciclo pasó a reconocer los extremos por los finales de carrera en vez de por la posición integrada. Con la posición, cualquier error de escala del accionamiento o cualquier cero mal colocado desplazaba el recorrido; con los finales, el ensayo recorre siempre lo mismo.

10.5. El modelo que no cerraba: la salida del cable desplazada

Síntoma. Con el primer modelo, que situaba la salida del cable justo encima de la articulación, la curva teórica no encajaba con los ensayos: el recorrido angular que predecía no era el que hacía la barra, y el ajuste sólo cuadraba forzando valores poco creíbles de la reducción.

Primer intento, y por qué tampoco cerró. La primera corrección fue admitir que la salida del cable no está sobre la vertical de la articulación y meter un desplazamiento horizontal dx como parámetro. Mejoró, pero no cerró, y el modo en que falló resultó revelador: para cuadrar con la medida, el ajuste tenía que invertir el sentido de la espiral y llevar la reducción a valores poco creíbles. La razón de fondo era que, midiendo el ángulo de la barra entre la línea articulación → salida y la propia barra, y acotándolo a 0° a 180°, el modelo *no podía representar la barra colgando* por debajo de cierto ángulo. Como el ensayo arranca precisamente ahí, el ajuste compensaba esa imposibilidad con la única libertad que le quedaba, que era el perfil del tambor.

Causa real. El cable no sale de un punto: sale *tangente* al tambor. El punto de salida está a r del eje y se desplaza cuando el radio cambia, de modo que a lo largo del recorrido se mueve tanto como $r_1 - r_0$, unos 28 mm en la pieza medida. Un punto fijo, con o sin dx, no puede representar ese desplazamiento. Y hay un segundo efecto del mismo orden: al moverse la barra, el punto de tangencia *migra* por la superficie del tambor y enrolla cable que el tambor no ha pagado, lo que obliga a trabajar con el cable equivalente de la sección 4.3.1 en lugar del cable libre.

Solución. Se reescribió el modelo con el eje del tambor como referencia (sección 4.3), con el ángulo de la barra medido desde la horizontal —que es lo que se puede comprobar con un inclinómetro, sin modelo de por medio— y resolviendo el contacto sobre el perfil real. Al no haber ya relación cerrada entre giro y ángulo, la trayectoria se integra paso a paso. La comprobación de que esta vez sí cierra no es un ajuste sino una identidad física: el balance de energía se cumple en la cuarta cifra significativa sin tocar ningún parámetro.

Lección. Un ajuste que sólo cuadra forzando un parámetro a un valor poco creíble no está midiendo ese parámetro: está avisando de que al modelo le falta un grado de libertad en otro sitio. Aquí el aviso fue la espiral invertida, y señalaba a la cinemática del cable, no al tambor.

10.6. Lo que la fuerza no puede decidir

Al ajustar los parámetros del mecanismo contra la señal de la célula se obtuvieron coeficientes de determinación altos ($R^2 \approx 0{,}93$ en los ensayos limpios), lo que parecía confirmar la geometría. No lo confirmaba. Como la célula no está calibrada, el ajuste incluye la sensibilidad como incógnita, y R^2 es el cuadrado de la correlación: no distingue una sensibilidad positiva de una negativa. Dos geometrías que predicen tensiones con tendencias opuestas dan exactamente el mismo R^2 (sección 4.6.1).

La consecuencia práctica llegó al añadir el par del accionamiento al contraste: con los mismos parámetros, el par medido y el del modelo resultaron *anticorrelados* (capítulo 11). El análisis se reorganizó entonces alrededor del par, que no depende de ninguna calibración pendiente, y la señal de la célula quedó como comprobación secundaria.

10.7. Tramo inicial sin carga y calibraciones incoherentes

Los ensayos de la segunda campaña muestran que la primera media revolución de motor transcurre sin tensión en el cable y con un par casi nulo: el mecanismo recoge holgura antes de empezar a levantar la masa. No es un fallo, pero invalida ese tramo para el análisis y obliga a definir explícitamente el recorrido con carga.

Por otra parte, la constante que relaciona cuentas y newtons ha resultado distinta en cada intento: 14,41 cuentas/N en la configuración de la interfaz, unas 20 al ajustar contra el modelo y 27 a 28 en los ensayos limpios; y en la campaña del 15 de septiembre el ajuste salía incluso con el signo contrario. Todo ello confirma que la célula necesita una calibración con masas patrón antes de dar valores absolutos de fuerza.

10.8. Segunda célula de carga en el mismo bus

Se evaluó añadir una segunda célula con una Load Cell 4 Click (NAU7802) en el mismo bus. Surgieron dos problemas. El primero, de convivencia: el ZSC31014 no toleraba transacciones ajenas entre la petición y la recogida del dato, y de ahí nació el árbitro de `i2c_bus.h`. El segundo, de ruido: con ganancia 128 y en reposo se midieron unas 18.000 cuentas pico a pico con el regulador interno a 4,5 V, unas 3.500 bajándolo a 3,0 V con calibración interna de cero, y unas 350 al activar el *chopper*, equivalentes a 0,49 µV en la entrada. La segunda célula se retiró del firmware final.

10.9. Discrepancias con la hoja de datos

Al contrastar el firmware con la hoja de datos del ZSC31014 se encontraron dos discrepancias que no afectan a los ensayos, pero conviene corregir:

1. **Bits de estado.** Según la hoja de datos, 00 = dato válido, 01 = modo comando, 10 = dato repetido y 11 = diagnóstico. El firmware define `LC_STATUS_STALE = 1` y `LC_STATUS_COMMAND = 2`, intercambiados. En la lectura normal sólo se acepta 00, así que no hay efecto; en la confirmación del modo comando la respuesta real (`0x5A`) se acepta por la segunda condición del código.
2. **Codificación de la ganancia.** El campo PreAmp_Gain no es lineal: 000 = ×1,5, 100 = ×3, 001 = ×6, 101 = ×12, 010 = ×24, 110 = ×48, 011 = ×96 y 111 = ×192, es decir, los tres bits en orden inverso. El selector de la interfaz supone un código lineal, de modo que para los códigos 1 a 6 la ganancia grabada no es la mostrada. Los códigos 0 y 7 coinciden. [**VERIFICAR:** comprobar qué código de ganancia estaba grabado durante los ensayos]

10.10. Otros problemas

Órdenes perdidas por los atajos de teclado

Un byte de CRC igual a una letra de atajo hacía que se perdiera la orden y se reiniciara el

servo. Se resolvió aceptando los atajos sólo con el decodificador en reposo.

Frecuencia de muestreo imposible
La interfaz llegó a mostrar 3813 Hz: tras un reinicio del microcontrolador su reloj volvía a cero y la ventana de medida no se podaba nunca. Se detecta el salto de reloj y se reinicia la ventana.

Desplazamientos fantasma
Tras la programación de la célula, que bloquea el lazo cerca de un segundo, la integración multiplicaba ese hueco por la última velocidad. Se anula la base de tiempo tras cada operación bloqueante.

Órdenes que "no hacían nada"
Una acción mal escrita en la interfaz se ignoraba sin avisar. Se añadió una confirmación para toda orden, con el motivo del fallo cuando lo hay.

Revisión A de la placa
Conflictos de red en el bloque RS-485, entradas analógicas en pines sin ADC y amplificadores sin alimentar (tabla 6.1).

10.11. Lecciones aprendidas

- **Leer la hoja de datos entera**: la ventana del modo comando, la firma de la EEPROM, el umbral de POR y la codificación no lineal de la ganancia están en secciones distintas y todas han resultado críticas.
- **Medir antes de suponer**: la alimentación parásita sólo se descubrió con un polímetro, y el retardo de la ventana, con un barrido.
- **Un ajuste bueno no es una validación**: un R^2 alto puede esconder una ambigüedad de signo. Hace falta una segunda magnitud, medida de forma independiente, para cerrar el contraste.
- **Cada operación debe responder**: las confirmaciones de órdenes y el registro por cambio de fase convirtieron fallos silenciosos en diagnósticos claros.
- **Conservar los datos brutos**: ha permitido recalibrar y reanalizar todos los ensayos sin repetirlos, incluso después de cambiar el modelo por completo.

Capítulo 11

Resultados

Este capítulo presenta las medidas del banco y las contrasta con el modelo del capítulo 4. El resultado central es que, sobre el recorrido que el mecanismo hace realmente con carga, **el par del tambor es prácticamente plano**, y que el perfil que lo explica no es la espiral tal y como está descrita en la configuración, sino esa misma espiral precedida por su arco de radio constante.

11.1. Campañas de ensayos

Los datos que sostienen el capítulo son los diez ensayos del 14 de octubre de 2026, todos con la misma configuración y el mismo ciclo B→A→B. No se ha descartado ninguno: los diez recorren 1,80 rev de motor con una desviación típica de 0,011 rev —un 0,6 %— y sus curvas se superponen dentro del espesor de la línea.

Tabla 11.1: Ensayos del 14 de octubre de 2026. Los diez son consecutivos, tomados en menos de tres minutos y con la misma configuración.

Ensayo	Muestras	Recorrido (rev de motor)
18:56:10	272	1,81
18:56:24	276	1,79
18:56:40	276	1,80
18:56:55	276	1,78
18:57:09	272	1,79
18:57:24	272	1,78
18:57:38	272	1,78
18:57:53	272	1,80
18:58:09	272	1,80
18:58:48	272	1,80

Las campañas anteriores (septiembre y 7 de octubre) se interpretaron, las primeras, con el modelo de punto de salida fijo, hoy superado. La del 7 de octubre, ya con el modelo tangente, da los mismos resultados que la que aquí se presenta —la meseta ajustada coincide en 61° y el orden de las hipótesis es idéntico—, con peor repetibilidad. Sus conclusiones geométricas no se mantienen y no se reproducen aquí; lo que sí se mantiene de ellas es la caracterización de la cadena de adquisición, que no depende del modelo.

11.2. Temporización del lazo de control

El firmware trabaja con un periodo fijo de 50 ms. Sobre las muestras de los ocho ensayos, el periodo medido es de 50,02 ms con una desviación típica de 0,13 ms, y el 98,3 % de los intervalos caen dentro de ±0,5 ms. El tiempo de trabajo declarado por el propio lazo es de 10,96 ms de media y 11,00 ms como máximo: un 22 % del periodo, con 39 ms de holgura.

Esa holgura importa porque la base de tiempos es lo que permite emparejar posición, fuerza y par en la misma muestra. Un lazo que se saltara periodos desplazaría unas magnitudes respecto de otras y contaminaría justo la comparación que se quiere hacer.

11.3. Ensayo tipo

La figura 11.1 muestra el último ensayo de la campaña. La secuencia es siempre la misma: el eje arranca en B y avanza hacia A; durante la primera media vuelta larga el cable aún no tira y las tres señales están en reposo; a partir de ahí la carga entra, la célula sube y el par se estabiliza; al llegar al final de carrera el ciclo invierte y vuelve a B recorriendo los mismos puntos en sentido contrario.

El tramo sin carga no es un defecto del ensayo sino una propiedad del mecanismo: en B la barra está a $-87°$, apenas 2,6° por debajo del reposo libre $\theta_{\text{libre}} = -84{,}4°$ de la ecuación (4.4), y ahí el brazo de la pesa es casi nulo. El mecanismo, sencillamente, no necesita fuerza para sostenerla. Todo el análisis cuantitativo se restringe por eso al tramo con carga, de 0,60 rev de motor en adelante.

11.4. Repetibilidad

Los diez ensayos alcanzan un recorrido máximo de 1,80 rev de motor con una desviación típica de 0,011 rev, un 0,6 %. Es un orden de magnitud mejor que en campañas anteriores, donde la dispersión del punto en que se detecta el final de carrera llegaba al 5 %. Con esta repetibilidad el promedio de los diez ensayos es prácticamente una sola curva.

11.5. Fuerza y par a lo largo del recorrido

La figura 11.2 promedia los diez ensayos por tramos de avance, separando la subida de la bajada. De esa separación salen, por la ecuación (4.13), el par gravitatorio τ_g y el de pérdidas τ_f.

Dos observaciones, ambas sobre el tramo con carga:

- **El par gravitatorio es plano.** Va de 38,0 a 36,1 unidades de accionamiento, es decir -5 % a lo largo de todo el tramo, con un valor medio de 38,5 y un rizado del 12,1 % en torno a él.
- **Las pérdidas son rozamiento seco.** τ_f vale 6,8 unidades de media y se mantiene entre 5,2 y 8,0 en todo el recorrido, sin tendencia. Es un 18 % del par gravitatorio. Que sea plano, y no proporcional a la velocidad, confirma que domina el rozamiento seco y justifica la hipótesis de la ecuación (4.13).

11.5.1. La escala del accionamiento

Ajustando el par medido al del modelo con una única constante positiva se obtiene

$$1 \text{ unidad de accionamiento} = 0{,}0553\,\text{N m}. \tag{11.1}$$

La cifra merece atención porque es una comprobación cruzada: un ajuste independiente sobre la campaña de septiembre, con otra geometría y otro tratamiento, daba 0,055 N m. Coinciden dentro del 1 %. Con esa escala, el par gravitatorio medio es de 2,13 N m, el de pico 2,24 N m y el de rozamiento 0,38 N m. En el eje del motor, con la reducción $N = 5$, el pico queda en 0,45 N m: un 35 % del par nominal del servomotor. Conviene subrayar que esta conversión a newton-metro es una cifra *derivada* del ajuste, no una medida independiente; lo que el accionamiento entrega sin ambigüedad es la forma de la curva y su signo.

11.6. Contraste con el modelo: qué perfil explica la medida

Éste es el núcleo del trabajo experimental. Se comparan cuatro hipótesis de tambor, todas con la misma geometría del mecanismo, ajustando en cada caso una única constante positiva de escala:

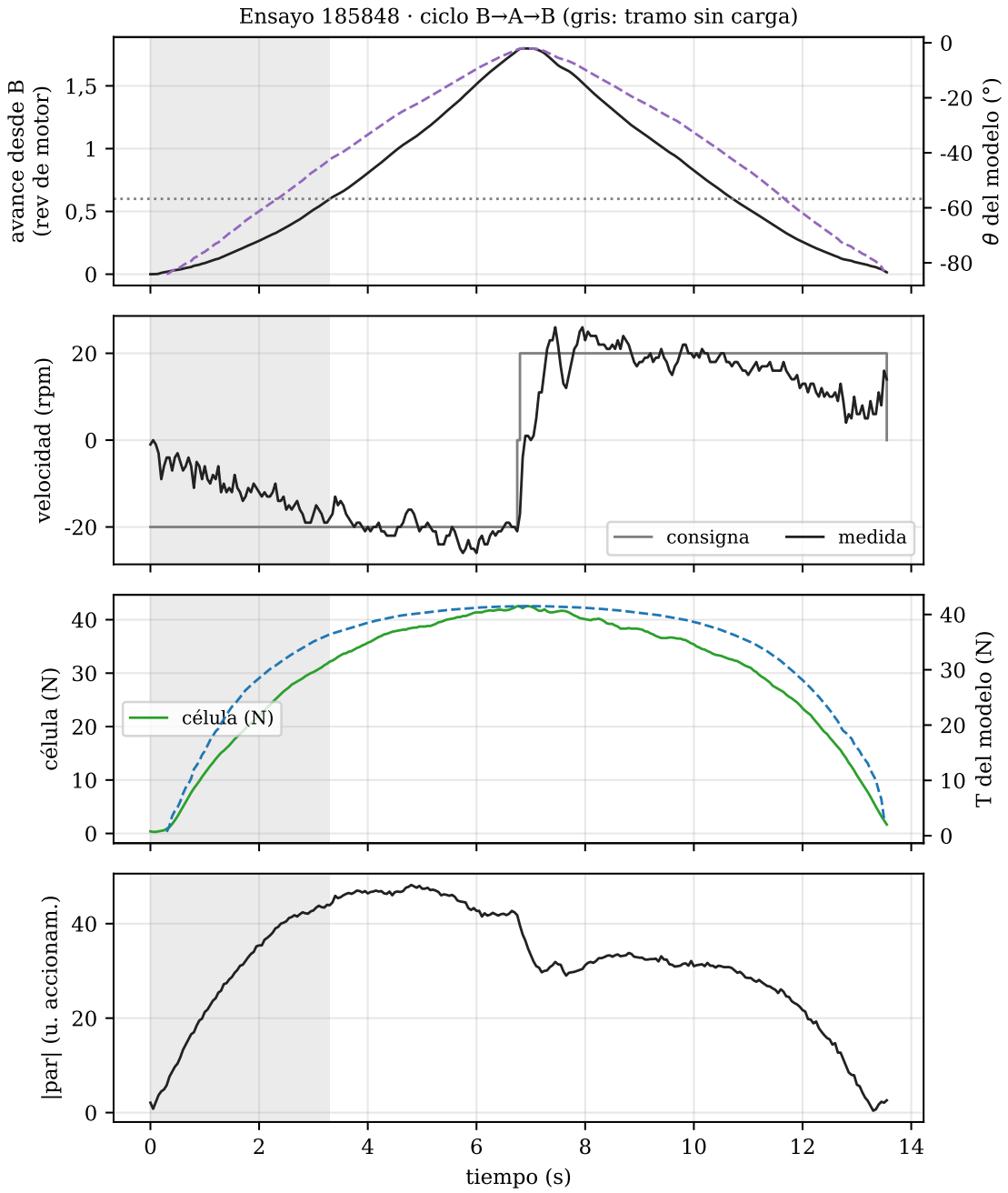


Figura 11.1: Ensayo del 14 de octubre a las 18:58:48. De arriba abajo: avance del motor con el ángulo de la barra que predice el modelo, velocidad frente a consigna, célula de carga frente a la tensión del modelo, y par del accionamiento. La banda gris es el tramo en que el cable todavía no tira.

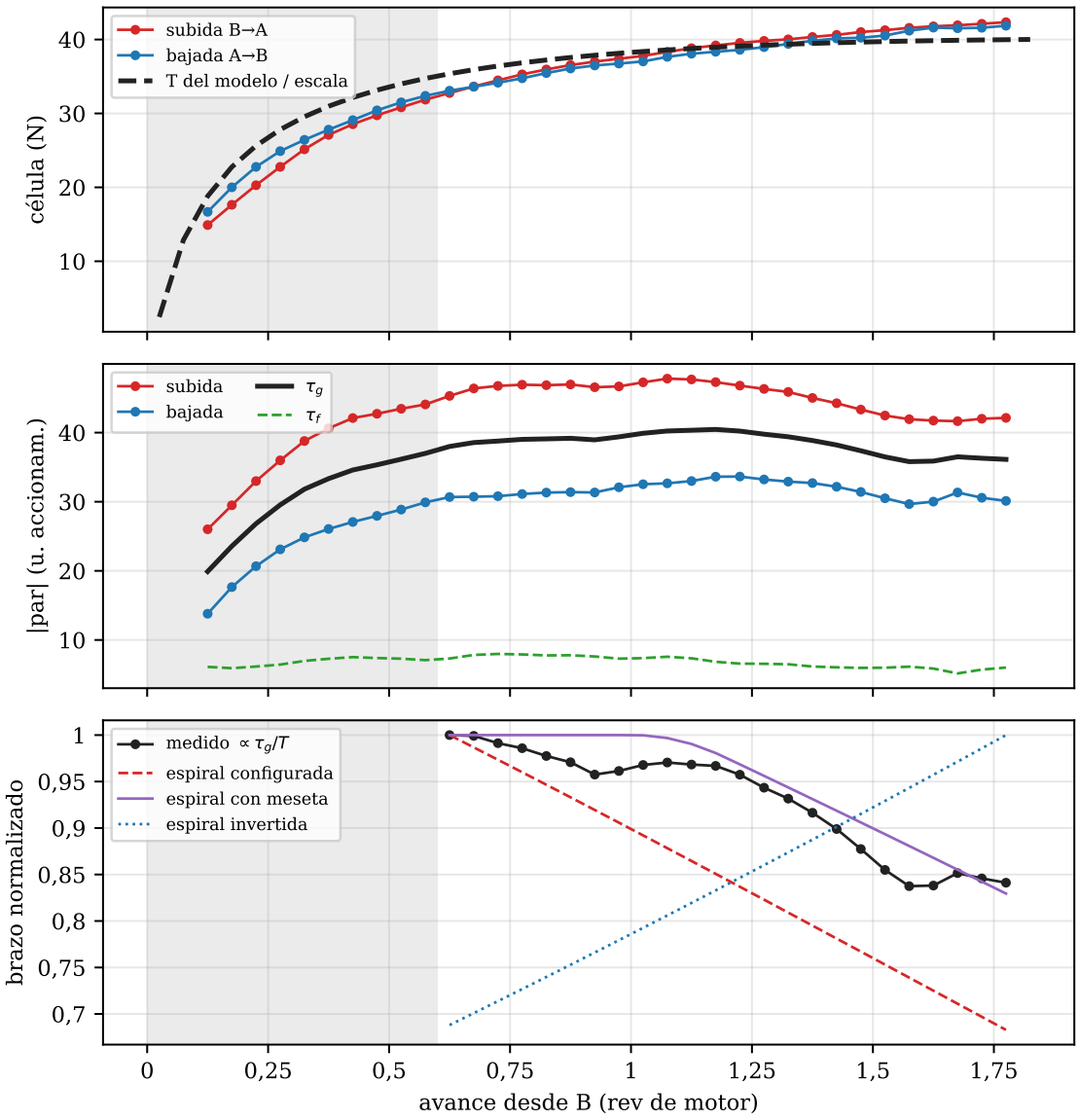


Figura 11.2: Promedio de los diez ensayos. Arriba: célula en subida y bajada frente a la tensión del modelo. Centro: par en los dos sentidos, con el par gravitatorio τ_g y el de pérdidas τ_f. Abajo: brazo efectivo deducido de la medida frente a las hipótesis de perfil.

1. **espiral configurada**, lineal de 56 a 28,18 mm en 0,367 rev de tambor;
2. **espiral con meseta**: la misma espiral, precedida de un arco de radio constante r_0, como la tiene la pieza real;
3. **cilindro** de radio constante 44,6 mm, el que recorre lo mismo;
4. **espiral invertida**, de 28,18 a 56 mm.

Tabla 11.2: Contraste de las hipótesis sobre el tramo con carga. R^2 ajusta una sola escala positiva (ecuación (4.15)); ρ^2 es el cuadrado de la correlación, ciego al signo.

Hipótesis	Bondad		Predicción sobre el tramo		
	R^2	ρ^2	var. τ	var. brazo	rizado
Espiral configurada	−1,23	0,56	−23 %	−32 %	25,2 %
Espiral con meseta	0,84	0,85	−5 %	−17 %	13,6 %
Cilindro, 44,6 mm	−1,78	0,17	15 %	0 %	13,4 %
Espiral invertida	−20,14	0,36	72 %	45 %	52,4 %
Medido			−5 %	−16 %	12,1 %

11.6.1. La meseta, y por qué la espiral sola no bastaba

La espiral tal y como está descrita en la configuración predice que el par caiga un 23 %; el medido cae un 5 %. El desacuerdo es grande ($R^2 = -1{,}23$, peor que predecir la media), y sin embargo el sentido es el correcto: el brazo decrece, como dice la medida. Lo que falla es *cuánto*.

La explicación está en el perfil real. La pieza no empieza a estrecharse en el punto en que arranca el ensayo: lleva antes un arco de radio constante. Si el cable arranca sobre esa meseta, el tambor gira un rato sin cambiar de radio y la espiral efectiva es más corta que la nominal. Ajustando la longitud de esa meseta como único parámetro adicional, el acuerdo salta a $R^2 = 0{,}84$, y la meseta que sale vale 0,17 rev de tambor, es decir 61°. La campaña del 7 de octubre, analizada igual, da exactamente la misma meseta con $R^2 = 0{,}90$.

Ese número es el mejor argumento a favor de la hipótesis, porque no se ha usado para construirla: *el perfil medido sobre el CAD tiene un arco a radio constante de unos* 50° antes de la espiral. El ajuste, que sólo ha visto pares del accionamiento, recupera esa meseta dentro de una decena de grados.

11.6.2. Lo que descarta el cilindro

El cilindro queda descartado sin ambigüedad ($R^2 = -1{,}78$, $\rho^2 = 0{,}17$). Y lo descarta por el *sentido*: con radio constante el par sigue a la tensión, que en esta geometría crece un 15 %; el par medido no crece, decrece ligeramente. El tambor, por tanto, está cambiando la relación de transmisión a lo largo del recorrido, que es lo que se quería comprobar.

La espiral invertida es la hipótesis peor con diferencia ($R^2 = -20{,}1$): predice que el par suba un 72 %. Queda así resuelta en sentido contrario la duda que dejó abierta la campaña de septiembre, donde el modelo antiguo favorecía precisamente esa orientación.

11.6.3. Las dos métricas no dicen lo mismo

La tabla ilustra de paso por qué la métrica importa. Con ρ^2, la espiral configurada puntúa 0,56 y parece una hipótesis aceptable; con R^2 y escala positiva es −1,23. ρ^2 premia que la curva tenga la forma adecuada aunque recorra el camino al revés o con la amplitud equivocada, y aquí eso basta para confundir. Es el mismo aviso que la sección 4.6.1 hace sobre la célula de carga, trasladado al par.

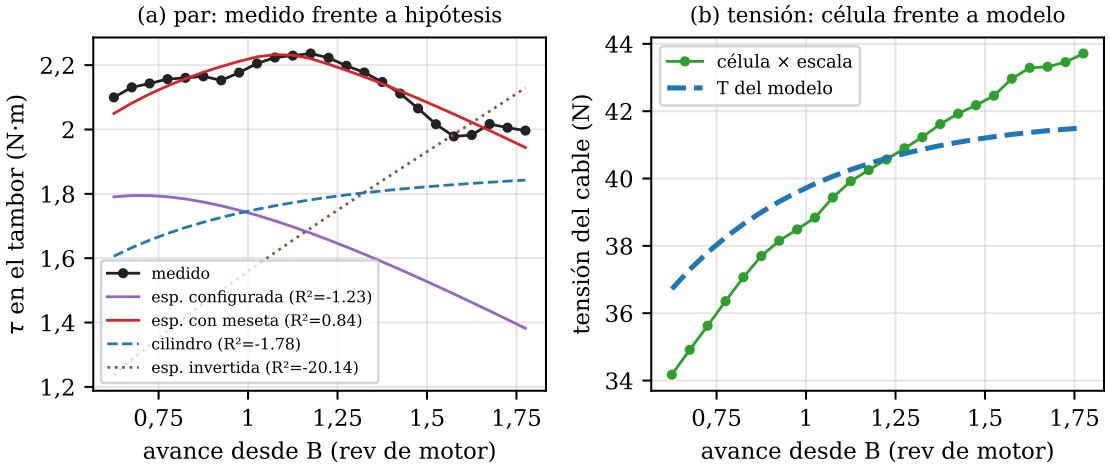


Figura 11.3: (a) Par gravitatorio medido frente a las cuatro hipótesis de tambor. (b) Tensión del cable: célula de carga, escalada, frente al modelo.

11.7. La tensión: célula frente a modelo

La figura 11.3b compara la señal de la célula con la tensión que predice el modelo. El acuerdo es parcial y conviene ser preciso sobre en qué.

En nivel, coinciden. Con la calibración provisional del firmware (−10,2 u.a./N) la célula da entre 33 y 42 N donde el modelo da entre 37 y 42 N; el factor de escala que las iguala es 1,04. Para una célula sin calibrar con masas patrón, un 4 % es un acuerdo notable, y confirma que el orden de magnitud del modelo es el correcto.

En pendiente, no. La célula crece un 28 % a lo largo del tramo con carga, y el modelo sólo un 13 %. La discrepancia es sistemática: aparece en los diez ensayos, con crecimientos medidos entre el 27 % y el 31 %, y aparecía igual en la campaña anterior. No es ruido ni un ensayo aislado.

Lo que sí sostiene la cadena completa es el producto: reconstruyendo el par como fuerza medida por el brazo del modelo se recupera el par del accionamiento con $R^2 = 0{,}55$. Es decir, las tres magnitudes son mutuamente consistentes en primera aproximación, pero queda una discrepancia de pendiente en la fuerza que el modelo no explica. Las sospechas razonables son la falta de calibración de la célula —su sensibilidad podría no ser lineal en este rango— y el valor de e_2, que es el parámetro al que más responde el brazo del peso. Ninguna de las dos se puede cerrar sin medir. [**VERIFICAR:** calibrar la célula con masas patrón y medir θ_{libre} con inclinómetro para fijar e_2]

11.8. Qué aporta realmente el tambor

Juntando la medida con la tabla 4.2 del capítulo 4 se puede responder a la pregunta de ingeniería, y la respuesta es más matizada de lo que el planteamiento inicial suponía.

- **El tambor actúa, y se mide.** El brazo efectivo cae un 16 % a lo largo del tramo con carga, y gracias a eso el par se queda casi plano donde un cilindro lo haría subir un 15 %. El mecanismo hace lo que se diseñó que hiciera.
- **La pieza compensa de más.** Su rizado teórico a igual recorrido es del 25 %, casi el doble que el de un cilindro. Lo que la salva en la práctica es precisamente la meseta: el tramo de radio constante acorta la espiral efectiva y deja el rizado real en el 12 % medido.

11.8.1. Cuánto motor se ahorra, exactamente

La pregunta que importa es cuánto baja el par de pico, por las razones de la sección 4.5.2. Y admite una respuesta casi sin modelo, apoyada en una identidad: como el tambor de referencia

y el real recorren lo mismo en las mismas vueltas, entregan el mismo trabajo en el mismo giro y por tanto *tienen el mismo par medio*. Un cilindro, además, copia la forma de la tensión, porque su radio no cambia. El beneficio se reduce entonces al cociente de dos relaciones pico/media, ambas medibles:

Tabla 11.3: Relación pico/media sobre el tramo con carga. Como el par medio es el mismo para todos los tambores que recorren lo mismo en las mismas vueltas, el cociente de estas relaciones es directamente lo que se ahorra en par de pico.

Magnitud	mín.	media	pico	pico/media
Par medido (caracola real)	35,8	38,5	40,5	**1,052**
Tensión medida (célula)	32,9	38,5	42,1	1,094
Tensión del modelo	36,7	40,0	41,5	1,037

El resultado es modesto y conviene no adornarlo: **el ahorro en par de pico está entre** $-1{,}5\,\%$ **y** $4\,\%$, según se tome la tensión del modelo o la de la célula. Con el modelo completo, comparando a igual recorrido, el cilindro equivalente (54,0 mm) pide un 1,5 % más de pico que la caracola real. En la práctica, en este banco la caracola no ahorra motor.

La horquilla es, además, otra consecuencia de la discrepancia abierta de la sección anterior: si la tensión varía el 13 % que dice el modelo, no hay nada que compensar y la caracola incluso estorba levemente; si varía el 28 % que dice la célula, el ahorro es de un 4 %. Resolver esa discrepancia es, por tanto, también lo que fijaría el beneficio real.

11.8.2. Y el cilindro pequeño, cuantificado

Queda la objeción natural que plantea la sección 4.5.2: si lo que se busca es menos par, ¿no bastaría un cilindro más pequeño? La respuesta general es que bajar el radio baja también el cable por vuelta, y cuesta vueltas. Aquí se puede poner número: el cilindro que iguala el par de pico de la caracola tiene 53,3 mm y necesita 1,80 rev de motor frente a las 1,77 rev de la caracola, es decir, **un** 1 % **más de vueltas**.

Ese 1 % es la medida exacta de lo que aporta el tambor de radio variable en este banco, y es despreciable. No porque el principio falle, sino porque *aquí no hay nada que repartir*: la tensión apenas varía. En un mecanismo donde la tensión variase el doble o el triple —una barra que arrancase más abajo, un cable con menos brazo, un muelle en el lazo— el mismo tambor recortaría el motor en esa misma proporción. El banco, por tanto, no refuta el principio: lo que demuestra es que esta geometría concreta no es el caso de uso donde el principio paga.

La conclusión de diseño es que una espiral mucho más suave —del orden de 52 a 44 mm— daría a la vez el rizado más bajo (5,5 %) y el par de pico más bajo. Es un cambio barato de aplicar y es la recomendación que se traslada al capítulo 13.

11.9. Limitaciones de las medidas

- La célula no está calibrada con masas patrón. Todo lo que se dice de la fuerza depende de una escala ajustada, y por eso el criterio de contraste es el par y no la fuerza.
- Las cotas del mecanismo proceden del CAD y de medidas sobre el banco. La más sensible, e_2, se puede fijar midiendo θ_{libre} con un inclinómetro, y conviene hacerlo.
- La escala del par del accionamiento es un ajuste, no una medida. Un ensayo con un par conocido —una masa colgada de un brazo conocido directamente del eje— la fijaría.
- El modelo es estático y los ensayos se hacen a velocidad constante; el tramo de arranque y las inversiones quedan fuera del análisis.
- El perfil del tambor se ha tomado de la sección del CAD, no medido sobre la pieza fabricada. La meseta ajustada (61°) y la del CAD (50°) difieren en una decena de grados, y medir el perfil real cerraría esa diferencia.

11.10. Trabajo pendiente para cerrar la validación

Tres medidas, ninguna costosa, convertirían el acuerdo actual en una validación cerrada:

1. **Calibrar la célula** con masas patrón, en el rango de 0 a 60 newton. Es lo que falta para que la fuerza sea una medida independiente y no una curva escalada.
2. **Medir** θ_{libre} con el cable flojo, y de ahí e_2. Es la cota a la que más responde el modelo y la que mejor explicaría la discrepancia de pendiente.
3. **Levantar el perfil real del tambor**, aunque sea con un calibre y un divisor angular, para contrastar la meseta y la ley de la espiral.

Capítulo 12

Gestión del proyecto

12.1. Ciclo de vida

El proyecto ha seguido un ciclo de vida incremental, con siete iteraciones en las que cada una entrega algo utilizable y se valida antes de pasar a la siguiente. No es una elección de estilo: en un trabajo con tanta incertidumbre sobre el mecanismo y los componentes, cada iteración ha servido para descartar una hipótesis. Las fases de definición y análisis se concentraron en la primera iteración; el diseño, la construcción y las pruebas se repiten en todas.

La validación se ha apoyado sistemáticamente en programas de prueba por subsistema (sección 6.11), de modo que ningún fallo de integración obligara a depurar todo el sistema a la vez.

12.2. Planificación

12.2.1. Planificación real

La planificación se ha reconstruido a partir del historial del repositorio: 36 *commits* entre el 4 de marzo y el 21 de septiembre de 2026, concentrados en nueve semanas (figura 12.1). El historial sólo refleja el trabajo sobre código y diseño electrónico; los periodos sin *commits* corresponden a montaje mecánico, pruebas sin cambios de código y periodo lectivo [**COMPLETAR:** describir las actividades de abril a junio y de julio a agosto].

F1. Exploración del accionamiento (2–10 de marzo)
Comunicación Modbus con una placa XIAO ESP32-S3: escaneo de direcciones, modos par y velocidad, monitor de terminal, parámetros del accionamiento y resistencia de frenado. Culmina en un firmware v1.0 con regulador de fuerza en modo par.

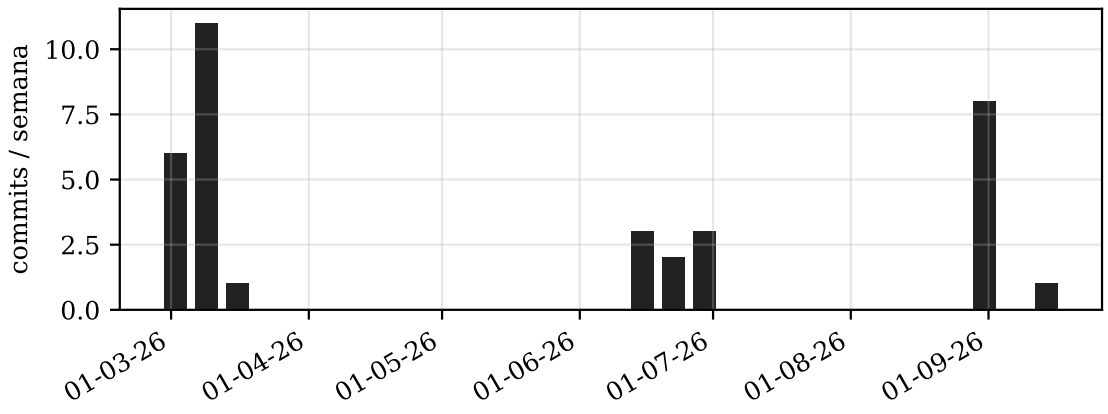


Figura 12.1: *Commits* por semana en el repositorio del proyecto.

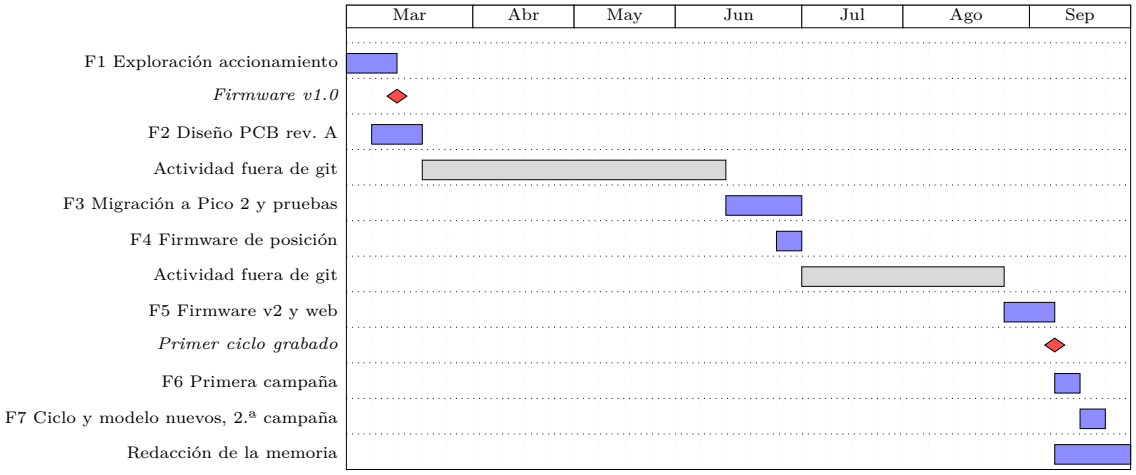


Figura 12.2: Diagrama de Gantt reconstruido (semanas desde el 2 de marzo de 2026).

F2. Diseño de la placa adaptadora (12–17 de marzo)
Esquema y colocación de la revisión A.

F3. Migración a Raspberry Pi Pico 2 y validación de la placa (20–30 de junio)
Programas de prueba de entradas y salidas, células de carga, RS-485 y escaneo Modbus. Versión de doble núcleo.

F4. Firmware de posición en modo velocidad (4 de julio)
Movimiento entre extremos con integración de la posición y monitor asociado.

F5. Firmware v2 y supervisión web (2–6 de septiembre)
Homing con dos finales de carrera, servidor FastAPI, página de supervisión, ciclos automáticos y grabación de ensayos.

F6. Primera campaña de ensayos (14–15 de septiembre)
Quince ensayos con el ciclo A→B→A, visor de ensayos con curva teórica y primer análisis.

F7. Rediseño del ciclo y del modelo, y segunda campaña (21 de septiembre)
Ciclo B→A→B con extremos por finales de carrera, modelo con la salida del cable desplazada, configuración única del mecanismo con simulador, segunda campaña de ensayos y análisis definitivo.

12.3. Presupuesto

12.3.1. Personal

La dedicación de un TFG se fija en créditos ECTS, cada uno de 25 a 30 horas de trabajo [10]. La tabla 12.1 reparte una dedicación estimada por fases [**VERIFICAR:** ajustar las horas y los créditos del TFG en la titulación]. Para valorar el trabajo se toma como supuesto un coste de 30 €/h para un ingeniero o ingeniera junior y de 45 €/h para la tutoría.

Tabla 12.1: Dedicación estimada y coste de personal.

Actividad	Horas	€/h	Coste (€)
Estudio y documentación	30	30	900
Exploración del accionamiento (F1)	35	30	1.050
Diseño de la placa (F2)	45	30	1.350
Pruebas de placa y comunicaciones (F3)	45	30	1.350
Firmware (F4–F5)	70	30	2.100
Software de supervisión (F5, F7)	60	30	1.800
Ensayos y análisis (F6, F7)	40	30	1.200
Redacción de la memoria	50	30	1.500
Subtotal trabajo técnico	375		11.250
Tutoría	25	45	1.125
Total recursos humanos			**12.375**

12.3.2. Material

Tabla 12.2: Material del sistema.

Elemento	Cant.	Precio unit. (€)	Total (€)
Raspberry Pi Pico 2	1	**[COMPLETAR:]**	
Transceptor RS-485 THVD1400	1	**[COMPLETAR:]**	
Interruptor de lado alto TPS4H000-Q1	1	**[COMPLETAR:]**	
Amplificador operacional MCP6294	1	**[COMPLETAR:]**	
Diodos zéner BZX384	4	**[COMPLETAR:]**	
Resistencias y condensadores SMD 0805	25	**[COMPLETAR:]**	
Bornas de tornillo y tiras de pines	–	**[COMPLETAR:]**	
Fabricación de la PCB	1	**[COMPLETAR:]**	
Load Cell 2 Click (ZSC31014)	1	**[COMPLETAR:]**	
Célula de carga	1	**[COMPLETAR:]**	
Servomotor AC de lazo cerrado con accionamiento	1	**[COMPLETAR:]**	
Fuente de alimentación del accionamiento	1	**[COMPLETAR:]**	
Finales de carrera	2	**[COMPLETAR:]**	
Caracola, barra, cable, estructura y masa de 2 kg	–	**[COMPLETAR:]**	
Total material			**[COMPLETAR:]**

12.3.3. Equipos y software

Todo el software empleado es libre o gratuito (KiCad, Arduino IDE con el núcleo Arduino-Pico, Python, FastAPI, Chart.js, git y LaTeX), por lo que su coste es nulo. Los equipos se imputan por amortización lineal,

$$A = C\,\frac{t_{\text{uso}}}{t_{\text{vida}}}, \tag{12.1}$$

con C el coste de adquisición, t_{uso} el tiempo de uso y t_{vida} la vida útil.

Tabla 12.3: Amortización de equipos.

Equipo	Coste (€)	Vida útil (meses)	Uso (meses)	Amortización (€)
Ordenador portátil	**[COMPLETAR:]**	48	7	
Polímetro	**[COMPLETAR:]**	60	7	
Fuente de laboratorio	**[COMPLETAR:]**	60	7	
Total				**[COMPLETAR:]**

12.3.4. Resumen de costes

Tabla 12.4: Presupuesto total.

Concepto	Importe (€)
Recursos humanos	12.375
Material	**[COMPLETAR:]**
Amortización de equipos	**[COMPLETAR:]**
Subtotal	**[COMPLETAR:]**
Costes indirectos (10 %, supuesto)	**[COMPLETAR:]**
Base imponible	**[COMPLETAR:]**
IVA (21 %)	**[COMPLETAR:]**
Total	**[COMPLETAR:]**

12.4. Impacto

12.4.1. Impacto técnico y económico

La consecuencia práctica del capítulo 4 es de dimensionado: para un recorrido dado, un perfil de par constante reduce el par de pico que debe dar el motor. Sobre el recorrido con carga de los ensayos, el perfil ideal rebaja el pico un 11 % respecto a un cilindro equivalente, lo que permitiría un motor y un accionamiento algo menores. A velocidad constante, las pérdidas por efecto Joule son proporcionales a $\int \tau^2 \, \mathrm{d}\varphi$, mínima cuando el par es constante (desigualdad de Cauchy-Schwarz), aunque en este caso la mejora es de apenas un 1 %.

Esas cifras deben leerse junto con el resultado del capítulo 11: el tambor instalado no aplana el par, sino que lo empeora respecto a un cilindro. El margen de mejora que ofrece el rediseño del perfil es, por tanto, bastante mayor que ese 11 %.

La plataforma desarrollada es además reutilizable: la placa, el firmware y la aplicación web forman un banco genérico para cualquier eje accionado por Modbus con medida de fuerza, con un coste de material muy inferior al de un sistema de adquisición comercial.

12.4.2. Objetivos de Desarrollo Sostenible

Tabla 12.5: Relación del trabajo con los Objetivos de Desarrollo Sostenible [36].

ODS	Contribución
4. Educación de calidad	Banco de ensayo abierto y documentado, útil como práctica de laboratorio de mecanismos, accionamientos y adquisición de datos.
7. Energía asequible y no contaminante	Un perfil de par uniforme reduce las pérdidas en el motor.
9. Industria, innovación e infraestructura	Integración de tecnologías industriales (Modbus, RS-485) con herramientas libres de bajo coste.
12. Producción y consumo responsables	Dimensionado de actuadores más pequeños; módulo microcontrolador en zócalo, reutilizable; datos brutos conservados para no repetir ensayos.

12.4.3. Aspectos éticos, legales y de seguridad

El prototipo es un equipo de laboratorio que no se comercializa. Su evolución hacia una máquina de uso por terceros tendría que considerar la Directiva 2006/42/CE de máquinas, que el Reglamento (UE) 2023/1230 sustituye a partir de enero de 2027, la Directiva 2014/30/UE de compatibilidad electromagnética y la Directiva 2011/65/UE sobre sustancias peligrosas (RoHS). La sección 5.5 discute las limitaciones de seguridad del diseño actual frente a ISO 13849-1 [17].

En cuanto a la ética profesional, se ha priorizado la trazabilidad: se conservan las medidas brutas, las figuras se generan con programas reproducibles y las limitaciones se señalan de forma explícita, incluso cuando el resultado contradice la hipótesis de partida.

Capítulo 13

Conclusiones

13.1. Conclusión

El objetivo general era construir y validar un banco de ensayos para un mecanismo de elevación con caracola de radio variable, y usarlo para contrastar el modelo del mecanismo. El banco está construido y funciona; el contraste ha dado un resultado distinto del esperado, y de ahí salen las conclusiones más interesantes del trabajo.

13.1.1. Sobre el banco de ensayos

1. El sistema cumple los requisitos de temporización: muestrea a 20,0 Hz con una desviación típica del periodo de 0,06 ms, y el trabajo del lazo ocupa el 22 % del periodo. Llegar a ese determinismo exigió abandonar la versión de doble núcleo en favor de un lazo de periodo fijo con un único dueño de cada bus.
2. El ciclo de ensayo B→A→B, con los extremos reconocidos por los finales de carrera y no por la posición integrada, repite el recorrido dentro del 0,6 % y cierra sobre el punto de partida. Esa decisión hace que el banco sea utilizable pese a la incertidumbre de la medida de posición.
3. Guardar los ensayos en cuentas brutas, con las constantes en la cabecera, ha permitido reanalizar por completo dos campañas después de cambiar el modelo y la calibración, sin repetir un solo ensayo. Es la decisión de diseño que más ha rendido.
4. Unificar el modelo en una sola implementación compartida por el simulador y el visor eliminó una fuente real de error: las páginas habían divergido hasta calcular con geometrías distintas.

13.1.2. Sobre el modelo del mecanismo

5. La posición de la salida del cable respecto a la articulación no es un detalle de dibujo. Con la salida en la vertical, la tensión resulta proporcional a la longitud libre de cable, un resultado simple y atractivo; con la salida desplazada, esa cancelación desaparece y la tensión pasa a depender de dos ángulos. El modelo del banco trabaja ya con el caso general.
6. El perfil de par constante es el que minimiza el par de pico para un recorrido y un giro de tambor dados. La demostración es independiente de la geometría: el trabajo está fijado, luego el máximo del par no puede ser menor que su media.

13.1.3. Sobre la validación experimental

7. **Medir sólo la fuerza no permite validar el modelo.** Con la célula sin calibrar, el ajuste incluye la sensibilidad como incógnita y el coeficiente de determinación es el cuadrado de la correlación: no distingue el signo ni la amplitud. El par del accionamiento sí decide, porque su escala y su signo son propios del equipo y no dependen de ninguna calibración pendiente. Es el criterio que se ha adoptado.

8. **El tambor actúa, y se mide.** Sobre el tramo con carga el par gravitatorio es prácticamente plano —varía un $-5\,\%$, con un rizado del $12{,}1\,\%$— donde un cilindro equivalente lo haría crecer un $15\,\%$. El brazo efectivo cae un $16\,\%$. El cilindro queda descartado sin ambigüedad ($R^2 = -1{,}78$) y la orientación invertida de la espiral, aún más ($R^2 = -20{,}1$).

9. **El perfil que explica la medida incluye la meseta de la pieza.** La espiral tal y como está descrita en la configuración sobrecompensa ($R^2 = -1{,}23$). Añadiendo el arco de radio constante que la pieza tiene antes de la espiral, y ajustando su longitud como único parámetro, el acuerdo sube a $R^2 = 0{,}84$. El valor ajustado, $61°$, coincide con los $50°$ medidos sobre el CAD: una confirmación independiente, porque el ajuste sólo ha visto pares del accionamiento.

10. **El tambor no ahorra motor en este banco, y eso es el resultado.** El criterio correcto es el par de pico, porque es lo que dimensiona el accionamiento, lo que calienta el motor —las pérdidas van con la corriente al cuadrado— y lo que el lazo de velocidad ve como perturbación. Como todos los tambores que recorren lo mismo en las mismas vueltas entregan el mismo par medio, el ahorro es el cociente de las relaciones pico/media: 1,052 para el par medido frente a 1,094 de la tensión medida o 1,037 de la del modelo. Sale entre $-1{,}5\,\%$ y $4\,\%$. Y el cilindro que iguala el pico de la caracola sólo necesita un $1\,\%$ más de vueltas.

11. **La razón no es que el principio falle, sino que aquí no hay nada que repartir.** La tensión sólo varía un $13\,\%$ (modelo) o un $28\,\%$ (célula) sobre el tramo útil, porque el brazo del peso y el del cable crecen a la vez. Donde la tensión variase el doble o el triple, el mismo tambor recortaría el motor en esa proporción. Perseguir además un perfil de par exactamente constante no compensa, y choca con el hecho de que el cable sólo se asienta sobre perfiles convexos de pendiente suave.

12. Las pérdidas del mecanismo son de tipo Coulomb: constantes a lo largo del recorrido (entre 5,2 y 8,0 unidades de accionamiento) y del orden del $18\,\%$ del par gravitatorio.

13. La escala del accionamiento, obtenida del ajuste, es de $0{,}0553\,\mathrm{N\,m}$ por unidad, y coincide dentro del $1\,\%$ con la que daba un ajuste independiente sobre la campaña anterior. Con ella el par de pico en el eje del motor es de $0{,}45\,\mathrm{N\,m}$, un $35\,\%$ del nominal.

14. Queda una discrepancia abierta: la célula crece un $28\,\%$ donde el modelo predice un $13\,\%$, de forma sistemática en los diez ensayos y también en la campaña anterior. Coinciden en nivel (factor 1,04) pero no en pendiente. Mientras no se calibre la célula con masas patrón y no se mida el ángulo de reposo libre de la barra, de donde sale el parámetro más sensible del modelo, esa diferencia no se puede atribuir. El trabajo deja el procedimiento montado y el diagnóstico hecho.

13.1.4. Grado de cumplimiento de los objetivos

Obj.	Resultado	Estado
O1	Modelo estático con salida tangente y contacto exacto sobre el perfil, verificado contra el balance de energía; perfil de par constante deducido y descartado con criterio (capítulo 4).	Cumplido
O2	Placa adaptadora con RS-485, entradas y salidas y dos zócalos mikroBUS; revisión A analizada y corregida en la revisión B (capítulo 6).	Cumplido; falta versionar la revisión B
O3	Firmware determinista con Modbus, célula, *homing*, ciclo B→A→B y protecciones (capítulo 7).	Cumplido
O4	Protocolo binario con CRC-8, telemetría por lotes y compatibilidad entre versiones (capítulo 8).	Cumplido
O5	Servidor y tres páginas web: monitor, configuración con simulador y visor de ensayos (capítulo 9).	Cumplido

Obj.	Resultado	Estado
O6	Caracterización del banco y contraste cilindro/caracola sobre dos campañas (capítulo 11).	Cumplido; validación abierta a falta de cotas y calibración
O7	Problemas y soluciones, planificación, presupuesto e impacto (capítulos 10 y 12).	Cumplido

13.2. Desarrollos futuros

13.2.1. Para cerrar la validación

- Medir las cotas del mecanismo y el perfil real de la caracola (radios, sentido y barrido), y sustituir con ellas los parámetros ajustados.
- Calibrar la célula de carga con masas patrón y estimar su incertidumbre; con ello, los valores de fuerza pasarían de orientativos a trazables.
- Confirmar la escala del registro de par del accionamiento para poder expresar los pares en N m.
- Eliminar o medir la holgura inicial del cable, para que el recorrido con carga coincida con el recorrido completo.
- Repetir la campaña y rehacer el contraste cilindro frente a caracola, que el banco ya automatiza.

13.2.2. Sobre el mecanismo

- Fabricar, por ejemplo por impresión 3D, un tambor cilíndrico y otro con el perfil ideal calculado en la sección 4, y ensayar los tres en el mismo banco. Sería la comparación definitiva, con el mecanismo como única variable.
- Revisar el sentido de montaje de la caracola actual, que según la medida trabaja al contrario de lo que describe la configuración.
- Ensayar con varias masas para comprobar la linealidad del modelo con la carga.

13.2.3. Sobre el sistema de medida

- Leer la posición del codificador del accionamiento por Modbus en lugar de integrar la velocidad.
- Leer velocidad y par en una sola transacción Modbus (registros 16385 a 16387), lo que ahorraría unos 3 ms por periodo y permitiría subir la frecuencia de muestreo.
- Enviar en cada muestra la posición del firmware y la lectura sin filtrar de la célula, y sincronizar la lectura con la señal de interrupción del acondicionador.
- Corregir la codificación de la ganancia y los bits de estado del ZSC31014 (sección 10.9).

13.2.4. Sobre la seguridad y la explotación

- Cablear los finales de carrera y una seta de emergencia a la entrada STO del accionamiento, de modo que la parada no dependa del software.
- Versionar y fabricar la revisión B de la placa, con los ajustes de resistencias propuestos.
- Hacer que el servidor escuche sólo en la máquina local o exija autenticación, y añadir pruebas automáticas del protocolo.

13.3. Valoración personal

[**COMPLETAR:** Valoración personal: qué se ha aprendido, qué dificultades han sido más formativas y qué se haría de otra manera.]

Apéndice A

Asignación de pines

A.1. Raspberry Pi Pico 2 (revisión B de la placa)

Tabla A.1: Asignación de GPIO según el esquema de la revisión B y el firmware.

GPIO	Pin	Señal	Función
GP0	1	INT1	Interrupción de la célula 1 (MD1)
GP1	2	EN1	Habilitación / reinicio de las placas Click (`LC1_EN_GPIO`)
GP2	4	SDA1	I^2C1 SDA (`LC_SDA_PIN`)
GP3	5	SCL1	I^2C1 SCL (`LC_SCL_PIN`)
GP4	6	INT2	Interrupción de la célula 2 (MD2)
GP5, GP6	7, 9	–	Libres
GP7	10	SIN4	Entrada del canal 4 del TPS4H000
GP8	11	SIN3	Entrada del canal 3
GP9	12	SIN2	Entrada del canal 2
GP10	14	SIN1	Entrada del canal 1
GP11	15	DST	Fallo global del TPS4H000 (drenador abierto, *pull-up*)
GP12	16	DE	Dirección del transceptor RS-485 (`MB_EN_GPIO`)
GP13	17	RO	Recepción RS-485, UART PIO (`MB_RX_GPIO`)
GP14	19	DI	Transmisión RS-485, UART PIO (`MB_TX_GPIO`)
GP15	20	DIAG_EN	Habilitación del diagnóstico del TPS4H000
GP16	21	DIN1	Final de carrera A, arriba (`PIN_LIMIT_A`)
GP17	22	DIN2	Final de carrera B, abajo (`PIN_LIMIT_B`)
GP18	24	DIN3	Entrada digital libre
GP19	25	DIN4	Entrada digital libre
GP20	26	COMP	Salida del comparador (umbral 2,5 V)
GP21, GP22	27, 29	–	Libres
GP26	31	AOUT3	ADC0: entrada analógica AIN3
GP27	32	AOUT1	ADC1: entrada analógica AIN1
GP28	34	AOUT2	ADC2: entrada analógica AIN2
VBUS	40	+5V	Alimentación de los amplificadores
3V3	36	+3.3V	Transceptor RS-485 y placas Click

Nota. El firmware define `LC1_INT_GPIO = 4`, mientras que en el esquema de la revisión B la interrupción de la célula 1 llega a GP0 y GP4 corresponde a la célula 2. El firmware sólo usa

ese pin para llevarlo a nivel bajo durante el apagado de la célula (sección 10.2) [**VERIFICAR:** comprobar a qué GPIO está conectado INT en la placa montada].

A.2. Bornas de campo

Tabla A.2: Bornas de la placa (revisión B).

Borna	Pin	Señal
J5 (RS-485)	1 / 2 / 3	A / B / GND
Terminal de salidas	1–8	SOUT4, SOUT3, –, VS_IN, SOUT2, SOUT1, AIN2, AIN1
Terminal de entradas	1–8	AIN3, AIN4, VS_IN, –, DI1, DI2, DI3, DI4
Filas de masa	1–8	GND

[**VERIFICAR:** comprobar el orden de las bornas sobre la placa fabricada]

Apéndice B

Especificación del protocolo binario

Formato común: `0xAA 0x55 LEN carga_útil[LEN] CRC8`. Enteros en *little-endian*. CRC-8 con polinomio 0x07 y valor inicial 0x00 sobre la carga útil. Los desplazamientos se dan en bytes dentro de la carga útil.

B.1. Paquete 0x02: telemetría por lotes

Tabla B.1: Cabecera del lote (14 bytes).

Despl.	Campo	Tipo	Descripción
0	packet_id	u8	0x02
1	base_t	u32	`millis()` de la primera muestra del lote
5	bridge	u16	Lectura bruta del ZSC31014 (14 bits)
7	lc_status	u8	Bits de estado de la última lectura
8	flags	u8	b0: célula válida; b1: servo conectado; b2: configuración aplicada
9	error	u8	Código de error (tabla B.4)
10	servo_id	u8	Dirección Modbus en uso
11	rpm_status	u8	Resultado de la lectura de velocidad (tabla B.5)
12	trq_status	u8	Resultado de la lectura de par
13	n	u8	Número de muestras (1–8)

Tabla B.2: Muestra (14 bytes), repetida n veces.

Despl.	Campo	Tipo	Descripción
+0	dt_ms	u16	Milisegundos desde base_t
+2	rpm	i16	Velocidad del motor (registro 16385)
+4	current_x10	i16	Par (registro 16387), décimas de u. a.
+6	ref_cmd	i16	Consigna de velocidad escrita (rpm); negativa hacia A
+8	base_read	i16	Lectura filtrada menos tara (cuentas)
+10	io	u8	b0: final A (arriba); b1: final B (abajo)
+11	state	u8	Estado de la máquina (tabla B.3)
+12	work_us	u16	Tiempo de trabajo de la vuelta (µs, saturado a 65535)

B.2. Paquete 0x03: estado de *homing* (11 bytes)

0	packet_id	u8	0x03
1	pos_mrev	i32	Posición × 1000
5	range_mrev	i32	Recorrido A→B medido × 1000
9	home_phase	u8	0 reposo, 1 buscando A, 2 buscando B, 3 yendo al origen, 4 completado, 5 fallo
10	flags	u8	b0: recorrido válido

B.3. Paquete 0x04: resultado de la célula (13 bytes)

0	packet_id	u8	0x04
1	req	u8	Orden respondida (0x09, 0x0A o 0x0B)
2	result	u8	Resultado (tabla B.6)
3	gain	u8	Ganancia grabada (código 0–7, no lineal: sección 10.9)
4	offset	u8	Cero del A/D grabado (0–15)
5	cfg_before	u16	B_Config antes
7	cfg_after	u16	B_Config después
9	ack	u8	Primer byte devuelto por el chip
10	diag	u8	b0: el reinicio corta; b1: hubo ACK a Start_CM; b7–b4: intento del barrido
11	cfg1	u16	ZMDI_Config1

B.4. Paquete 0x05: estado del ciclo (18 bytes)

0	packet_id	u8	0x05
1	phase	u8	0 reposo, 1 colocándose en B, 2 hacia A, 3 volviendo a B, 4 completado, 5 abortado
2	done	u16	Repeticiones completadas
4	target	u16	Repeticiones pedidas (0 = sin fin)
6	pos_mrev	i32	Posición actual × 1000
10	target_mrev	i32	Recorrido B→A pedido × 1000
14	pos_b_mrev	i32	Posición en la que ha quedado fijado B × 1000

La versión anterior de este paquete ocupaba 14 bytes y no llevaba el último campo. El servidor acepta las dos longitudes: con la trama corta asume `pos_b_mrev = 0`, de modo que un firmware y un servidor desparejados siguen informando de la fase (sección 10.4).

B.5. Órdenes

Orden	Carga útil	Notas
INIT	`01 [addr]`	addr opcional, 1–247
STOP	`02`	
SET_PARAM	`03 id vv vv`	valor i16
MOVE_A / MOVE_B	`04` / `05`	A es arriba; B, abajo
SHUTDOWN	`06`	
SCAN	`07`	bloqueante
HOME	`08`	
LC_PROGRAM	`09 bb bb cc cc`	B_Config u16, ZMDI_Config1 u16
LC_QUERY	`0A`	
LC_HOLD	`0B park`	0: alta impedancia; 1: líneas a masa
CYCLE_START	`0C mm tt tt vv vv rr rr`	mm = 0: hasta el final de carrera; 1: hasta la distancia indicada. Recorrido i16 (centi-rev), velocidad i16, repeticiones u16
CYCLE_STOP	`0D`	

Atajos de terminal (sólo con el decodificador en reposo): `i` INIT, `a` MOVE_A, `b` MOVE_B, `x` STOP, `h` HOME, `q` SHUTDOWN.

B.6. Tablas de códigos

Tabla B.3: Estados de la máquina.

0	IDLE (servo sin habilitar)
1	SCANNING
2	CONFIGURING
3	STOPPED (habilitado, consigna 0)
4	MOVING_A
5	MOVING_B
6	ERROR
7	HOMING
8	PROGRAMMING (célula)

Tabla B.4: Códigos de error.

0x00	Sin error
0x01	Célula no encontrada
0x02	Fallo en el escaneo del servo
0x03	Fallo al configurar el servo
0x04	Límite de fuerza

Tabla B.5: Resultados de ModbusMaster.

0x00	Correcto
0x01–0x04	Excepción del esclavo
0xE0	Dirección de esclavo incorrecta
0xE1	Función incorrecta
0xE2	Tiempo de espera agotado
0xE3	CRC incorrecto

Tabla B.6: Resultados de las órdenes de la célula.

0	Nunca se ha intentado
1	Grabado y verificado tras un ciclo de alimentación
2	Ya estaba así: no se ha escrito la EEPROM
3	Rechazado: el eje no está parado
4	Rechazado: bus ocupado
5	El chip se reinicia pero no entra en modo comando
6	Escrito, pero la relectura no coincide
7	Sólo lectura de la configuración
8	Sigue alimentada con el reinicio activo: hay otro camino de alimentación
9	La célula no contesta: sin tensión o sin bus
10	Reinicio mantenido para medir

Apéndice C

Registros Modbus del accionamiento

Dirección = 256·grupo + índice (índice en hexadecimal). Esclavo 1, 115200 baudios, 8N1. [**VERIFICAR:** contrastar denominaciones y unidades con el manual del accionamiento]

Tabla C.1: Registros utilizados por el firmware.

Dirección	Parámetro	Acceso	Descripción	Valor escrito
0	C00.00	E	Modo de control (1 = velocidad, 2 = par)	1
5	C00.05	E	Nivel de rigidez	10
16	C00.10	E	Selección de resistencia de frenado (1 = externa)	1
17	C00.11	E	Potencia de la resistencia de frenado	1000
18	C00.12	E	Valor óhmico de la resistencia de frenado	235
19	C00.13	E	Disipación de la resistencia de frenado	30
801	C03.21	E	Consigna de velocidad (rpm, con signo: negativa hacia A)	cada periodo
802	C03.22	E	Aceleración	5
804	C03.24	E	Deceleración	5
1041	C04.11	E	Habilitación del servo (0/1)	$0 \rightarrow 1$
16385	0x4001	L	Velocidad del motor (rpm)	–
16387	0x4003	L	Par del motor (décimas de u. a.)	–

Registros usados en versiones anteriores, con el accionamiento en modo par: 833 (consigna de par, C03.41), 839 y 840 (límites de par), 1542 (C06.06, apantallamiento de STO) y 1576 (C06.28, retardo de desconexión tras STO).

C.1. Secuencia de configuración

Al recibir la orden INIT, el firmware escribe en este orden: C04.11 = 0 (deshabilitar), C00.00 = 1 (modo velocidad), C03.21 = 0 (consigna nula), C00.05 (rigidez), C03.22 y C03.24 (rampas), C00.10 a C00.13 (resistencia de frenado) y C04.11 = 1 (habilitar). Entre escrituras se deja un silencio de 700 µs. Si falla una escritura esencial se repite la secuencia completa, y tras tres intentos el sistema pasa a estado de error con el código 0x03.

Apéndice D

Formato de los ficheros de ensayo

Cada ensayo se graba en `cycles/AAAAMMDD-HHMMSS_ciclo.csv`, codificado en UTF-8. Las líneas que empiezan por `#` forman la cabecera, con las constantes en pares `clave=valor`; después viene una línea de nombres de columna y una fila por muestra.

```
# ensayo 2026-09-21 18:08:46
# lc1_cuentas_por_N=14.41  lc1_cero_N=51.78  lc1_int16=0
# repeticiones_pedidas=1  B=1.000 rev (posicion)
# lc1_base ya lleva restada la tara del firmware; lc1_raw es cruda
t_ms,t_rel_s,rep,fase,pos_rev,lc1_raw,lc1_base,rpm,par_x10,ref_rpm,lim_a,lim_b,estado,work_us
```

Código D.1: Comienzo de un fichero de la campaña del 21 de septiembre.

Tabla D.1: Columnas del fichero de ensayo.

Columna	Unidad	Descripción
t_ms	ms	Reloj del microcontrolador
t_rel_s	s	Tiempo desde la primera muestra del ensayo
rep	–	Repetición en curso (desde 1)
fase	–	`hacia_A` (subida) o `volviendo_a_B` (bajada). En los ensayos de la campaña preliminar, `hacia_B` y `volviendo_a_A`
pos_rev	rev	Posición del eje del motor integrada por el servidor
lc1_raw	cuentas	Lectura bruta del ZSC31014 (una por lote)
lc1_base	cuentas	Media móvil de 8 lecturas menos la tara del firmware
rpm	rpm	Velocidad medida por el accionamiento
par_x10	0,1 u. a.	Par medido por el accionamiento
ref_rpm	rpm	Consigna de velocidad (negativa hacia A, positiva hacia B)
lim_a, lim_b	0/1	Estado de los finales de carrera
estado	–	Estado de la máquina (tabla B.3)
work_us	µs	Tiempo de trabajo de la vuelta del lazo

D.1. Conversión y análisis

La conversión a fuerza es $F\,[\mathrm{N}] = \mathrm{lc1_base}/k + F_0$, con $k =$ `lc1_cuentas_por_N` y $F_0 =$ `lc1_cero_N`. Conviene recordar que esa constante es provisional (sección 11.9).

Para el análisis, el *avance del ciclo* se obtiene restando la posición de la primera muestra, que es el punto B donde arranca el ensayo, y corrigiendo el signo con el de la muestra más alejada:

$$p_i = \left(\mathrm{pos_rev}_i - \mathrm{pos_rev}_0\right) \cdot \mathrm{signo}\left(\mathrm{pos_rev}_{\mathrm{lejana}} - \mathrm{pos_rev}_0\right). \tag{D.1}$$

Así el avance es siempre positivo hacia A, y los ensayos de las dos campañas se pueden comparar pese a tener sentidos de recorrido opuestos.

D.2. Notas sobre las cabeceras

- Los ensayos de la campaña preliminar se grabaron con una versión del servidor que todavía no escribía la calibración: en ellos aparece `lc1_cuentas_por_N=0`.
- El texto "(posicion)" de la tercera línea se escribe siempre que el recorrido pedido sea mayor que cero, aunque el ensayo se haya hecho contra los finales de carrera, que es el caso de todos los ensayos de esta memoria. Es un defecto conocido de la cabecera y no afecta a los datos.

Apéndice E

Manual de uso

E.1. Instalación

E.1.1. Firmware

1. Instalar Arduino IDE y añadir en *Preferencias* el gestor de placas de Arduino-Pico: `https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json`
2. Instalar el paquete *Raspberry Pi Pico/RP2040/RP2350* y la biblioteca *ModbusMaster* desde el gestor de bibliotecas.
3. Abrir `firmware_posicion_v2/firmware_posicion_v2.ino` y seleccionar la placa *Raspberry Pi Pico 2*.
4. Conectar la Pico con el botón BOOTSEL pulsado y cargar el programa.

E.1.2. Aplicación de supervisión

```
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi "uvicorn[standard]" pyserial
python web_monitor.py            # o: python web_monitor.py /dev/ttyACM0
```

En Linux, la cuenta de usuario debe pertenecer al grupo `dialout` para acceder al puerto. La interfaz se abre en `http://localhost:8080`, con tres páginas enlazadas por la barra inferior: *monitor* (`/`), *ensayos* (`/ensayos`) y *mecanismo* (`/sim`).

E.2. Configuración del mecanismo

Antes de ensayar conviene revisar la geometría en la página *mecanismo*: tipo de tambor (caracola, cilindro o leva), radios y barrido, reducción, sentido, cotas L_2, x_t, L_3, e_1, L_4, e_2, masa y ángulo θ_0 de la barra en el punto B. Los cambios no se aplican hasta pulsar *Guardar y propagar*; desde ese momento los usan tanto el simulador como el visor de ensayos.

La página avisa si la geometría es imposible (por ejemplo, si el recorrido pedido obligaría a la barra a pasar de sus topes) e indica, mediante la inversa, cuántas revoluciones de motor hacen falta para llevar la barra a un ángulo concreto: es el recorrido que tendría que hacer el ciclo.

E.3. Procedimiento de ensayo

1. **Comprobaciones previas**: masa colgada, cable sin cocas y correctamente asentado en la caracola, finales de carrera en reposo (indicadores apagados) y zona de movimiento despejada.

2. **Inicializar**: pestaña *Servo*, dirección 1, botón *Init*. El estado pasa a STOPPED y el indicador del servo a conectado.
3. **Homing**: fijar la velocidad de *homing* y el desplazamiento del origen, pulsar *Aplicar parámetros* y después *Home*. Al terminar se muestra el recorrido A→B. El eje queda en A, arriba.
4. **Tara**: con el mecanismo en reposo, pulsar *Tara* para poner a cero la célula.
5. **Ciclo**: en la pestaña *Ciclo B→A→B*, elegir hasta dónde sube (final de carrera A, recomendado, o una distancia desde B), la velocidad y las repeticiones, y pulsar *Iniciar ciclo*. El eje baja primero a B para colocarse —ese tramo no se graba— y a partir de ahí empieza el ensayo. La grabación se abre y se cierra sola.
6. **Análisis**: en la página *ensayos*, abrir el fichero y activar la curva teórica para compararla con la medida.
7. **Fin**: *Shutdown* para deshabilitar el accionamiento.

E.4. Programación de la célula de carga

Con el eje parado, abrir *Configuración avanzada* en la pestaña de la célula, elegir la ganancia y el cero del A/D y pulsar *Programar*. El eje queda bloqueado cerca de un segundo. El resultado aparece debajo; si es "sigue alimentada con el reinicio activo", usar *Mantener reset* y medir la tensión de alimentación de la placa Click, que debe caer a 0 V. Téngase en cuenta que el código de ganancia de la interfaz no coincide con la ganancia real para los valores intermedios (sección 10.9).

E.5. Diagnóstico

Síntoma	Causa probable y acción
Servo desconectado, lecturas con código 0xE2	Tiempo de espera Modbus: revisar el cableado A/B, la masa común, la dirección y la velocidad del accionamiento. Probar con `test_modbus_scan`.
Código 0xE3	CRC incorrecto: ruido o polaridad A/B invertida.
Estado ERROR con código 0x03	Tres fallos de configuración: el accionamiento no acepta las escrituras (¿alimentado?, ¿alarma activa?).
Final de carrera siempre activo	Contacto abierto o cable roto (entrada NC).
El ciclo termina enseguida y no graba nada	El eje ya estaba en el extremo o la fase de colocación no llegó a B: revisar en `web_monitor.log` las líneas de cambio de fase.
La interfaz no muestra la fase del ciclo	Firmware y servidor desparejados: recargar el firmware o actualizar el servidor.
Célula no válida	Sin lecturas frescas en 150 ms: revisar la placa Click y el bus I^2C con `test_loadcell`.
La curva teórica no se parece a la medida	Revisar la geometría en *mecanismo*: el aviso de escala y el de ángulo indican qué parámetro no cuadra.

E.6. Precauciones

- Las protecciones son de software: mantener las manos fuera del recorrido con el servo habilitado y tener a mano el corte de alimentación del accionamiento.
- No programar la célula ni usar *Mantener reset* con carga en movimiento (el firmware lo impide, pero la medida de fuerza queda interrumpida).
- El servidor acepta órdenes de cualquier equipo de la red: usarlo en una red de confianza.

Apéndice F

Estructura del repositorio

Ruta	Contenido
Firmware	
`firmware_posicion_v2/`	Firmware final (Pico 2): Modbus, célula, *homing*, ciclo B→A→B y protocolo. `i2c_bus.h`: árbitro del bus I^2C.
`firmware_posicion/`	Versión anterior en modo velocidad entre extremos.
`firmwarerasp/`	Primera versión para Pico 2, con doble núcleo.
`firmware/`	Versión para XIAO ESP32-S3 con regulador de fuerza en modo par.
Programas de prueba	
`pcbtest/`	Entradas analógicas, entradas digitales y salidas de potencia.
`test_limit_switches/`	Finales de carrera.
`test_rs485*/`, `test_modbus_scan/`	Bucle local PIO, transmisión, recepción y comunicación Modbus.
`test_loadcell*/`	ZSC31014 y lectura dual con NAU7802.
`test_servo_modbus.py`	Barrido Modbus desde el PC con adaptador USB-RS485.
Ordenador	
`web_monitor.py`	Servidor FastAPI: puente serie-WebSocket, grabación y configuración del mecanismo.
`mecanismo.json`	Geometría del mecanismo (la escribe el servidor).
`static/index.html`	Monitor en tiempo real.
`static/ensayos.html`	Visor de ensayos con curva teórica y ajuste.
`static/sim.html`	Configuración del mecanismo y simulador.
`static/mecanismo.js`	Modelo del mecanismo compartido por las páginas.
`static/tema.css`, `tema.js`, `nav.js`	Tema visual, colores de las gráficas y navegación común.
`static/fui/`	Kit de interfaz externo (copia sin modificar).
`leer_ciclo.py`	Resumen de un ensayo en consola.
`monitor.py`, `monitor_posicion.py`	Monitores de terminal de versiones anteriores.
`cycles/`	Ficheros de ensayo (CSV).
Hardware y documentación	
`tfg_adaptador/`	Proyecto KiCad de la placa (revisión A) y copias de seguridad.
`image (1).png`	Esquema de la revisión B.
`README.MD`	Notas sobre la UART de la Pico y el doble núcleo.
`memoria/`	Esta memoria: `TFG.tex`, `capitulos/`, `figuras/` y `bibliografia/`.

Ruta	Contenido
`figuras/modelo_mecanismo.py`	Modelo del capítulo 4: port de `static/mecanismo.js`. Ejecutarlo comprueba el balance de energía.
`figuras/generar_campana.py`	Figuras y cifras de una campaña de ensayos. Por defecto, los diez ensayos más recientes.
`figuras/generar_overleaf.py`	Genera la versión ligera para Overleaf.
`figuras/generar_figuras.py`	Campaña preliminar. Usa el modelo antiguo: sus cifras no se citan.
`memoria/_v1/`	Versión anterior de la memoria, antes de adoptar la plantilla de la ETSIDI.
`old/`	Pruebas iniciales en modo par.

F.1. Reproducción de los resultados

Con el repositorio y Python con `numpy`, `scipy` y `matplotlib` instalados:

```
python memoria/figuras/modelo_mecanismo.py    # valida el modelo (balance de energía)
python memoria/figuras/generar_campana.py     # los 10 ensayos mas recientes
```

El segundo admite `--ultimos N` (cuántos ensayos recientes tomar, por defecto 10), `--patron` (un glob sobre `cycles/`, para analizar una campaña concreta) y `--prefijo` (con qué nombre escribir las figuras). Con los valores por defecto escribe `n14_*.pdf` y `datos14.json`, que son las cifras que cita el capítulo 11. Analizar una campaña anterior para comparar es, por ejemplo, `--patron '20261007-*.csv' --prefijo n07`.

`modelo_mecanismo.py` es un port fiel de `static/mecanismo.js`, de modo que la memoria, el simulador y el visor de ensayos calculan lo mismo. Si se cambia la geometría hay que cambiarla en el JS, que es la fuente, y volver a ejecutar el port para comprobar que sigue cuadrando el balance de energía. El diseño del perfil de leva de par constante existe sólo en el JS y no se ha portado: el capítulo 4 explica por qué no se recomienda.

Bibliografía

[1] M. Arsenault. Design of convex variable radius drum mechanisms. *Mechanism and Machine Theory*, 129:175–190, 2018.

[2] O. Baser and E. I. Konukseven. Theoretical and experimental determination of capstan drive slip error. *Mechanism and Machine Theory*, 45(6):815–827, 2010.

[3] J. Bieber, D. Bernstein, and M. Beitelschmidt. Variable radius drum design for cable-driven parallel robots based on maximum load profile. In *Cable-Driven Parallel Robots (CableCon 2023), Mechanisms and Machine Science*, volume 132, Cham, 2023. Springer.

[4] Chart.js contributors. Chart.js.

[5] P. Davies, M. François, N. Lacotte, T. D. Vu, and D. Durville. An empirical model to predict the lifetime of braided HMPE handling ropes under cyclic bend over sheave (CBOS) loading. *Ocean Engineering*, 97:74–81, 2015.

[6] G. Endo, H. Yamada, A. Yajima, M. Ogata, and S. Hirose. A passive weight compensation mechanism with a non-circular pulley and a spring. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2010.

[7] I. Fette and A. Melnikov. The websocket protocol, rfc 6455. IETF, 2011.

[8] K. Feyrer. *Wire Ropes: Tension, Endurance, Reliability*. Springer, Berlin, 2007.

[9] F. Gao, Y. Liu, and W.-H. Liao. Cam profile generation for cam-spring mechanism with desired torque. *Journal of Mechanisms and Robotics*, 10(4), 2018.

[10] Gobierno de España. Real decreto 1125/2003, por el que se establece el sistema europeo de créditos y el sistema de calificaciones en las titulaciones universitarias. BOE n.º 224, 2003.

[11] Just L. Herder. *Energy-free systems: theory, conception and design of statically balanced spring mechanisms*. PhD thesis, Delft University of Technology, 2001.

[12] K. Hoffmann. *An Introduction to Stress Analysis and Transducer Design using Strain Gauges*. HBM, Darmstadt, 2012.

[13] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[14] J. Hwangbo, V. Tsounis, H. Kolvenbach, and M. Hutter. Cable-driven actuation for highly dynamic robotic systems. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8543–8550, 2018.

[15] IEC. Iec 61800-5-2: Adjustable speed electrical power drive systems — part 5-2: Safety requirements — functional, 2016.

[16] E. F. Philhower III et al. Arduino-pico: Raspberry pi pico arduino core for rp2040 and rp2350 boards.

[17] ISO. Iso 13849-1: Safety of machinery — safety-related parts of control systems — part 1: General principles for design, 2023.

[18] G. L. H. Johnston, A. L. Orekhov, and N. Simaan. Design considerations and robustness to parameter uncertainty in wire-wrapped cam mechanisms. *Journal of Mechanisms and Robotics*, 16(2), 2023.

[19] J. H. Jung, N. Pan, and T. J. Kang. Capstan equation including bending rigidity and non-linear frictional behavior. *Mechanism and Machine Theory*, 43(6):661–675, 2008.

[20] T. Kenjo and A. Sugawara. *Stepping Motors and Their Microprocessor Controls*. Oxford University Press, 2nd edition, 1994.

[21] KiCad Project. Kicad eda 9.0.

[22] David S. Landes. *Revolution in Time: Clocks and the Making of the Modern World*. Harvard University Press, Cambridge, MA, 1983.

[23] E. Levin. Friction experiments with a capstan. *American Journal of Physics*, 59(1):80–84, 1991.

[24] C. Liechti. pyserial.

[25] F. L. Litvin, I. Gonzalez-Perez, A. Fuentes, and K. Hayasaka. Design and investigation of gear drives with non-circular gears applied for speed variation and generation of functions. *Computer Methods in Applied Mechanics and Engineering*, 197(45–48):3783–3802, 2008.

[26] H. A. Lloyd. The origin of the fusee. *The Antiquaries Journal*, 31(3–4):188–191, 1951.

[27] D. Ludovico, P. Guardiani, F. Lasagni, J. Lee, F. Cannella, and D. G. Caldwell. Design of non-circular pulleys for torque generation: A convex optimisation approach. *IEEE Robotics and Automation Letters*, 6(2):958–965, 2021.

[28] A. Mazumdar, S. J. Spencer, C. Hobart, J. Dabling, T. Blada, K. Dullea, M. Kuehl, and S. P. Buerger. Synthetic fiber capstan drives for highly efficient, torque controlled, robotic applications. *IEEE Robotics and Automation Letters*, 2(2):554–561, 2017.

[29] H. A. McKenna, J. W. S. Hearle, and N. O'Hear. *Handbook of Fibre Rope Technology*. Woodhead Publishing, Cambridge, 2004.

[30] Microchip Technology. *MCP6291/1R/2/3/4/5: 1.0 mA, 10 MHz rail-to-rail op amps — Datasheet*.

[31] MikroElektronika. Load cell 2 click — documentación del producto.

[32] MikroElektronika. mikrobus standard specification, v2.00.

[33] Modbus Organization. Modbus over serial line: Specification and implementation guide v1.02, 2006.

[34] Modbus Organization. Modbus application protocol specification v1.1b3, 2012.

[35] A. Musa. Capstan drive: a high precision speed reducer using rope. `https://www.aaedmusa.com/projects/capstandrive`, 2024. Proyecto en linea; codigo y modelos en `https://github.com/aaedmusa/Capstan-Drive`. Consultado el 30 de septiembre de 2026.

[36] Naciones Unidas. Transformar nuestro mundo: la agenda 2030 para el desarrollo sostenible. resolución a/res/70/1, 2015.

[37] Nexperia. *BZX384 series: Voltage regulator diodes — Datasheet*.

[38] Nuvoton Technology. *NAU7802: 24-bit dual-channel ADC for bridge sensors — Datasheet*.

[39] NXP Semiconductors. *UM10204: I2C-bus specification and user manual, rev. 7.0*, 2021.

[40] R. Pallàs-Areny and J. G. Webster. *Sensors and Signal Conditioning*. Wiley, Nueva York, 2nd edition, 2001.

[41] S. Ramírez. Fastapi.

[42] Raspberry Pi Ltd. *RP2350 Datasheet y Raspberry Pi Pico 2 product brief.*

[43] Renesas Electronics. *ZSC31014 RBicLite Digital Output Sensor Signal Conditioner — Datasheet*, 2016.

[44] S. Seriani and P. Gallina. Variable radius drum mechanisms. *Journal of Mechanisms and Robotics*, 8(2):021016, 2016.

[45] T. Shen and K. Yano. An innovative spiral pulley that optimizes cable tension variation for superior balancing performance. *Journal of Robotics and Mechatronics*, 34(3):599–606, 2022.

[46] Telecommunications Industry Association. Tia/eia-485-a: Electrical characteristics of generators and receivers for use in balanced digital multipoint systems, 1998.

[47] Texas Instruments. *THVD1400: 3.3-V to 5-V RS-485 transceiver with IEC ESD protection — Datasheet.*

[48] Texas Instruments. *TPS4H000-Q1: 40-V, 1000-mΩ 4-channel smart high-side switch — Datasheet.*

[49] M. P. Vlasblom and R. L. M. Bosman. Predicting the creep lifetime of HMPE mooring rope applications. In *Proc. OCEANS 2006*, pages 1–10, 2006.

[50] D. Walker. Modbusmaster: Enlighten your arduino to be a modbus master.

[51] J. D. Weaver. The theory of the fusee. *Journal of Physics E: Scientific Instruments*, 13(4):396–402, 1980.

[52] J. Werkmeister and A. Slocum. Theoretical and experimental determination of capstan drive stiffness. *Precision Engineering*, 31(1):55–67, 2007.